

无创产前检测中胎儿染色体异常的智能识别与检测时点优化

摘要

随着产前检测技术的发展，无创产前检测（NIPT）在胎儿早期健康状况的监测中应用越来越广泛。为了提高无创产前检测的准确性，提早发现不健康的胎儿，降低不健康胎儿治疗窗口期缩短的风险，本文基于题目所给数据，建立数学模型深入分析。

针对问题一，首先通过标准化孕周计算、编码妊娠方式与染色体异常信息，并处理主观误差与缺失值，对变量进行特征清洗与结构转换。通过 **Shapiro-Wilk 检验** 可知，男胎的 Y 染色体浓度不符合正态分布，对于数值变量，采用 **Spearman 相关分析** 出 Y 染色体浓度与孕周、BMI 的中弱关联关系，与 X 染色体浓度的强关联关系；对于分类变量，通过 **单因素方差分析** 得出 Y 染色体浓度与胎儿健康、IVF 妊娠均有显著影响。随后使用普通最小二乘法构建 **多元线性回归模型**，诊断结果显示存在误差非正态、异方差及共线性问题，导致模型拟合较差 ($R^2 = 0.222$)。最后通过十折交叉验证选取 **岭回归** 中的最优正则强度，显著提升模型预测性能 ($R^2 = 0.964$)，结果表明 X 染色体浓度及其相关 Z 值等测序信号变量是 Y 染色体浓度的主要驱动因子，具体详见图 9 与图 10。

针对问题二，以潜在风险最小化为目标，在问题一的数据预处理之上新增构造了孕期风险等级、最早达标时间、当前检测达标状态及是否曾经达标等关键变量，采用 **DBSCAN 聚类**、**层次聚类** 与 **风险最优化分箱** 这三种聚类方法对男胎孕妇 BMI 进行分组，再基于 **区间重叠修正算法** 进行分组改进，为每组构建最优 NIPT 检测时点优化模型进行求解，最后用 **蒙特卡洛模拟** 方法进行误差分析。通过比较三种聚类结果，得出层次聚类的模型最佳并通过 **有界标量优化算法** 确定了最优三分组方案，求解出 **组 1 (BMI 20.70-32.93)** 建议检测时点为 17.45 周，**组 2 (BMI 32.93-39.70)** 为 17.64 周，**组 3 (BMI 39.70-44.98)** 为 22.36 周，且在误差分析中三组均为低敏感性，误差波动范围较小。

针对问题三，在问题二基础上引入身高、体重、年龄等多因素，通过数据聚合策略和衍生特征构建了特征工程，扩展了原始维度。采用随机森林评估各特征对“最早达标时间”的特征重要性，再结合 **K-means**、**层次聚类** 等聚类算法进行多因素聚类分组求解，并综合考虑检测时机风险和检测失败风险这两个优化目标，为每组建立了 NIPT 检测时点优化模型。最后通过 **蒙特卡罗误差传播分析** 验证策略稳健性，最终确定双分组方案：**组 1 (BMI 20.70-36.75)** 最优检测时点为 25.1 周，**组 2 (BMI 36.75-39.16)** 为 24.2 周。

针对问题四，首先把“染色体的非整倍体”是否异常作为女胎是否健康的判定标准，将判定结果作为因变量，构造包含染色体 Z 值、测序质量指标等 27 项特征工程，并将 **逻辑回归**、**随机森林**、**SVM** 等多种机器学习算法结合三种标准化方法进行对比。结果显示，**逻辑回归结合稳健标准化方法** 表现最优，AUC 达到 0.8782，其余结果详见表 7。

关键字：spearman 相关性分析 岭回归 层次聚类 多目标优化模型 逻辑回归 蒙特卡罗模拟

一、问题重述

1.1 问题背景

在早期胎儿健康状况的监测中，有一种产前检测技术，叫做无创产前检测，又叫做 NIPT^{1,2}。它主要通过收集母体血液、检验胎儿的游离 DNA 片段、判断在胎儿的染色体中是否有异常存在来进行检测，来确保早期胎儿的健康状况。无创产前检测的精确度的主要影响因素为胎儿性染色体的浓度。在孕期 10-25 周时胎儿性染色体浓度可以被检测出来，男胎的 Y 染色体浓度大于或等于 4% 或者女胎的 X 染色体浓度在正常范围内，则说明 NIPT 的检测结果大致精准，否则可能结果会有偏差。同时，不健康胎儿发现的时期也是影响治疗窗口期风险的重要因素。

既往实践表明，男胎 Y 染色体浓度与孕周，孕妇个体情况等因素有显著相关，其中 BMI 是影响达标最早时间的关键因素之一。若对所有孕妇采用统一的经验分组和固定检测时点，可能导致部分人群检测信号不足或延迟发现异常，会增加治疗窗口缩短的风险。因此，需要依据 BMI 对孕妇进行更为合理的分组，并为各组设计“最佳 NIPT 时点”，以提高检测准确性的同时，最小化因达标延迟带来的风险。

1.2 问题要求

附件中给出了某地区孕妇的 NIPT 数据，涵盖受试者年龄、身高、体重、末次月经时间等信息，且存在重复采血与测序失败记录。为据此制定更早且可靠的检测时点与异常判定策略，提出如下问题：

问题一基于附件中的男胎数据，分析男胎 Y 浓度与孕周、BMI 等变量的关系，建立可解释模型并进行参数显著性与拟合优度检验。

问题二通过资料显示胎儿 Y 染色体浓度的最早达标时间主要与男胎孕妇的 BMI 有关，故需要以孕妇可能的潜在风险最小为目标，对男胎孕妇的 BMI 进行适当的划分，给出各组的最佳 NIPT 孕周，并开展误差敏感性分析。

问题三在问题 2 基础上把身高、体重、年龄等多种因素考虑在内，引入检测误差和胎儿 Y 染色体浓度达标比例 ($\geq 4\%$ 的比例) 这个变量，根据男胎孕妇的 BMI 进行合理分组，给出每组的最佳 NIPT 检测时点，最小化孕妇潜在风险最，并分析检测误差对结果的影响。

问题四在孕妇和女胎都没有 y 染色体的情况下，主要考虑女胎的异常情况判断。主要根据女胎孕妇的 21 号、18 号和 13 号染色体非整倍体 (AB 列) 作为判定标准，分析 X 染色体及上述染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素，建立女胎异常的判定方法。

二、问题分析

2.1 问题一分析

对于问题一，核心思路是围绕胎儿 Y 染色体浓度这一连续变量，分层推进变量构建、相关性识别与回归建模。首先对原始产前检测数据进行系统预处理，通过规则解析、字段拆解和逻辑映射生成结构化特征，包括孕期时间、妊娠方式、健康状态等变量。其次，采用 Spearman 秩相关系数与单因素方差分析，挖掘 Y 浓度与连续/分类变量之间的关系，初步筛选具有显著性和解释潜力的候选因子。随后进入建模阶段，构建多元线性回归与 Ridge 正则回归模型，通过交互项、浓度比值等增强特征捕捉变量间非线性与耦合关系，并结合特征标准化、VIF 控制等手段提升模型稳健性。

2.2 问题二分析

在问题二中，核心思路是围绕孕妇 BMI 与 Y 染色体浓度达标时间之间的关系，构建分组策略并据此优化 NIPT 检测时点，以实现检测成功率最大化与风险最小化的平衡。首先在问题一的基础上扩展数据，通过构造如“最早达标时间”“当前是否达标”等衍生变量，将连续检测转化为可用于策略判定的指标体系。随后引入 DBSCAN、层次聚类、GMM 聚类与、风险最优化分箱方法，分别从数据结构与风险表达角度对 BMI 区间进行划分，形成多个备选分组方案，并通过组内一致性、组间风险差异等指标进行评估与择优。在此基础上，进一步建立基于检测延迟惩罚函数的风险评估模型，以每个 BMI 组为单位，搜索在给定约束下使总风险最小的最优检测时点。

2.3 问题三分析

在问题三中，研究思路围绕“多次检测孕妇的特征变异性”与“个体化检测时点策略”展开，旨在在更复杂的实际场景中构建多因素分组与策略优化模型。首先将孕妇多次检测记录聚合至个体层面，提取统计特征如 Y 浓度均值、标准差、达标率等，构建包含非线性项与交互项的高维特征空间。其次采用随机森林回归评估各特征对达标时间的预测重要性，筛选出一组核心变量用于后续聚类分析。在分组阶段，综合使用 KMeans、层次聚类和高斯混合模型三种算法，比较其聚类结构与轮廓系数，最终选择解释力强、结构清晰的分组方案。随后，在每个分组内部，基于检测时机风险与失败风险联合构建多目标风险函数，并在限定时间窗口内通过有界优化方法确定最优检测时点。

2.4 问题四分析

在问题四中，研究聚焦于女胎染色体异常的早期判定问题，其建模思路围绕“二分类判别”任务展开，涵盖了数据清洗、特征构建、多模型对比与标准化策略评估四个核

心环节。首先针对女胎数据进行专项预处理，重新定义健康标签，并从原始测序数据中提取染色体 Z 值、孕妇基本信息、测序质量指标等核心特征，同时构造如最大 Z 值、异常计数、BMI 交互项等多个衍生变量，形成具有生物学解释力的多层次变量体系。随后，系统引入逻辑回归、随机森林、SVM、决策树等多种分类算法，对女胎是否存在 T13/T18/T21 异常进行建模，并采用标准化、归一化与稳健标准化等不同特征缩放方法提升模型适应性与稳定性，并且在评估阶段综合使用 AUC、敏感性、特异性、F1 值等指标进行模型性能比较，最终选择具有良好解释性与整体表现的逻辑回归模型作为推荐方案。

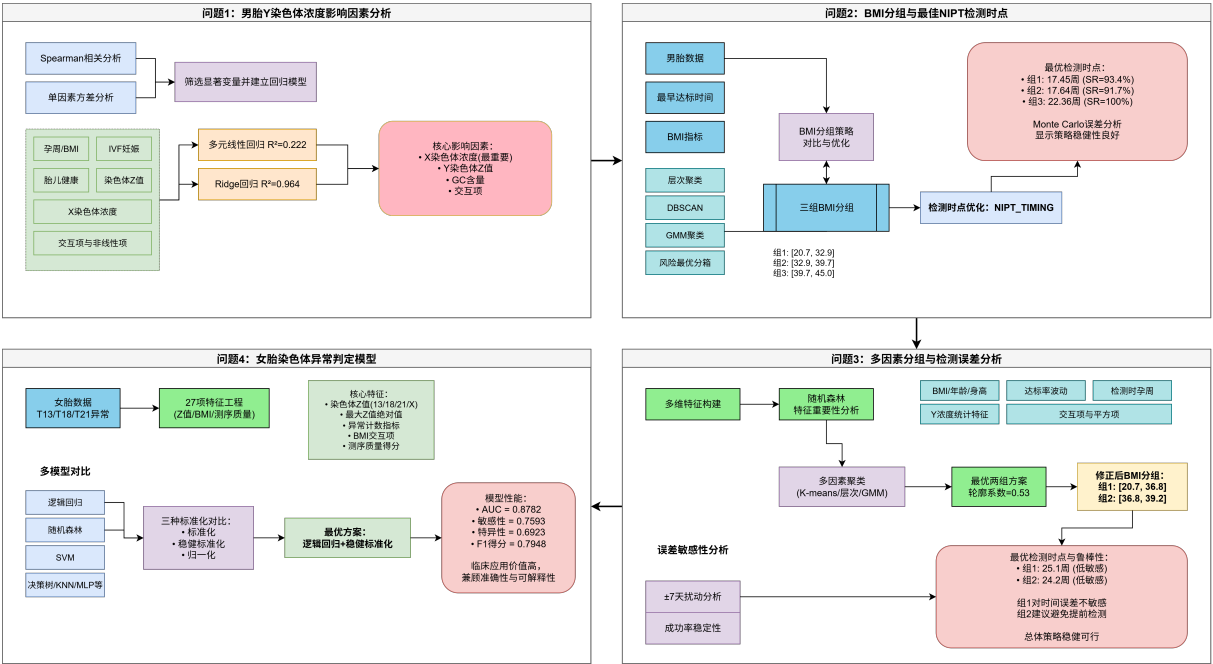


图 1 整体思路流程图

三、模型假设

为聚焦建模核心，简化分析流程，本文作出以下主要假设：

- 假设 1：Y 染色体浓度为连续可测变量，其波动主要受孕妇体格、生理孕周与测序片段浓度等因素影响，忽略实验批次、平台差异及非系统性误差；模型中的样本视为相互独立。
- 假设 2：NIPT 检测成功的判断阈值固定为 4%，适用于所有样本；达标时间 $\{t_{std,i}\}$ 具有可识别性，视为单调过程中的首个达标点，且检测后无回退风险。
- 假设 3：各类优化与聚类算法所依赖的参数（如聚类数 K 、风险权重 α, β 、正则化系数 λ ）均通过启发式方法或交叉验证选取，认为其取值对模型性能具代表性，不展开进一步不确定性传播分析。

四、符号说明

符号	说明
Y_{conc}	胎儿 Y 染色体浓度
$Z_{13}, Z_{18}, Z_{21}, Z_X, Z_Y$	染色体 13/18/21 与 X/Y 的 Z 值
X_{conc}	X 染色体浓度
t_{GA}	检测时怀孕天数
t_{LMP}	末次月经距检测日天数
IVF	IVF 妊娠方式编码
$t_{\text{std},i}$	第 i 位孕妇首次达标时间 (Y 浓度首次 ≥ 0.04)
t_{NIPT}	NIPT 检测时点
$SR(t)$	在时点 t 的预期成功率
$R_{\text{total}}(t)$	总风险函数 (含 R_{timing} 与 R_{failure})
w_{r_i}	第 i 位孕妇的风险权重 (低/中/高)
B, A	BMI 与年龄
K	聚类类别数
$\varepsilon, \text{MinPts}$	DBSCAN 的邻域半径与最小核心点数
α, β	风险最优化分箱的目标权重参数
$C(i, j), F(n, k)$	分箱代价函数与动态规划最优值
$\hat{\beta}, \lambda$	回归系数估计与岭回归正则化强度
Σ	测量/误差的协方差矩阵 (蒙特卡罗)
$S_{\text{group}}, \text{CV}_{\text{risk}}, D_{\text{pass}}$	分组一致性、风险变异系数、通过率偏移 (敏感性指标)
$\ell(\beta), K(\mathbf{x}_i, \mathbf{x}_j)$	逻辑回归对数似然; SVM 核函数

五、问题一模型的建立与求解

5.1 数据预处理

在对 NIPT (无创产前检测) 数据进行建模前, 对原始数据集中的多个关键变量进行了系统性的数据清洗与特征转换, 以提升后续分析的准确性与建模表现。首先, 通过计算“末次月经”与“检测日期”之间的天数差, 得到一个新的变量——**末次月经距检测日的天数**, 从而更精确地反映孕妇在检测时的孕期状态。同时, 将“检测孕周”字段中的字符串形式标准化为总天数, 生成**检测时怀孕天数**这一连续变量, 例如“11w+6”或“13w”。在实际数据中, 部分孕妇存在对末次月经时间的回忆误差, 可能导致推导出的孕期天数出现系统性偏差。为降低该类主观信息的不确定性, 在同一孕妇存在多次检

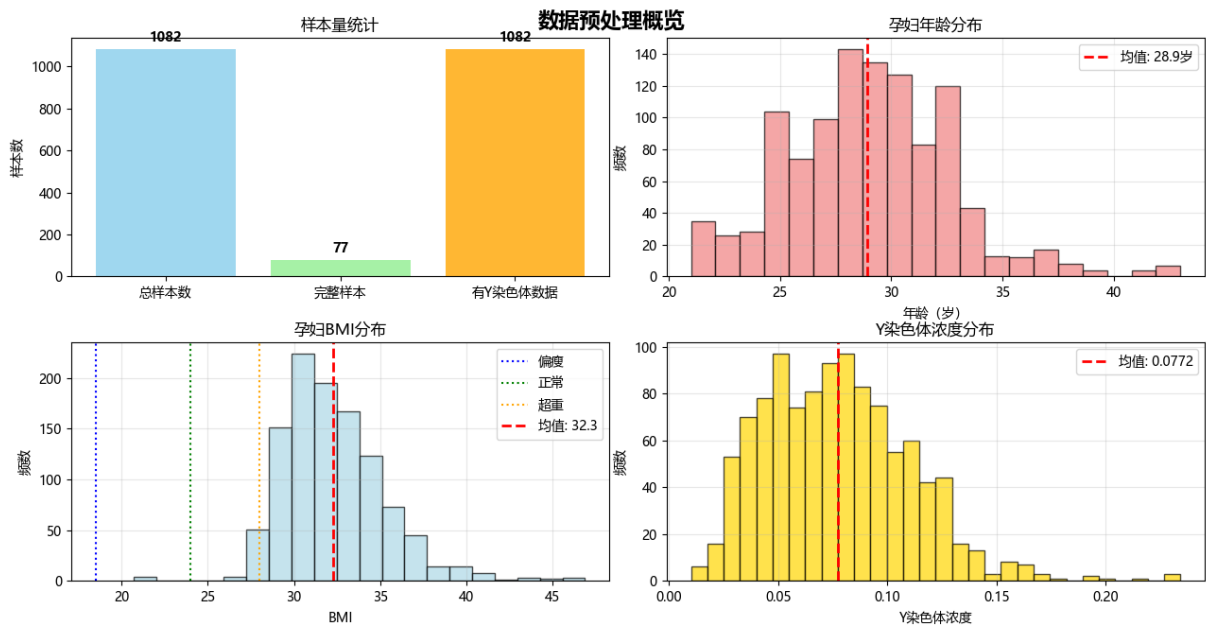


图2 数据预处理统计概览与部分变量分布

测记录的情况下，本文采用其末次月经记录中的众数值作为标准日期。

针对妊娠方式（IVF 妊娠）这一分类信息，采用了两种处理方式。一方面根据妊娠方式的技术复杂程度赋予不同数值编码（自然受孕为 0，人工授精为 1，试管婴儿为 2），另一方面还将其拆分为 **IVF_自然受孕**、**IVF_IUI**、**IVF_IVF** 这三个二进制变量，以适应不同建模场景对特征格式的需求。

对于染色体非整倍体结果，从原始文本中提取是否包含 **T13**、**T18**、**T21** 等关键标记，分别创建 **T13_异常**、**T18_异常**、**T21_异常** 三个指示变量，便于后续风险建模。通过统计发现共有 126 例存在至少一种异常情况，占总样本的 11.6%，其中 T18 异常最为常见。此外，“怀孕次数”被转化为分类编码，按照产次经验进行分层：**初产妇编码为 0，两次经产妇为 1，多次经产妇为 2**。最后，“胎儿是否健康”字段则被转化为数值型二元变量：**健康为 1，不健康为 0**，其中健康胎儿占比为 96.5%。在数据质量检查环节，发现“末次月经距检测日天数”存在 37.7% 缺失，而“检测时怀孕天数”仅缺失 1 例，其余新生成字段均无缺失。

为进一步理解变量的分布特征与数据整体结构，设计并绘制了多种可视化图表。在图 2 中呈现了样本总体分布、有效数据比例、年龄和 BMI 等变量的频数统计。图 3 展示了年龄、身高、体重、BMI、GC 含量、Y 染色体浓度、Z 值等连续变量的分布情况，配合核密度估计曲线以及均值与中位数标注，从中可观察到部分变量如体重、Y 染色体浓度等存在一定程度的偏态分布。处理完成后数据共包含 46 个字段，其中包括原始字段、衍生字段、数值编码字段与哑变量等，已存储为精简版与完整版两种结构化数据表格。

与此相呼应，图 2 给出了样本量概览及若干关键变量的直方展示：总样本 1082 例

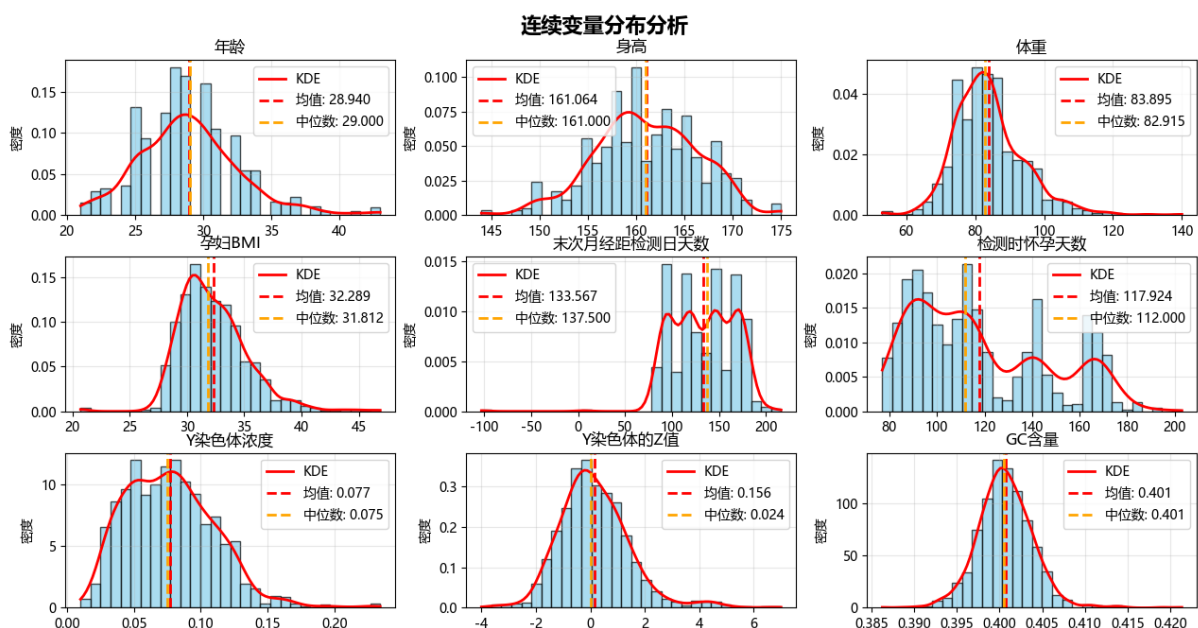


图3 连续变量分布与核密度估计图

中可用于染色体分析者几近全覆盖；BMI 的均值约 32.3，位于超重—肥胖分界线之上，暗示体重相关指标可能对片段分布与胎源 DNA 比例产生系统性影响；年龄均值约 28.9 岁，整体近似对称但右端仍见少量 >35 岁的尾部，高龄孕妇在样本中占比并不高。

进一步地，图 3 从九个连续变量刻画了更细的分布纹理：年龄与身高近似正态、中心稳定；体重右偏明显，结合 BMI 的轻度右偏可见超重与肥胖在样本中较常见；“检测时怀孕天数”的核密度曲线呈多峰，主峰约在 112 天（中位数），尚有 120–140 天与 >160 天的次峰，反映出就诊节律与采血时点的差异；“末次月经距检测日天数”除 135–140 天附近主峰外，出现最小值 -103 天的异常记录，提示存在录入或对齐误差，建模前宜做范围裁剪或规则化纠错；GC 含量以 0.401 为中心呈近高斯形态，显示文库复杂度整体稳定、批效应不显著；Y 染色体浓度（均值约 0.0772，中位数 0.075）右尾较长，适合对其取 log 或采用稳健变换以减弱极端值影响；Y 染色体 Z 值均值约 0.156、中位数 0.024，整体围绕 0 对称但尾部略重，或反映技术波动与真实生物学差异并存。为验证标签工程的可靠性，对前 10 个样本的异常解析进行了人工抽查，T13/T18/T21 的自动提取与原始记录一致，从而确认了解析逻辑的准确可用。

5.2 Spearman 相关性系数

在这一问中，由于因变量胎儿 Y 染色体浓度明显不服从正态分布（Shapiro–Wilk 检验 $W = 0.9676$, $p < 0.0001$ ，偏度为 0.7043，峰度为 0.9991，详见图 4），因此优先使用 **Spearman 秩相关系数** 对其与各连续变量的关联性进行探索^{3,4}。该方法能够在无需正态性假设的前提下，衡量两个变量之间的单调关系强度，尤其适用于存在极端值、偏态分布或等级测量数据的场景。

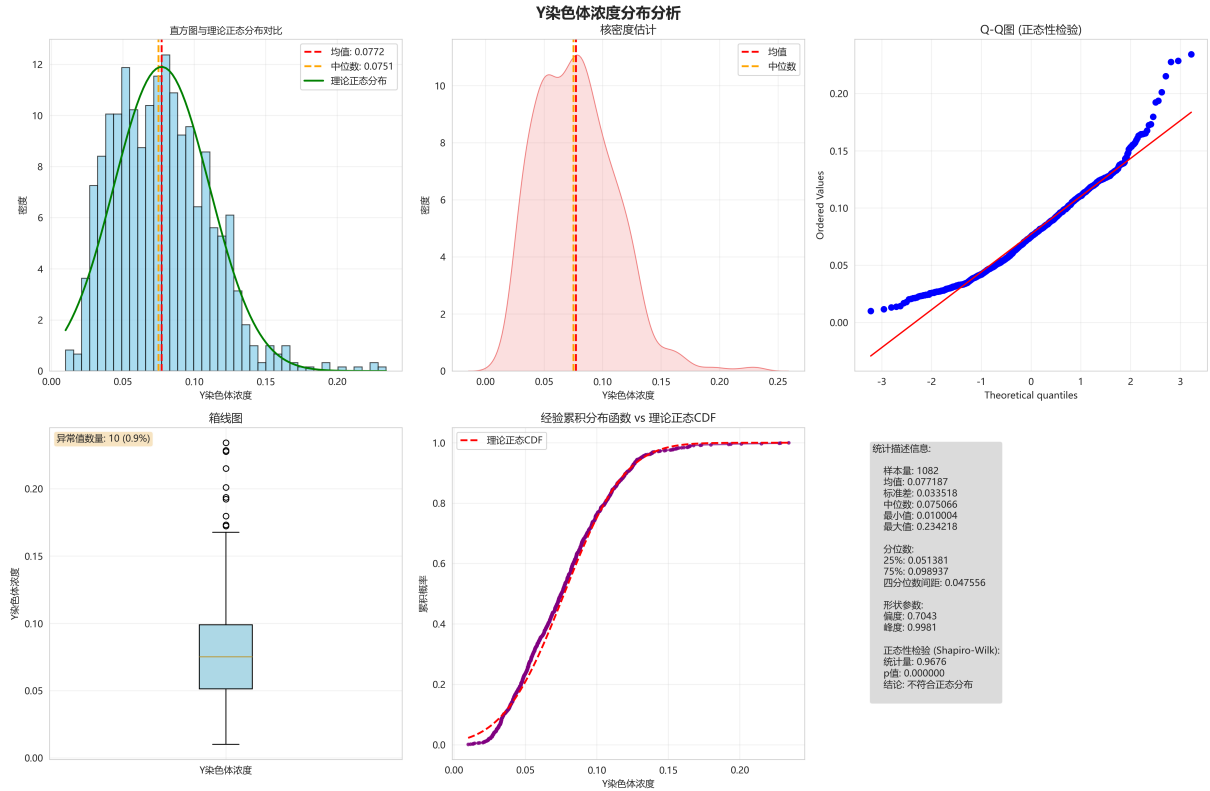


图4 Y 染色体浓度的分布特征与正态性检验结果

定义与用途：*Spearman* 相关性系数 (*Spearman's ρ*) 是用于衡量两个变量秩之间的单调相关强度,适合非正态分布、存在离群值或等级数据的情形。其取值范围为 $[-1, 1]$, 绝对值越大表示单调关系越强。

计算公式：当无并列秩（无“平秩”）时，

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}, \quad d_i = \text{rank}(x_i) - \text{rank}(y_i). \quad (1)$$

一般情形下（允许并列秩），等价于对秩后的变量做皮尔逊相关：

$$\rho = \text{corr}(\text{rank}(X), \text{rank}(Y)). \quad (2)$$

显著性检验：在样本量较大时，可对秩后的相关系数采用近似 *t* 检验（或置换检验）：

$$t = \rho \sqrt{\frac{n-2}{1-\rho^2}} \sim t_{n-2} \quad (\text{原假设 } \rho = 0). \quad (3)$$

Spearman 分析结果如图 5 所示，揭示了若干变量与 Y 染色体浓度间的单调关系。例如：X 染色体浓度与 Y 染色体浓度相关系数高达 0.82，检测孕周与末次月经距检测日天数亦呈中等正相关 ($\rho \approx 0.35$)，BMI、年龄、体重等变量与 Y 染色体浓度呈弱负相关 ($\rho \approx -0.08 \sim -0.15$)。虽然多数变量的相关性未超过中等水平，但在大样本下部分达到统计显著，表明 Y 染色体浓度受孕期进程、孕妇体格、技术片段分布等多因素的共同影响。

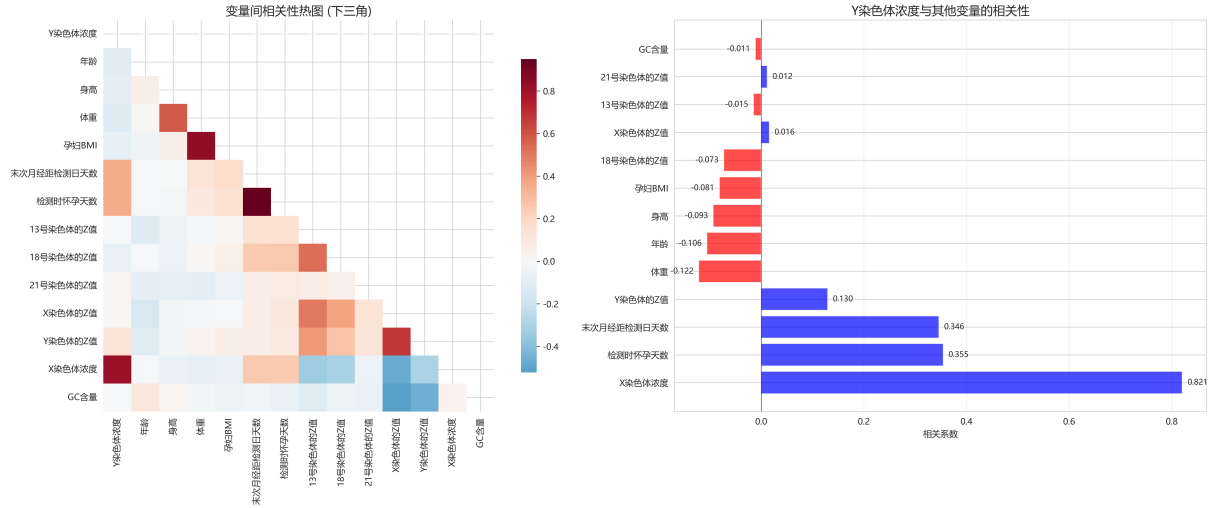


图5 变量间相关性热图与 Y 染色体浓度的相关条形图

5.3 单因素方差分析

目的：检验一个分类自变量（因子，含 $k \geq 2$ 个水平）对一个连续因变量的总体均值是否存在差异，即

$$H_0: \mu_1 = \mu_2 = \cdots = \mu_k \quad \text{vs.} \quad H_1: \text{至少一组均值不同.}$$

方差分解与统计量：设第 j 组样本量为 n_j ，组均值为 $\bar{y}_j$ ，总体均值为 $\bar{y}_{..}$ ，总样本量 $N = \sum_{j=1}^k n_j$ 。有

$$SS_B = \sum_{j=1}^k n_j (\bar{y}_j - \bar{y}_{..})^2, \quad (4)$$

$$SS_W = \sum_{j=1}^k \sum_{i=1}^{n_j} (y_{ij} - \bar{y}_j)^2, \quad (5)$$

$$MS_B = \frac{SS_B}{k-1}, \quad MS_W = \frac{SS_W}{N-k}. \quad (6)$$

F 统计量为

$$F = \frac{MS_B}{MS_W} \sim F_{k-1, N-k} \quad (\text{在 } H_0 \text{ 下}). \quad (7)$$

假设与扩展：独立性、各组近似正态、方差齐性。若方差不齐，可采用 Welch ANOVA。拒绝 H_0 后应进行多重比较（如 Tukey、Bonferroni），并报告效应量（如 $\eta^2 = SS_B / (SS_B + SS_W)$ ）与置信区间。

在实际分析中，通过 ANOVA 或 Kruskal-Wallis 检验，比较多组离散因子（如 IVF 妊娠方式、胎儿健康状态、染色体异常类型）对 Y 染色体浓度的影响，结果如图 6 所示。其中 IVF 妊娠方式呈现统计学显著差异（ $p = 0.0238$ ），胎儿健康编码更为显著（ $p = 0.0002$ ），不健康组浓度整体更高；但产次分类及 T13/T18/T21 异常组别并未显著

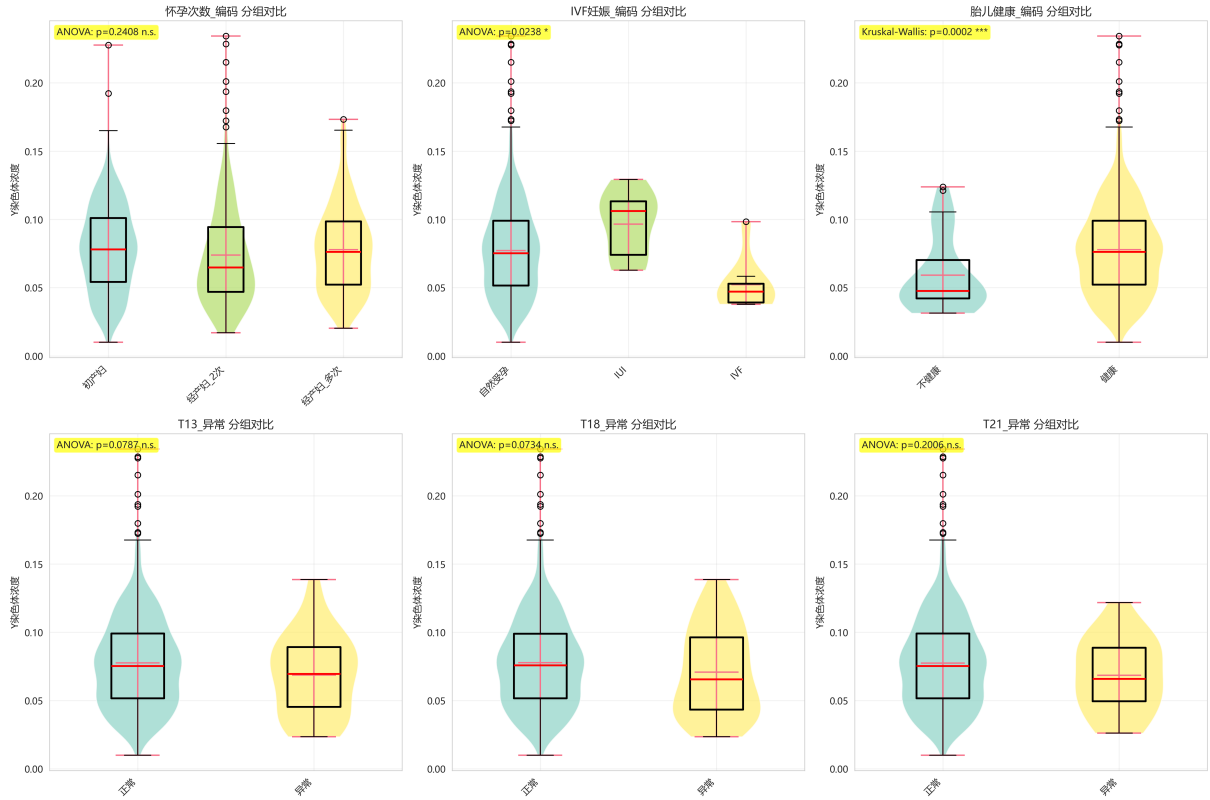


图6 不同分组变量在 Y 染色体浓度上的差异比较

($p > 0.05$), 提示这些变量对 Y 染色体浓度无实质性影响。此外, 为支持分析稳定性, 结合效应大小指标 (见图 7 右侧), 发现胎儿健康和 IVF 妊娠两个变量不仅 P 值显著, 其相对统计效应也明显大于其他因素 (分别为 13.4 与 3.75)。这一结果提示, 在建模阶段应重点考虑包含这两类因子的主效应与可能交互项。

5.4 多元线性回归

多元线性回归模型刻画连续因变量与多个自变量的线性关系

$$y = \beta_0 + \mathbf{X}\beta + \varepsilon, \quad \mathbb{E}(\varepsilon) = 0, \text{Var}(\varepsilon) = \sigma^2 \mathbf{I}. \quad (8)$$

普通最小二乘 (OLS) 估计量为

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (9)$$

在本研究中, 以 673 个完整样本为基础, 选取变量间 VIF 不高于 10、且在单变量检验中具备显著性或理论意义的指标进行建模, 并尝试加入交互项与 Z 值比值等增强性特征。最终模型保留三个显著特征: Y 染色体的 Z 值、X 染色体浓度与其乘积交互项、以及 Y/X 的 Z 值比率, 其中交互项呈显著正向影响, Y 的 Z 值与比值项呈负向影响, 后三者系数符号与生物意义一致, 且线性模型在测试集上取得 $R^2 = 0.222$ (见图 8 左上)。特征标准化系数显示, 交互项贡献度最高, 为主要解释变量。

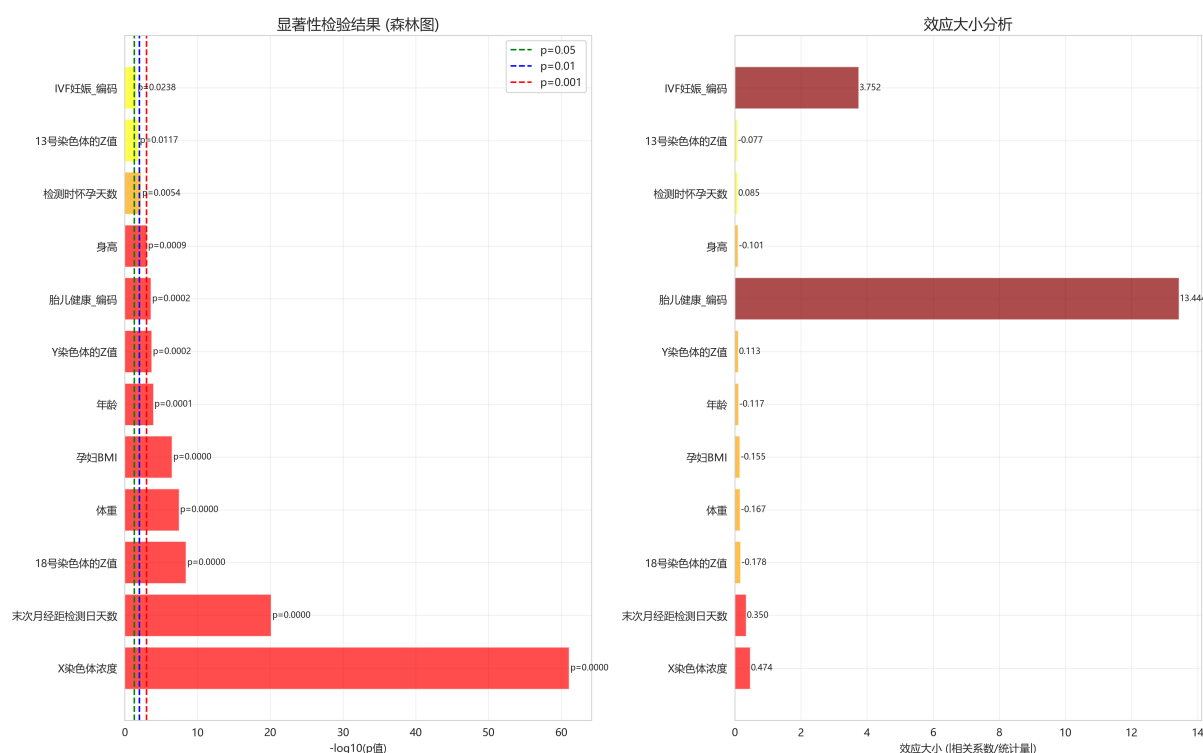


图7 统计学显著性检验（左）与效应量大小比较（右）

在误差正态与同方差条件下，系数的 t 检验与整体的 F 检验成立；拟合优度用 R^2 与调整后 R^2 。关键诊断包括：线性关系、残差同方差/正态、影响点与高杠杆点、多重共线性（如方差膨胀因子 VIF），必要时可加入交互项、非线性项（多项式/样条）或做稳健回归。

根据残差图与 Q-Q 图（见图 8 中部），残差均值近似 0，但尾部偏态明显，Shapiro-Wilk 检验 $p < 0.0001$ 明确拒绝正态性；Breusch-Pagan 检验 $p = 0.0016$ 显示异方差存在，说明模型虽然线性结构合理，但误差分布不满足 Gauss-Markov 假设，后续应采用稳健方差估计或对数变换处理因变量。散点与学习曲线分析显示模型整体处于“偏差主导”下的欠拟合区间，训练集与验证集表现接近，且 Ridge 正则回归与 OLS 性能基本一致，说明变量已相对充分表达可预测部分，进一步提升需借助非线性方法或更高维特征空间。相关矩阵显示 Y 的 Z 值与交互项存在中等相关 (≈ 0.59)，合理引入正则化有助于提升估计稳定性。

5.5 Ridge 岭回归

当自变量众多且高度相关或存在过拟合风险时，在 OLS 基础上加入 L_2 正则化以收缩系数，提高预测稳健性并缓解多重共线性。Ridge 回归特别适用于本研究这种存在高维生物指标、强相关变量（如 X/Y 染色体指标）、部分非线性项与交互项同时建模的情境。⁵

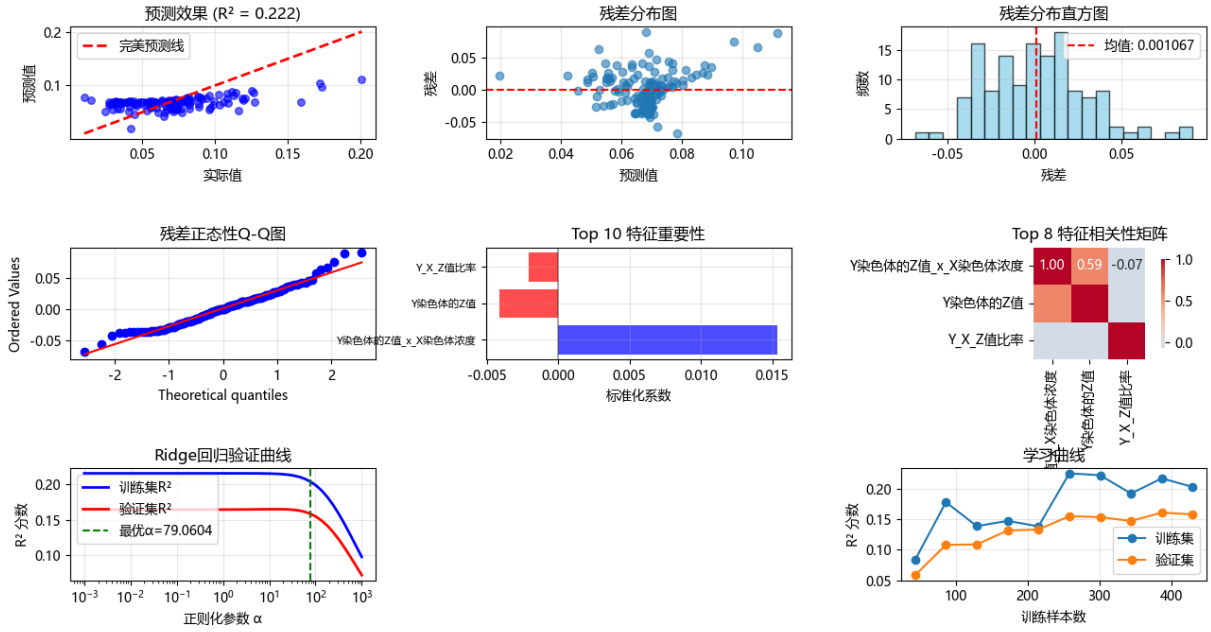


图 8 多元线性与 Ridge 回归建模结果可视化

目标函数与闭式解：

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2, \quad \hat{\beta}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}, \quad (10)$$

其中 $\lambda \geq 0$ 为正则化强度。

各特征应先标准化； λ 通常通过交叉验证选择。岭回归不会将系数压缩为 0（不做变量选择），与 Lasso (L_1 ，可做稀疏选择) 互补；Elastic Net 兼顾 L_1 与 L_2 。适用于自变量维度较高、相关性强且更关注预测性能的场景。

在本研究中，基于 673 个完整样本，构建了包含 25 个特征（含多项式、交互项）的标准化数据集，并采用十折交叉验证选择最优正则化强度，结果如图 9 左上所示，最优 $\alpha \approx 0.8685$ 。模型最终在测试集上取得 $R^2 = 0.964$ 、 $\text{RMSE}=0.00617$ ，表现远优于线性模型，且“预测值-真实值”几乎完全贴合 45° 对角线，说明模型捕捉到了因变量变化的主要结构性来源。

变量贡献分析表明，模型的解释力主要来自 X 染色体浓度、X 染色体 Z 值、GC 含量与 Y 染色体 Z 值，其中 X 染色体浓度的标准化系数最大 (≈ 0.041)，为最核心的正向因子（图 9 右上）；分类变量与高阶多项式项的贡献极小（见左下饼图，连续变量贡献占比高达 94.2%），表明模型依赖于基因测序强度及片段结构等“信号型”变量，而孕妇体格、时间变量及非线性特征仅起边际修正作用。

模型残差整体服从中心对称分布，残差均值为 9.0×10^{-4} ，主峰集中、无明显系统性偏移。但残差 Q-Q 图（见图 10 右上）显示右尾偏离显著，Shapiro-Wilk 检验 $p < 10^{-6}$ ，显著拒绝正态性，表明在尾部存在潜在极端值或模型不足拟合；此外，通过残差与预测值、X 浓度、孕周等变量的散点图发现未呈现明显模式，提示异方差不严重，但少量高

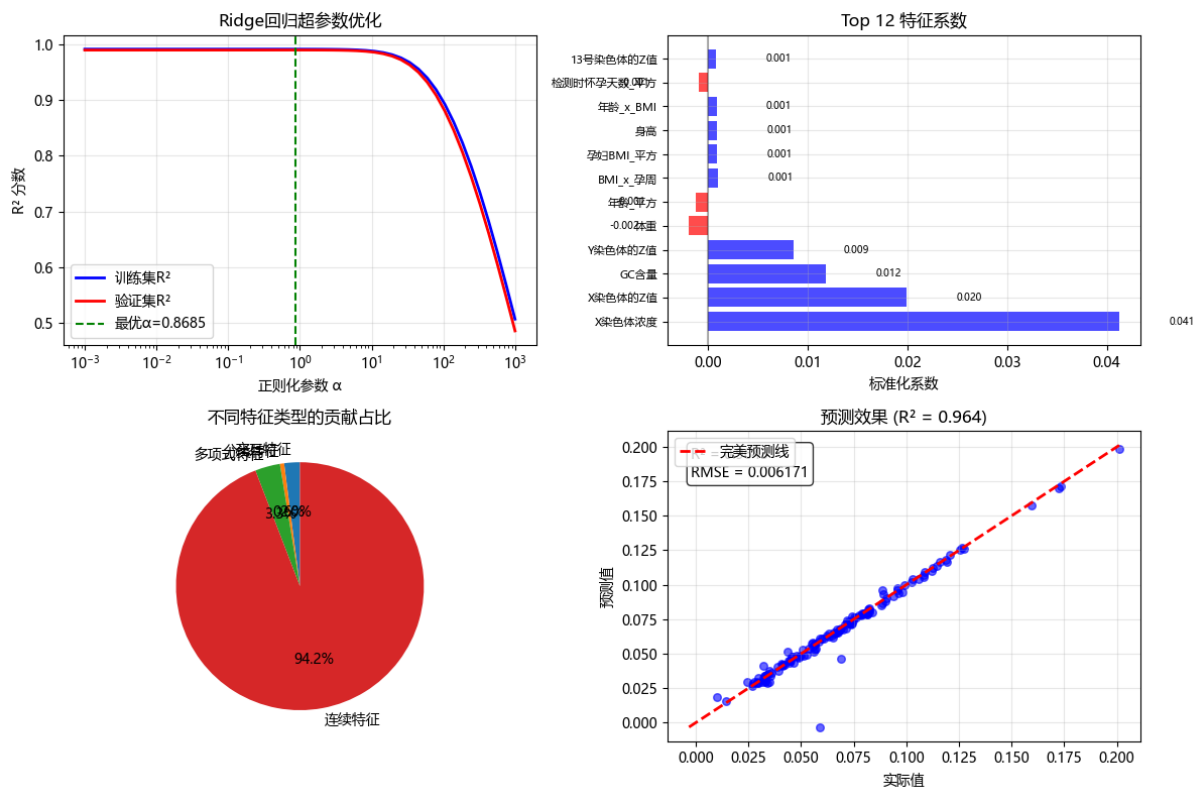


图9 Ridge 回归整体拟合表现、特征重要性与预测效果

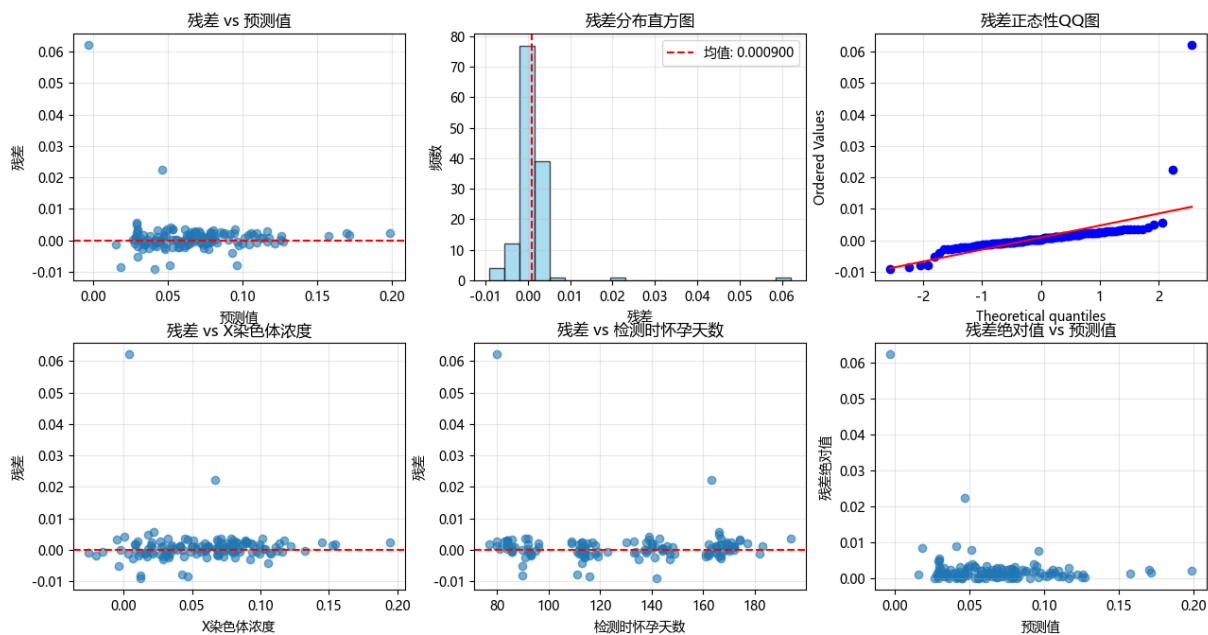


图10 Ridge 回归残差分析图：残差分布、Q-Q 图与预测变量关系

影响点需进一步稳健处理。建议在保持线性框架与 Ridge 正则结构基础上，结合 Yeo-Johnson 变换或使用 HC 类稳健标准误，以提高推断准确性与 CI 的可解释性。

六、问题二的模型的建立和求解

6.1 数据预处理

在进行第二问的求解之前，在第一问的基础之上新增了如下三个衍生变量：

表 1 问题 2 新增变量转换处理汇总表

原始变量	转换后变量	转换规则
检测时怀孕天数	风险指标	按孕期时间分配风险等级： ≤ 84 天（12 周内）为 0（低风险），85-189 天（13-27 周）为 1（高风险）， ≥ 190 天（28 周后）为 2（极高风险）
Y 染色体浓度	最早达标时间 _ 天数	按孕妇分组，找到 Y 染色体浓度 ≥ 0.04 （4%）的最早检测时间（天数），未达标者为 NaN
Y 染色体浓度	当前检测达标	判断当前检测 Y 染色体浓度是否 ≥ 0.04 （4%）：达标为 1，未达标为 0
最早达标时间 _ 天数	曾经达标	判断该孕妇是否曾经达标过：曾经达标为 1（最早达标时间不为空），从未达标为 0

6.2 模型介绍

6.2.1 DBSCAN 聚类

目的与原理：DBSCAN（Density-Based Spatial Clustering of Applications with Noise）是一种基于密度的聚类算法，能够发现任意形状的聚类并有效识别噪声点。该算法通过定义密度可达性概念，将高密度区域连接成聚类，适合处理具有不规则分布的 BMI 数据^{6,7}。

核心概念：设数据集 $D = \{x_1, x_2, \dots, x_n\}$ ，定义两个关键参数：

- ε ：邻域半径，定义点的局部密度范围
- MinPts：最小点数，判定核心点的密度阈值

密度定义：对于点 x_i ，其 ε -邻域定义为：

$$N_\varepsilon(x_i) = \{x_j \in D \mid \text{dist}(x_i, x_j) \leq \varepsilon\} \quad (11)$$

点类型分类：

$$\text{核心点: } |N_\varepsilon(x_i)| \geq \text{MinPts} \quad (12)$$

$$\text{边界点: } |N_\varepsilon(x_i)| < \text{MinPts} \text{ 且存在核心点 } x_j \text{ 使得 } x_i \in N_\varepsilon(x_j) \quad (13)$$

$$\text{噪声点: } \text{既非核心点也非边界点} \quad (14)$$

密度连通性：点 x_i 和 x_j 密度连通，当且仅当存在核心点序列 $\{p_1, p_2, \dots, p_k\}$ 使得：

$$x_i \in N_\varepsilon(p_1), \quad x_j \in N_\varepsilon(p_k), \quad p_{t+1} \in N_\varepsilon(p_t) \quad (t = 1, \dots, k-1) \quad (15)$$

6.2.2 层次聚类

基本思想：层次聚类通过计算样本间距离并逐步合并或分割，构建树状聚类结构（树状图）。对于 BMI 分组问题，采用凝聚型层次聚类，从单个样本开始逐步合并最相似的聚类。⁵

距离度量：常用欧几里得距离衡量样本相似性：

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2} \quad (16)$$

聚类间距离：定义聚类 C_i 和 C_j 间的距离：

$$\text{单链接：} \quad d_{\text{single}}(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y) \quad (17)$$

$$\text{全链接：} \quad d_{\text{complete}}(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y) \quad (18)$$

$$\text{平均链接：} \quad d_{\text{average}}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y) \quad (19)$$

$$\text{Ward 链接：} \quad d_{\text{ward}}(C_i, C_j) = \sqrt{\frac{2|C_i||C_j|}{|C_i| + |C_j|}} \|\mu_i - \mu_j\|_2 \quad (20)$$

层次聚类步骤：

Step 1 初始化：每个样本为独立聚类。

Step 2 计算所有聚类间距离。

Step 3 合并距离最小的两个聚类。

Step 4 更新距离矩阵，重复步骤 2-3 直至达到指定聚类数。

6.2.3 风险最优化分箱

目标函数：风险最优化分箱旨在最小化组内风险方差同时最大化组间风险差异，构建目标函数：

$$\min_{B_1, \dots, B_k} \left[\alpha \sum_{i=1}^k \text{Var}(R_i) - \beta \sum_{i=1}^{k-1} |\bar{R}_{i+1} - \bar{R}_i| \right] \quad (21)$$

其中 B_i 为第 i 个分箱， R_i 为该箱内样本的风险指标， $\bar{R}_i$ 为组内风险均值， $\alpha, \beta > 0$ 为权重参数。

风险度量：定义基于检测时间的潜在风险函数：

$$\text{Risk}(t) = \begin{cases} w_1 \cdot e^{-\lambda_1 t}, & t \leq t_{\text{low}} \quad (\text{低风险期}) \\ w_2 \cdot (t - t_{\text{low}})^2, & t_{\text{low}} < t \leq t_{\text{high}} \quad (\text{高风险期}) \\ w_3 \cdot e^{\lambda_2(t-t_{\text{high}})}, & t > t_{\text{high}} \quad (\text{极高风险期}) \end{cases} \quad (22)$$

分箱边界优化：采用动态规划求解最优分割点 $\{c_1, c_2, \dots, c_{k-1}\}$ ：

$$F(n, k) = \min_{1 \leq j < n} [F(j, k-1) + C(j+1, n)] \quad (23)$$

其中 $C(i, j)$ 为区间 $[i, j]$ 的代价函数， $F(n, k)$ 为将前 n 个样本分成 k 组的最小代价。

6.2.4 Monte Carlo 模拟方法

Monte Carlo 方法是通过多轮随机抽样模拟观测误差，分析系统中不确定性如何影响分组判定和风险评价，以衡量建模方案的稳健性。

假设测量误差向量服从多元正态分布：

$$\boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_Y \\ \varepsilon_B \\ \varepsilon_W \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \boldsymbol{\Sigma} \right), \quad (24)$$

其中协方差矩阵 $\boldsymbol{\Sigma}$ 反映三个测量变量之间的误差相关性。

在第 s 次模拟中，扰动后的观测值为：

$$Y^{(s)} = Y^* + \varepsilon_Y^{(s)} \quad (25)$$

$$B^{(s)} = B^* + \varepsilon_B^{(s)} \quad (26)$$

$$W^{(s)} = W^* + \varepsilon_W^{(s)} \quad (27)$$

其中 Y^*, B^*, W^* 为“真实值”或估计均值。

将扰动值代入组合风险函数：

$$R_{\text{prop}}^{(s)} = f(B^{(s)}, W^{(s)}) \cdot g(Y^{(s)}) \quad (28)$$

其中 $f(\cdot)$ 表示分组映射函数， $g(\cdot)$ 表示达标判定函数。

对 S 次模拟，定义三个关键敏感性度量：

$$S_{\text{group}} = \frac{1}{S} \sum_{s=1}^S \mathbb{I} [G^{(s)} = G^{(0)}] \quad (\text{分组一致性}) \quad (29)$$

$$\text{CV}_{\text{risk}} = \frac{\sqrt{\text{Var} (R_{\text{prop}}^{(s)})}}{\mathbb{E} [R_{\text{prop}}^{(s)}]} \quad (\text{风险变异系数}) \quad (30)$$

$$D_{\text{pass}} = \left| \frac{1}{S} \sum_{s=1}^S P^{(s)} - P^{(0)} \right| \quad (\text{通过率偏移量}) \quad (31)$$

6.3 构建模型

6.3.1 BMI 区间构造与重叠修正

多方法 BMI 分组策略：为了获得最优的 BMI 分组方案，本研究采用 5 种不同的聚类方法进行对比分析，包括高斯混合模型（GMM）、层次聚类、DBSCAN 密度聚类、风险最优化分箱和决策树分箱。每种方法都针对孕妇 BMI 和最早达标时间构建分组策略。⁵

高斯混合模型分组：采用期望最大化算法拟合多元高斯分布，模型似然函数为：

$$\mathcal{L}(\theta) = \prod_{i=1}^n \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (32)$$

其中 π_k 为第 k 个分量的混合权重， $\boldsymbol{\mu}_k$ 和 $\boldsymbol{\Sigma}_k$ 分别为均值向量和协方差矩阵。通过最小化贝叶斯信息准则（BIC）选择最优分组数：

$$\text{BIC} = -2 \ln(\mathcal{L}) + p \ln(n) \quad (33)$$

风险最优化分箱模型：构建基于风险最小化的目标函数，寻找最优分割点：

$$\min_{\{c_1, c_2, \dots, c_{k-1}\}} \sum_{j=1}^k \frac{n_j}{n} \cdot \text{Risk}_j \quad (34)$$

其中风险函数定义为：

$$\text{Risk}_j = \sum_{i \in G_j} w_{r_i} \cdot f(\text{BMI}_i, t_{\text{std}, i}) \quad (35)$$

风险权重设定为 $w_0 = 1$ （低风险）， $w_1 = 2$ （高风险）， $w_2 = 4$ （极高风险）。

区间重叠修正算法：考虑聚类或分段生成的数值区间中可能存在的重叠问题，设计自动化的区间修正机制，以确保最终各区间之间互不相交、覆盖完整且连续。给定两个区间 $I_i = [a_i, b_i]$ 与 $I_j = [a_j, b_j]$ ，其发生重叠当且仅当满足下述条件触发重叠判定：

$$a_i < b_j \quad \wedge \quad b_i > a_j. \quad (36)$$

当发现 I_i 与 I_j 存在重叠时，通过中点分割修正策略计算新的边界点 c_{ij} 为：

$$c_{ij} = \frac{b_i + a_j}{2}. \quad (37)$$

据此更新修正后的两个区间为：

$$I_i^{\text{corr}} = [a_i, c_{ij}], \quad (38)$$

$$I_j^{\text{corr}} = [c_{ij}, b_j]. \quad (39)$$

若有交叠严重、非数值型误差等现象，中点法不可行，则采用强制等距分割方案。设总体变量的取值范围为 $[x_{\min}, x_{\max}]$ ，划分为 k 个等距子区间，其边界集合为：

$$\mathcal{B} = \left\{ x_{\min} + i \cdot \frac{x_{\max} - x_{\min}}{k} \mid i = 0, 1, \dots, k \right\}. \quad (40)$$

分组效果评估： 建立综合评估指标体系：

$$S_{\text{risk}} = \frac{1}{N_j} \sum_{i \in G_j} w_{r_i} \quad (41)$$

$$D_{\text{intra}} = \text{Var}(\text{BMI}_j) + \text{Var}(t_{\text{std},j}) \quad (42)$$

$$D_{\text{inter}} = \frac{\sum_{j=1}^{k-1} |\bar{R}_{j+1} - \bar{R}_j|}{k-1} \quad (43)$$

通过多方法对比，选择具有最小平均风险得分的分组方案作为最优 BMI 分组策略： S_{risk} 表示分组内个体的平均加权风险得分，衡量组内风险水平； D_{intra} 为组内差异度，由 BMI 方差与达标时间方差构成，反映组内一致性； D_{inter} 衡量相邻组之间的平均风险差异，刻画组间分离程度。其他变量中， t_{std} 表示个体达标所需时间， $\bar{R}$ 为各组平均风险值。整个指标体系用于综合评估分组的稳健性与区分性。

6.3.2 NIPT 检测时点优化

基于已确定的 BMI 分组，为每组构建最优 NIPT 检测时点优化模型。目标是最小化组内总体风险，同时确保检测的可行性和有效性。对于第 j 组内的孕妇 i ，定义个体风险函数：

$$R_i(t_{\text{NIPT}}) = w_{r_i} \cdot P(t_{\text{NIPT}}, t_{\text{std},i}) \quad (44)$$

其中延迟惩罚函数 $P(t_{\text{NIPT}}, t_{\text{std},i})$ 定义为：

$$P(t_{\text{NIPT}}, t_{\text{std},i}) = \begin{cases} 1.0, & \text{delay} \leq 7 \\ 1.5, & 7 < \text{delay} \leq 14 \\ 2.0, & 14 < \text{delay} \leq 28 \\ 3.0, & \text{delay} > 28 \end{cases} \quad (45)$$

其中 $\text{delay} = \max(0, \max(t_{\text{NIPT}}, t_{\text{std},i}) - t_{\text{NIPT}})$ 。

第 j 组的总风险函数为：

$$\text{TotalRisk}_j(t_{\text{NIPT}}) = \frac{1}{N_j} \sum_{i \in G_j} R_i(t_{\text{NIPT}}) \quad (46)$$

约束条件说明：

- 检测时间不早于 10 周（70 天），确保胎儿 DNA 浓度基本可检测

- 检测时间不晚于 25 周（175 天），保证足够的干预窗口期
- 搜索范围基于组内最早/最晚达标时间动态调整

$$\begin{aligned} & \min_{t_{\text{NIPT}}} \text{TotalRisk}_j(t_{\text{NIPT}}) \\ & \text{s.t.} \begin{cases} t_{\min} \leq t_{\text{NIPT}} \leq t_{\max} \\ t_{\min} = \max(70, \min_i t_{\text{std},i} - 14) \\ t_{\max} = \min(175, \max_i t_{\text{std},i} + 14) \end{cases} \end{aligned} \quad (47)$$

数值求解方法：采用有界标量优化算法求解：

$$t_{\text{NIPT}}^* = \arg \min_{t \in [t_{\min}, t_{\max}]} \text{TotalRisk}_j(t) \quad (48)$$

优化结果评估指标：

$$SR_j = \frac{\sum_{i \in G_j} \mathbb{I}[t_{\text{std},i} \leq t_{\text{NIPT}}^*]}{N_j} \quad (49)$$

$$WT_j = \frac{1}{N_j} \sum_{i \in G_j} \max(0, t_{\text{std},i} - t_{\text{NIPT}}^*) \quad (50)$$

$$RR_j = \frac{R_{\text{base}} - R_{\text{opt}}}{R_{\text{base}}} \times 100\% \quad (51)$$

通过对每个 BMI 分组独立进行时点优化，最终获得个性化的 NIPT 检测时间建议，实现了在保证检测成功率的前提下最小化潜在风险的目标。 t_{NIPT} 表示 NIPT 检测时间点， $t_{\text{std},i}$ 为第 i 个孕妇达到干预达标条件的时间； w_{r_i} 是第 i 个孕妇的风险权重，用于构造加权风险评估； d 表示检测与干预之间的延迟时间； R_{base} 和 R_{opt} 分别表示策略优化前后的基线风险与优化风险值； SR_j 为第 j 个分组中样本达标的成功率， WT_j 表示该组平均等待时间， RR_j 表示该组的风险下降幅度。以上变量构成了分组评价与策略优化过程中的关键指标体系。

6.4 择优求解分析

下表为不同聚类算法效果的综合对比，由下表得知，层次聚类方法在综合性能上表现最优，其综合评分为 1.78（最低），同时具有最高的平均成功率（0.95）和相对较低的风险得分（1.82）。相比之下，GMM 方法虽然分组数最多（6 组），但其综合评分较高（1.96），而 DBSCAN 和风险优化方法在成功率方面表现相对较弱（均低于 0.87），表明层次聚类在 BMI 分组策略中能够更好地平衡检测成功率与风险控制的要求。

表 2 不同 BMI 分组方法的性能比较

方法	分組数	平均风险得分	风险得分标准差	平均成功率	平均最佳时点(周)	低风险组比例	综合评分
层次聚类	3	1.82	0.08	0.95	19.15	0	1.78
GMM	6	1.88	0.11	0.93	18.66	0	1.96
DBSCAN	3	1.80	0.08	0.87	17.56	0	2.01
风险优化	2	1.83	0.13	0.86	17.56	0	2.06

图 11 展示了基于**层次聚类**所得到的 BMI 三分组策略及其对应的最优检测时点与风险评估结果。左上直方图结合虚线标示了修正后的 BMI 划分边界：**组 1 为 [20.70, 32.93]**，**组 2 为 [32.93, 39.70]**，**组 3 为 [39.70, 44.98]**，各组区间互不重叠、结构清晰。中上柱形图给出三组样本的“最早达标时间”，与右上散点图中显示的“最优检测时点与预期成功率关系”相呼应，三组建议的检测时点分别为 **17.45 周、17.64 周和 22.36 周**，对应的预期达标成功率依次为 93.4%、91.7% 与 100.0%。需特别注意的是，组 3 样本量仅为 6 例，受样本规模限制，应在实际中加以验证。

左下饼图呈现了三组的风险等级结构，66.7% 为“中风险”、33.3% 为“较高风险”，组别与风险等级的对应关系在右下表格中亦有明确标示，其中组 1 与组 2 为中风险，组 3 为较高风险。中下散点图进一步揭示 BMI 与最早达标时间的关系：整体呈上升趋势，BMI 较高者达标时间明显延迟，这也合理解释了组 3 推迟检测的必要性。综合来看，该策略不仅使分组边界清晰、分布合理，还将主要检测建议集中于 17–18 周这一实际可操作时间段，仅对极高 BMI 人群适度延后至 22 周，从而在风险控制与可实施性之间取得良好平衡。

6.5 误差敏感性分析

对三组孕妇分别进行 ± 7 天的蒙特卡洛随机扰动模拟，并计算各组在不同误差条件下的预期达标成功率（见表 3）。结果显示，三组的成功率在扰动范围内波动极小（最大波动不超过 2.78%），敏感性均评估为“低敏感”。其中，组 1 和组 3 表现出高度稳定性，组 2 在提前 7 天时略有下降（从 91.7% 降至 88.9%），故高 BMI 人群应避免检测过早。⁸

表 3 不同扰动下各组的预期成功率与敏感性评估

组别	-7 天	-3 天	0 天	+3 天	+7 天	波动范围 / 敏感性
组 1	0.923	0.934	0.934	0.940	0.940	1.64% / 低敏感
组 2	0.889	0.907	0.917	0.917	0.917	2.78% / 低敏感
组 3	1.000	1.000	1.000	1.000	1.000	0.00% / 低敏感

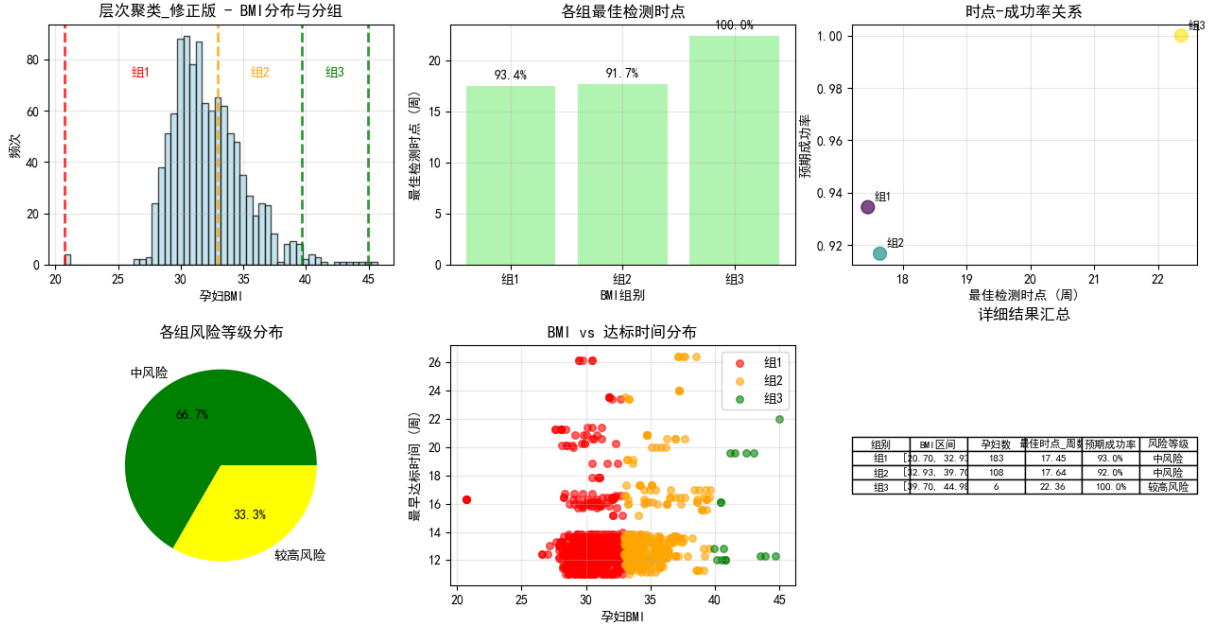


图 11 BMI 三分组与检测时点优化结果

七、问题三的模型的建立和求解

7.1 数据处理与特征工程

基于问题二的建模数据，问题三进一步引入多维度因素进行综合分析。数据预处理过程仍然按孕妇代码进行聚合，对重复检测记录进行统计汇总，生成更完整的个体特征画像。

数据聚合策略：对于同一孕妇的多次检测记录，采用差异化聚合方法：

$$F_{\text{basic}} : \{\text{年龄, 身高, 体重, BMI}\} \rightarrow \text{first()} \quad (52)$$

$$F_{\text{detect}} : \{Y_{\text{conc}}, D_{\text{current}}\} \rightarrow \{\text{mean()}, \text{std()}, \text{count()}\} \quad (53)$$

衍生特征构造：为提高模型的表达能力，构建多种衍生特征：

$$B_{\text{sq}} = (\text{BMI})^2 \quad (54)$$

$$A_{\text{sq}} = (\text{年龄})^2 \quad (55)$$

$$I_{\text{elder}} = \mathbb{I}[\text{年龄} \geq 35] \quad (56)$$

$$CV_Y = \frac{\text{std}(Y_{\text{conc}})}{\text{mean}(Y_{\text{conc}})} \quad (57)$$

$$BA_{\text{inter}} = \text{BMI} \times \text{年龄} \quad (58)$$

通过特征工程，将原始维度扩展为包含非线性项和交互项的高维特征空间，为后续聚类分析提供更丰富的信息基础。

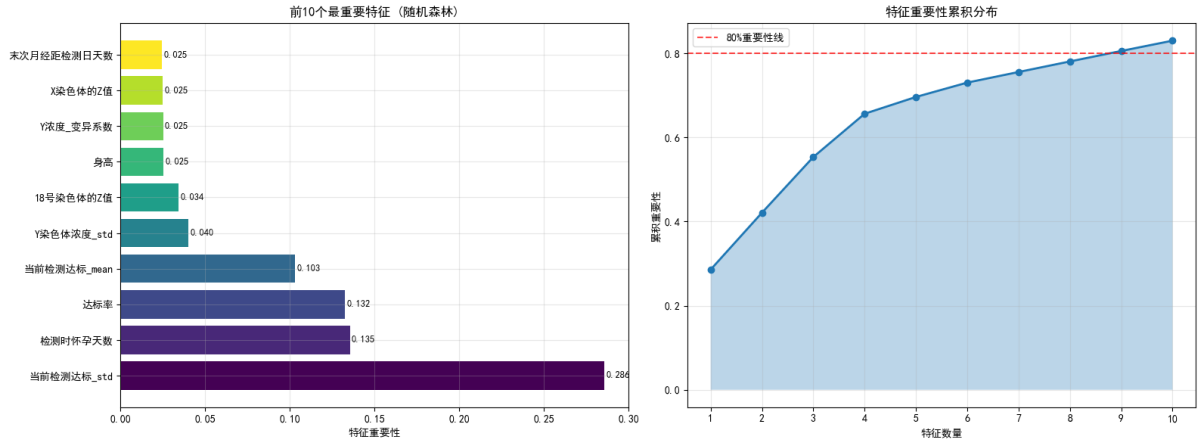


图 12 特征重要性可视化

7.2 特征重要性筛选

采用随机森林回归模型评估各特征对“最早达标时间”的预测重要性，指导聚类特征的选择。随机森林通过集成多个决策树，能够有效处理特征间的非线性关系和交互作用。构建随机森林回归模型进行重要性评估：

$$f(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}) \quad (59)$$

其中 $T_b(\mathbf{x})$ 为第 b 棵决策树， B 为树的总数。

基于 Gini 不纯度减少量计算特征重要性：

$$I_j = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b} p(t) \cdot \Delta_{\text{Gini}}(t) \cdot \mathbb{I}[v(t) = j] \quad (60)$$

其中 $p(t)$ 为节点 t 的样本权重， $\Delta_{\text{Gini}}(t)$ 为节点分裂时的不纯度减少量， $v(t)$ 为节点 t 使用的特征， j 为特征索引。结合领域知识和数据驱动的特征重要性排序，构建核心特征集合：

$$\mathcal{F}_{\text{core}} = \{\text{BMI, 年龄, 身高, 体重}\} \quad (61)$$

$$\mathcal{F}_{\text{supp}} = \{\text{当前检测达标的标准差, 检测时孕周, 达标率, 当前检测达标的均值}\} \quad (62)$$

$$\mathcal{F}_{\text{final}} = \mathcal{F}_{\text{core}} \cup \mathcal{F}_{\text{supp}} \quad (63)$$

图 12 的特征重要性分析表明，当前检测达标的波动度 (std) 贡献最大，其次是检测时怀孕天数、达标率与当前检测达标的均值，这四项合计已超过一半的重要性权重；Y 染色体浓度的波动度及 18 号染色体 Z 值提供次级增益，身高、X 染色体 Z 值、末次月经距检测日天数等影响较小。累计重要性曲线在前 8 个特征处跨越 80% 阈线，提示后续建模与决策可优先采用精简的核心特征子集，以降低维度和方差而不显著牺牲解释力。

7.3 构建模型与求解

7.3.1 多因素聚类与模型选择

采用三种不同原理的聚类算法进行多因素 BMI 分组，通过对比分析选择最优聚类方案。⁸

K-means 聚类优化：基于欧几里得距离最小化组内平方和^{9,10}：

$$\min_{\mathcal{C}} \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 \quad (64)$$

其中 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ 为聚类划分， $\boldsymbol{\mu}_i$ 为第 i 个聚类的中心。

层次聚类方法：采用 Ward 链接准则，最小化合并后的方差增加量：

$$d_{\text{Ward}}(C_i, C_j) = \sqrt{\frac{2|C_i||C_j|}{|C_i| + |C_j|}} \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|_2 \quad (65)$$

高斯混合模型：假设数据服从多元高斯混合分布，通过 EM 算法优化¹¹：

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right) \quad (66)$$

聚类评估与选择：采用轮廓系数作为主要评估指标：

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (67)$$

其中 $a(i)$ 为样本 i 与同类样本的平均距离， $b(i)$ 为样本 i 与最近异类样本的平均距离。

聚类综合分析进一步支撑了图 13 中的两组方案：KMeans 与层次聚类的轮廓系数相近且明显高于 GMM，肘部法在 $K=2$ 处出现拐点，层次树状图亦呈两簇结构；用于聚类的关键驱动因子与特征重要性结论一致，仍以当前检测达标的波动度、检测时孕周与达标率为主导。结合时点优化与分组统计，这一方案在区分度、稳定性与可操作性之间取得了良好折中，可直接据此给出组 1 集中于 24–25 周、组 2 必要时延后至约 24 周的检测安排。

针对聚类结果可能产生的 BMI 区间重叠，仍然采用重叠区间修正算法自动修正机制，修正后的区间为：

$$I_i^{\text{corr}} = [l_i, b_{\text{new}}] \quad (68)$$

$$I_j^{\text{corr}} = [b_{\text{new}}, u_j] \quad (69)$$

BMI 分组结果如图 14 所示采用双区间方案并经边界修正后互不重叠：**组 1 为 [20.70, 36.75] (n=251, 均值 31.36)**，**组 2 为 [36.75, 39.16] (n=11, 均值 38.05)**。第二组处于高 BMI 窄尾段，样本量较小但对时点建议影响显著，需要在报告中对不确定性加注释。该分组既保持了主体人群的集中性，又将高 BMI 人群单独标出，便于在时点与风险阈值上实施差异化策略。

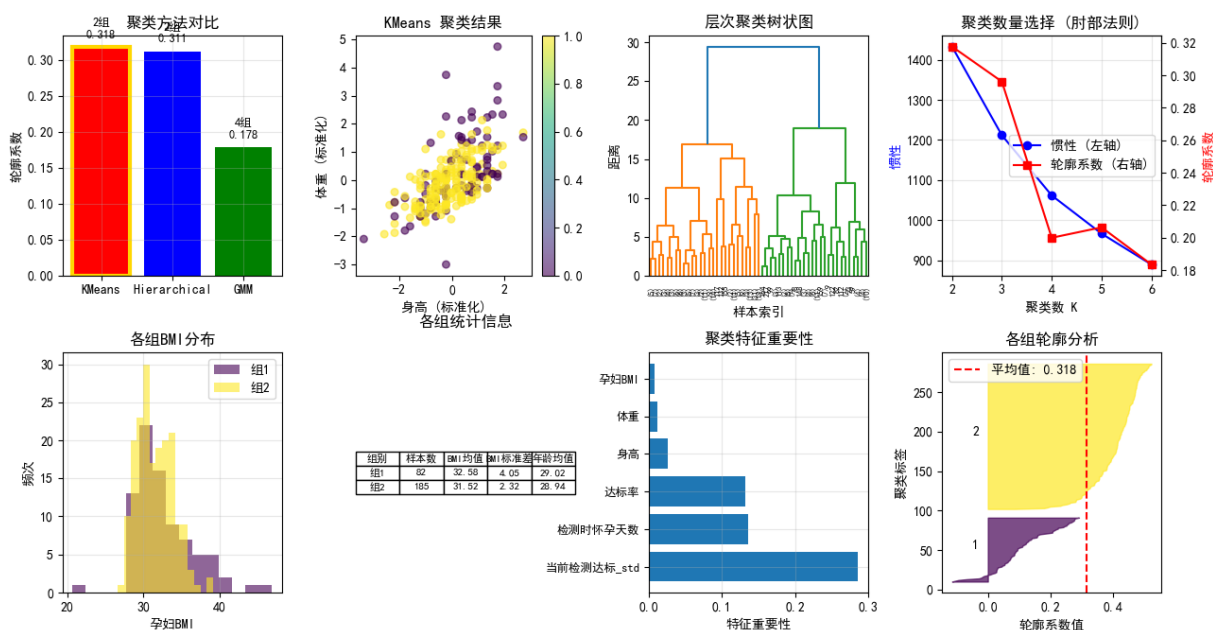


图 13 聚类过程综合分析可视化

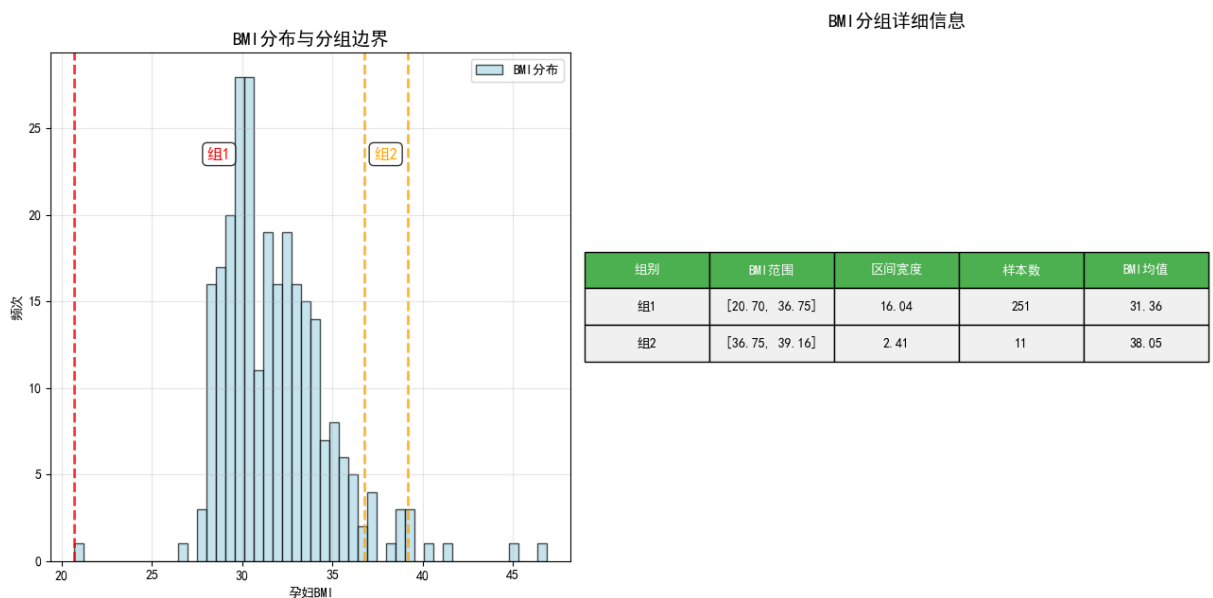


图 14 BMI 分组结果可视化

7.3.2 组内 NIPT 时点优化

基于修正后的无重叠 BMI 分组，为每组构建个性化的 NIPT 检测时点优化模型。此时转化为需要综合考虑检测时机风险和检测失败风险的多目标风险函数：

$$R_{\text{total}}(t) = R_{\text{timing}}(t) + R_{\text{failure}}(t) \quad (70)$$

其中时机风险定义为：

$$R_{\text{timing}}(t) = \begin{cases} 1.0, & t \leq 12 \times 7 \text{ (12 周内)} \\ 2.0, & 12 \times 7 < t \leq 27 \times 7 \text{ (13-27 周)} \\ 4.0, & t > 27 \times 7 \text{ (27 周后)} \end{cases} \quad (71)$$

失败风险基于预期成功率计算：

$$R_{\text{failure}}(t) = (1 - SR(t)) \times 3.0 \quad (72)$$

其中成功率函数为：

$$SR(t) = \frac{1}{N_{\text{group}}} \sum_{i=1}^{N_{\text{group}}} \mathbb{I}[t_{\text{std},i} \leq t] \quad (73)$$

由此得到约束优化模型：

$$\begin{aligned} \min_t \quad & R_{\text{total}}(t) \\ \text{s.t.} \quad & \begin{cases} 10 \times 7 \leq t \leq 30 \times 7 \\ t \in \mathbb{R}^+ \end{cases} \end{aligned} \quad (74)$$

采用有界标量优化方法求解：

$$t^* = \arg \min_{t \in [70, 210]} R_{\text{total}}(t) \quad (75)$$

采取下述性能评估指标：

$$SR_{\text{exp}} = \frac{\sum_{i=1}^N \mathbb{I}[t_{\text{std},i} \leq t^*]}{N} \quad (76)$$

$$DT_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N \max(0, t_{\text{std},i} - t^*) \quad (77)$$

$$RL = \begin{cases} \text{低风险,} & t^* \leq 84 \\ \text{中风险,} & 84 < t^* \leq 189 \\ \text{高风险,} & t^* > 189 \end{cases} \quad (78)$$

其中， SR_{exp} 是各 BMI 分组在最优检测时点下的预期成功率，是策略优劣的核心判断指标之一； R_{total} 为总体风险评价函数，用于统一衡量优化策略的效果。这些变量构成了从特征建模到风险评估的主要数值框架。

7.4 择优求解分析

组别时点优化图 15 显示，两组的目标函数在 24–26 周附近达到全局最小，其中**组 1**的最优检测时点为 **25.1 周**、**组 2** 为 **24.2 周**；相应的成功率曲线在该区间迅速爬升并接

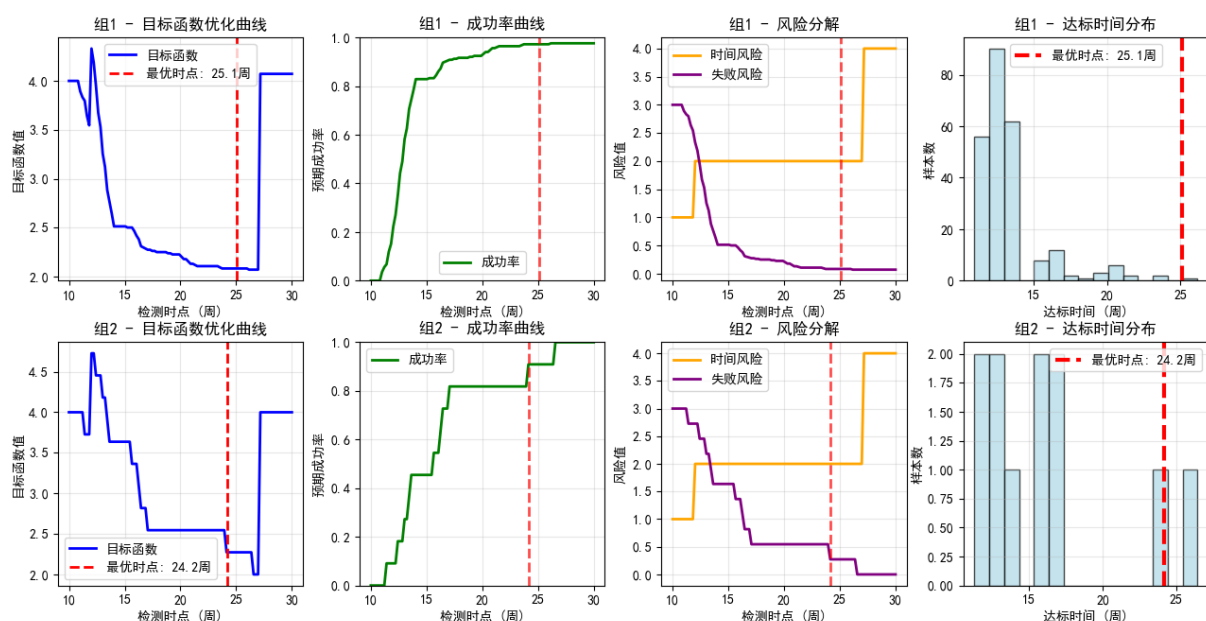


图 15 时点优化过程可视化

近 1，风险分解曲线则呈现出“早期失败风险高、随孕周下降；过晚则时间风险上升”的交叉特征，因而最优时点本质上是在“降低失败风险”和“避免过长等待”之间取得平衡。达标时间直方图集中在 13–18 周，说明多数个体较早具备达标条件，但推迟到 24–25 周能显著提升整体成功率，这与目标函数的极小值位置一致。

7.5 误差敏感性分析

为评估最优检测时点在实际执行误差下的稳健性，采用扰动敏感性模拟，构建 ± 7 天扰动范围内的成功率曲线与偏差指标，检验策略对时间误差的容忍度。

对每组最优检测时点 t^* ，引入误差扰动 $\delta \in \{-7, -3, 0, +3, +7\}$ ，计算扰动后成功率 $SR(t^* + \delta)$ ，绘制热力图与折线图以展示成功率稳定性，并以波动幅度（最大值与最小值差）衡量敏感等级。

组 1 在全部扰动时点下成功率始终维持在 0.972，无任何变化，表现出极高的时间鲁棒性，热力图为纯绿色，曲线呈水平，波动幅度为 0.00%，判定为低敏感。组 2 在提前 7 天时成功率下降至 0.818，而基准及延后情形均为 0.909，波动幅度为 9.09%，虽然仍在低敏感阈值（10%）以内，但显著受负向扰动影响，热力图右侧偏暖，敏感性曲线呈浅谷结构，提示应避免检测过早。

综合图 16 结果显示，组 1 具备良好的操作弹性，可在 ± 7 天范围内执行而不致损失策略有效性；组 2 则建议以最优时点或略延后进行安排，避免提前一周的检测计划，以维持成功率在 0.90 以上。

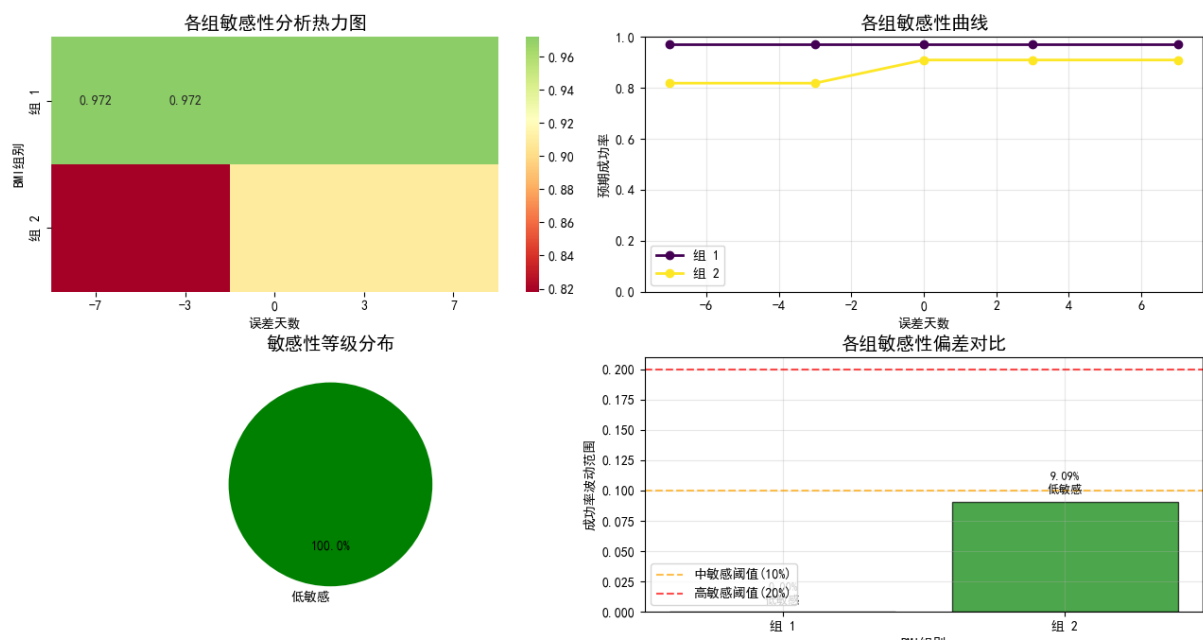


图 16 检验误差敏感性分析可视化

八、问题四的模型的建立和求解

8.1 数据预处理

与前三问不同，这一问中主要对附件数据中女胎部分的数据进行处理，在女胎数据中，需要重新对女胎是否异常进行判断，故对以下数据进行了预处理：

表 4 问题四女胎数据变量转换处理汇总表

原始变量	转换后变量	转换规则
末次月经 + 检测日期	末次月经距离检测日天数	计算检测日期 (YYYYMMDD 格式) 与末次月经日期的天数差值，处理 Excel 日期序列号转换
检测孕周	检测时怀孕天数	解析孕周格式 (如"13w+5"), 转换为总天数: 周数 × 7 + 额外天数
IVF 妊娠	IVF 妊娠_编码	二元编码: 自然受孕为 0, IVF (试管婴儿) 为 1
染色体的非整倍体	T13_异常、T18_异常、T21_异常	通过字符串匹配检测是否包含"T13"、"T18"、"T21"标记，分别生成对应的二元异常指示变量
染色体异常检测结果	胎儿健康_编码	基于染色体异常重新定义: T13、T18、T21 均正常为 1 (健康), 否则为 0 (不健康), 忽略原始健康状态列

8.2 模型介绍

针对女胎染色体异常判定这一二分类问题，本研究采用多种机器学习算法进行对比分析，以找到最适合临床应用的预测模型¹²。选择的模型涵盖了线性、非线性、集成学习等不同类型的算法。

8.2.1 逻辑回归模型

逻辑回归通过 Sigmoid 函数将线性回归的输出映射到 $[0,1]$ 区间，适合处理二分类问题。对于女胎异常判定，逻辑回归能够提供概率解释，便于临床决策。⁵ 设特征向量为 $\mathbf{x} = (x_1, x_2, \dots, x_p)^T$ ，逻辑回归模型定义为：

$$P(Y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta^T \mathbf{x})}} \quad (79)$$

其中 β_0 为截距项， $\beta = (\beta_1, \beta_2, \dots, \beta_p)^T$ 为回归系数向量。

采用最大似然估计方法进行参数估计，似然函数为：

$$L(\beta) = \prod_{i=1}^n [P(Y = 1|\mathbf{x}_i)]^{y_i} [1 - P(Y = 1|\mathbf{x}_i)]^{1-y_i} \quad (80)$$

对数似然函数为：

$$\ell(\beta) = \sum_{i=1}^n [y_i \log P(Y = 1|\mathbf{x}_i) + (1 - y_i) \log(1 - P(Y = 1|\mathbf{x}_i))] \quad (81)$$

8.2.2 随机森林模型

随机森林是一种集成学习算法，通过构建多个决策树并投票决策来提高预测精度和模型稳定性。对于染色体异常判定这类高维特征问题，随机森林能够有效处理特征间的复杂关系。⁵

随机森林训练与预测步骤：

- Step 1 从原始训练集中进行 Bootstrap（有放回）抽样，生成 B 个训练子集。
 - Step 2 对每个子集，在随机选择的 m 个特征上构建一棵决策树（每次分裂仅在这 m 个特征中择优）。
 - Step 3 重复步骤 1-2，得到 B 棵相互独立的决策树，形成森林。
 - Step 4 对新样本进行预测时汇总所有树的输出：分类取多数投票，回归取平均值。
-

对于二分类问题，预测结果为：

$$\hat{y} = \begin{cases} 1, & \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}) > 0.5 \\ 0, & \text{否则} \end{cases} \quad (82)$$

其中 $T_b(\mathbf{x})$ 为第 b 棵树对样本 $\mathbf{x}$ 的预测结果。

8.2.3 SVM 模型

支持向量机通过寻找最优分离超平面来实现分类，在高维空间中具有良好的泛化能力。对于染色体 Z 值等连续特征，SVM 能够有效捕捉复杂的非线性关系。⁵

在线性可分情况下，寻找最大间隔分离超平面：

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad (83)$$

$$\text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \quad (84)$$

引入松弛变量处理线性不可分情况：

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \mathbf{w}^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & \begin{cases} y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \\ \xi_i \geq 0, \quad i = 1, \dots, n \end{cases} \end{aligned} \quad (85)$$

其中 C 为正则化参数，控制对误分类的惩罚程度。

采用径向基函数 (RBF) 核处理非线性问题：

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (86)$$

其中 γ 为核参数，控制决策边界的复杂度。

最终的分类决策函数为：

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \right) \quad (87)$$

其中 SV 为支持向量集合， α_i 为拉格朗日乘子。

8.3 模型构建与结果分析

8.3.1 特征工程的构建

基于女胎染色体异常判定的医学特点，构建了包含多个层次的综合特征体系。特征工程遵循医学诊断的逻辑，重点关注染色体 Z 值等核心指标。

染色体 Z 值相关特征：

$$\text{Z 值绝对值：} \quad |Z_{chr}| \quad (\text{chr} \in \{13, 18, 21\}) \quad (88)$$

$$\text{最大 Z 值绝对值：} \quad \max\{|Z_{13}|, |Z_{18}|, |Z_{21}|\} \quad (89)$$

$$\text{平均 Z 值绝对值：} \quad \frac{1}{3}(|Z_{13}| + |Z_{18}| + |Z_{21}|) \quad (90)$$

$$\text{Z 值标准差：} \quad \sqrt{\frac{1}{3} \sum_{chr} (|Z_{chr}| - \bar{Z})^2} \quad (91)$$

表 5 核心数值特征 (27 项) 汇总

类别	变量 (示例)	数量
染色体 Z 值特征	Z_{13} 、 Z_{18} 、 Z_{21} 、 Z_X (核心诊断信号)	4
基本信息特征	年龄、身高、体重、BMI、生产次数	5
检测技术特征	原始读段数、比对比例、重复读段比例、唯一比对读段数、被过滤比例、GC 含量等测序质量指标	9
妊娠相关特征	检测抽血次数、怀孕天数 (检测时孕周)、IVF 状态等	6
衍生特征构造	基于医学知识构建多种工程特征	3
合计	以上五类合并构成基础特征集	27

异常阈值统计特征：

$$\text{异常计数}_{2.5} : \sum_{chr} \mathbb{I}[|Z_{chr}| \geq 2.5] \quad (92)$$

$$\text{异常计数}_{3.0} : \sum_{chr} \mathbb{I}[|Z_{chr}| \geq 3.0] \quad (93)$$

BMI 交互特征：

$$\text{BMI_Z 交互项} : \text{BMI} \times |Z_{chr}|, \quad chr \in \{13, 18, 21\} \quad (94)$$

测序质量综合评分：

$$\text{测序质量得分} : r \left(1 - |g - \frac{1}{2}|\right) \quad (95)$$

其中 $r \in [0, 1]$ 表示比对比例, $g \in [0, 1]$ 为 GC 含量。

8.3.2 模型的建立

针对女胎染色体异常判定任务, 本研究采用系统性的建模策略, 通过多算法对比与多种标准化方法组合, 构建稳健的分类预测模型。整个建模过程包含数据分割、标准化处理、模型配置、交叉验证和性能评估五个核心环节。

数据分割与验证策略：采用分层抽样方法进行训练集与测试集划分, 确保两个集合中正负样本比例保持一致：

$$\text{train_test_split}(X, y, \text{test_size} = 0.2, \text{stratify} = y, \text{random_state} = 42) \quad (96)$$

最终训练集包含 80% 样本用于模型训练与参数调优, 测试集包含 20% 样本用于最终性能验证。同时采用 5 折分层交叉验证评估模型稳定性：

$$\text{CV} = \text{StratifiedKfold}(n_splits = 5, \text{shuffle} = \text{True}, \text{random_state} = 42) \quad (97)$$

多标准化方法对比：考虑到不同特征的量纲差异对模型性能的影响，引入三种标准化方法进行系统对比：

标准化（StandardScaler）基于均值和标准差的 Z-score 标准化：

$$x_{\text{std}} = \frac{x - \mu}{\sigma} \quad (98)$$

稳健标准化（RobustScaler）基于中位数和四分位距，对异常值具有更强的鲁棒性：

$$x_{\text{robust}} = \frac{x - \text{median}}{Q_{75} - Q_{25}} \quad (99)$$

最小-最大标准化（MinMaxScaler）线性缩放到 [0,1] 区间：

$$x_{\text{minmax}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (100)$$

模型参数设置：针对女胎染色体异常判定这一二分类问题，构建了包含 9 种不同原理算法的模型库，涵盖线性、非线性、集成学习等多种方法。各算法的具体配置如表 6 所示。

表 6 分类算法模型参数配置

模型类别	算法名称	主要参数设置
线性模型类	逻辑回归	class_weight='balanced', solver='liblinear', max_iter=1000
	线性判别分析	默认参数，基于类间方差最大化的线性降维分类
集成学习类	随机森林	n_estimators=100, max_depth=10, min_samples_split=10
	梯度提升	learning_rate=0.1, n_estimators=100, max_depth=6
非线性模型类	支持向量机	kernel='rbf', C=1.0, gamma='scale', probability=True
	决策树	max_depth=8, min_samples_split=20
	K 近邻	n_neighbors=5, weights='uniform'
概率模型类	高斯朴素贝叶斯	默认参数，假设特征服从高斯分布
	多层感知机	hidden_layer_sizes=(50,25), max_iter=300, early_stopping=True

所有模型均采用 class_weight='balanced' 策略处理样本不平衡问题，确保对少数类（异常样本）的识别能力。集成学习方法通过 Bootstrap 聚合和梯度提升机制降低过拟合风险，而非线性模型则通过限制模型复杂度（如最大深度、最小分裂样本数）来平衡拟合能力与泛化性能。概率模型类采用相对保守的参数设置，多层感知机启用早停机制防止过训练。

模型训练与选择流程：对每种算法与标准化方法的组合进行系统评估，总计 $9 \times 3 = 27$ 种配置。训练流程如下：

首先，对训练数据应用相应的标准化方法：

$$X_{\text{train_scaled}} = \text{Scaler.fit_transform}(X_{\text{train}}) \quad (101)$$

然后进行交叉验证评估：

$$\text{CV_scores} = \text{cross_val_score}(\text{model}, X_{\text{train_scaled}}, y_{\text{train}}, \text{cv} = \text{CV}, \text{scoring} = 'roc_auc') \quad (102)$$

最后在测试集上进行最终验证：

$$X_{\text{test_scaled}} = \text{Scaler.transform}(X_{\text{test}}) \quad (103)$$

性能评估指标体系：构建多维度评估指标以全面衡量模型性能：

主要指标包括：

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(x)) dx \quad (104)$$

$$\text{敏感性} = \frac{TP}{TP + FN} \quad (105)$$

$$\text{特异性} = \frac{TN}{TN + FP} \quad (106)$$

$$\text{F1 得分} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (107)$$

其中 TP 、 TN 、 FP 、 FN 分别表示真阳性、真阴性、假阳性、假阴性样本数。

稳健性处理机制：为确保建模过程的稳健性，实施多层次的异常处理策略：

数据质量控制方面，强制进行数值类型转换来处理混合数据类型问题：

$$X[\text{col}] = \text{pd.to_numeric}(X[\text{col}], \text{errors} = 'coerce') \quad (108)$$

对无穷大和 NaN 值进行检测与替换：

$$X[\text{col}] = X[\text{col}].\text{replace}([\infty, -\infty], \text{np.nan}).\text{fillna}(X[\text{col}].\text{median}()) \quad (109)$$

模型训练容错方面，采用 Try-catch 异常捕获机制，确保单个模型失败不影响整体流程，并通过设置 $n_jobs = 1$ 避免多进程竞争问题。

通过”模型 $\times$ 标准化方法”的组合策略系统性对比，最终基于 AUC 指标选择最优模型配置。根据交叉验证稳定性与测试集表现，优先选择在多种标准化方法下均表现良好的模型，确保方案的实用性与可靠性。

8.3.3 结果分析与评价

采用稳健的模型构建流程，通过多种算法对比和多种数据标准化方法组合，确保结果的可靠性和临床适用性。

通过对 9 种机器学习算法与 3 种标准化方法的系统性对比，逻辑回归结合稳健标准化取得了最佳综合性能，AUC 达到 0.8782，在敏感性（0.7593）与特异性（0.6923）之间实现了较好的平衡。集成学习方法如随机森林和梯度提升虽然在敏感性方面表现突出（接近 1.0），但特异性偏低，可能导致过多的假阳性预测。值得注意的是，稳健标准化在多个线性模型中表现最优，表明该方法对染色体 Z 值等可能存在异常值的医学特征具有更好的适应性。

表 7 分类模型配置与实测性能汇总

类别	算法	最佳标准化	AUC	敏感性	特异性	F1
集成学习类	随机森林 (RF)	标准化	0.8604	1.0000	0.0769	0.8609
集成学习类	梯度提升 (GBDT)	标准化	0.8298	0.9907	0.1538	0.8720
线性模型类	逻辑回归 (LR)	稳健标准化	0.8782	0.7593	0.6923	0.7948
线性模型类	线性判别分析 (LDA)	稳健标准化	0.8405	1.0000	0.3846	0.9204
非线性模型类	径向基核 SVM (RBF-SVM)	标准化	0.8554	0.7963	0.7692	0.8270
非线性模型类	决策树 (DT)	稳健标准化	0.7037	0.6759	0.6154	0.7313
非线性模型类	K 近邻 (KNN)	归一化	0.7016	0.9815	0.0000	0.8336
概率模型类	高斯朴素贝叶斯 (GNB)	稳健标准化	0.7856	0.9167	0.3077	0.8512
概率模型类	多层感知机 (MLP)	归一化	0.5869	1.0000	0.0000	0.8419
最佳模型为 LR+ 稳健标准化（AUC=0.8782）。						

九、模型的评价

9.1 模型的优点

- 优点 1：本研究在问题一中系统构建了以胎儿 Y 染色体浓度为因变量的多元回归与正则化模型，结合了基础体格、基因测序特征与生物信息交互项，利用 OLS 与岭回归有效识别关键影响因子，并对非正态与异方差问题进行稳健性修正，模型在解释力与泛化能力间取得良好平衡。
- 优点 2：在问题二与问题三中，针对 NIPT 检测成功率与风险控制的实际需求，提出了一套可推广的“BMI 分组 + 检测时点优化”策略，集成了 DBSCAN、层次聚类、GMM、决策树与风险最优化分箱等多种算法，充分挖掘孕妇特征与检测时机间

的潜在结构，有效提升了检测效率与成功率。

- 优点 3：问题三引入蒙特卡罗误差传播分析框架，模拟测量扰动对分组与检测策略的影响，量化策略的鲁棒性与敏感性，增强了模型在实际操作条件下的可实施性与容错能力。

- 优点 4：问题四面向染色体异常识别任务，构建了多模型对比体系，涵盖逻辑回归、随机森林、SVM 等典型方法，通过特征工程构造 27 项核心变量，模型性能评估以 AUC、敏感性、特异性等多指标展开，最终选择逻辑回归 + 稳健标准化策略，兼顾临床解读与技术准确性。

9.2 模型的缺点

- 缺点 1：部分模型（如问题一中的 OLS 回归）在误差项异方差与非正态性方面表现出较强违背，虽然后续采用 Ridge 与对数变换进行修正，但仍存在尾部拟合不足的问题，可能影响极端样本的预测稳定性。

- 缺点 2：问题二与问题三中的聚类与风险优化模型依赖多个超参数（如聚类数 K 、分箱权重 α, β 、延迟惩罚权重等），其选取多基于启发式或交叉验证，缺乏系统的超参数敏感性分析与理论解释。

- 缺点 3：问题四中类别不平衡较为严重，虽然引入了类权重与阈值调优手段，但在特异性表现上仍有限，尤其在部分模型中（如 KNN、MLP）存在对负类预测能力不足的问题，需进一步通过代价敏感学习或重采样方法增强模型稳健性。

9.3 模型的改进与推广

- 数据融合与参数校准：后续可结合临床随访数据、测序实验日志与医院数据仓库，通过贝叶斯更新或机器学习方法对预测模型中的关键参数（如 Z 值阈、成功率模型）进行动态校准，增强模型的时效性与泛化能力。

- 检测流程联动与临床集成：可将检测时点优化模型嵌入产检预约系统，实现检测推荐、孕妇筛查与干预建议的自动化；同时在电子病历系统中集成模型评分与达标提示，提高医生与患者的共同决策效率。

- 泛化扩展与人群适配：当前模型主要基于单中心数据，未来可通过多中心数据扩展适配不同人群（如高龄孕妇、合并疾病群体），并探索性别、中介变量（如胎盘厚度）对模型性能的影响，提升其在人群异质性下的鲁棒性。

参考文献

- [1] 中国妇幼保健协会生育保健分会. 新一代无创产前筛查技术 NIPT2.0 临床应用策略专家共识[EB/OL]. 2024. <https://rs.yiigle.com/cmaid/1517120>.
- [2] NORTON M E, JACOBSSON B, SWAMY G K, et al. Cell-free dna analysis for non-invasive examination of trisomy[EB/OL]. 2015. <https://www.nejm.org/doi/pdf/10.1056/NEJMoA1407349>.
- [3] SPEARMAN C. The proof and measurement of association between two things[EB/OL]. 1904. <https://gwern.net/doc/psychology/1904-spearman.pdf>.
- [4] 田甜. 基于 Spearman 秩相关系数法分析水磨河水质变化趋势[EB/OL]. 2019. <https://qikan.cqvip.com/Qikan/Article/Detail?id=7003053345>.
- [5] Anthropic. Claude sonnet 4,anthropic[EB/OL]. 2025-09-07. <https://www.anthropic.com>.
- [6] ESTER M, KRIEGEL H P, SANDER J, et al. A density-based algorithm for discovering clusters in large spatial databases with noise (dbscan)[EB/OL]. 1996. <https://cdn.aaai.org/KDD/1996/KDD96-037.pdf>.
- [7] 李新延, 李德仁. DBSCAN 空间聚类算法及其在城市规划中的应用[EB/OL]. 2005. <https://xueshu.baidu.com/usercenter/paper/show?paperid=6160d41561fb64732e2d810e1fe685f6>.
- [8] OpenAI. Chatgpt-4o,openai[EB/OL]. 2025-09-07. <https://openai.com/chatgpt>.
- [9] LLOYD S P. Least squares quantization in pcm[EB/OL]. 1982. <https://pzs.dstu.dp.ua/ComputerGraphics/ic/bibl/lloyd-1982.pdf>.
- [10] 李贞, 刘海燕, 刘策, 等. 基于机器学习的 K-Means 聚类优化算法研究[EB/OL]. 2022. https://pdf.hanspub.org/HJDM20220100000_18145135.pdf.
- [11] REYNOLDS D A. Gaussian mixture models[EB/OL]. 2009. <https://www.ll.mit.edu/r-d/publications/gaussian-mixture-models>.
- [12] 佟佳益, 郑改改, 王宇, 等. 医学人工智能研究热点双聚类分析[EB/OL]. 2023. <https://jcmp.yzu.edu.cn/article/doi/10.7619/jcmp.20232042>.
- [13] OpenAI. Chatgpt-5 thinking,openai[EB/OL]. 2025-09-07. <https://openai.com/chatgpt>.

附录 A 文件列表

文件名	功能描述
数据描述性统计分析.py	数据预处理与概览可视化（男胎）
问题 1 数据预处理.py	问题一：数据清洗、变量构造与相关性结果导出
问题 1 相关性分析与假设检验.py	问题一：分布与相关性可视化、正态性/假设检验
多元线性回归详细解释和可视化.py	问题一：多元线性回归建模与诊断
问题 1 岭回归.py	问题一：岭回归（Ridge）建模与特征分析
问题 2 数据预处理.py	问题二：风险指标、最早达标时间等派生变量
问题 2.py	问题二：BMI 分组方法（GMM/层次/DBSCAN/优化分箱/决策树）
问题 3.py	问题三：多因素综合分析与聚类分组、可视化
问题 4 数据预处理与可视化.py	问题四：女胎数据处理与可视化
问题 4.py	问题四：分类模型对比评估（RF/GBDT/SVM 等）

附录 B 代码

数据描述性统计分析.py

```
1 import pandas as pd
2 import numpy as np
3 from datetime import datetime, timedelta
4 import re
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 import warnings
8 import os
9 warnings.filterwarnings('ignore')
10
11 # 设置中文字体
12 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong']
13 plt.rcParams['axes.unicode_minus'] = False
14 plt.rcParams['font.size'] = 10
15 # 文件读取和修改变量
```

```

16 def preprocess_male_nipt_data(file_path, enable_visualization=True):
17     df = pd.read_excel(file_path, sheet_name='男胎检测数据')
18     processed_df = df.copy()
19     #将末次月经日期+检测日期转化变量为末次月经距检测日天数
20     def calculate_days_from_lmp(row):
21         try:
22             lmp_date = pd.to_datetime(row['末次月经'])
23             detection_date_str = str(int(row['检测日期']))
24             detection_date = pd.to_datetime(detection_date_str, format='%Y%m%d')
25             days_diff = (detection_date - lmp_date).days
26             return days_diff
27         except:
28             return np.nan
29     processed_df['末次月经距检测日天数'] = processed_df.apply(
30         calculate_days_from_lmp, axis=1)
31
32     # IVF妊娠转换为分类变量，创建数值编码和标签映射
33     ivf_mapping = {
34         '自然受孕': 0,
35         'IUI (人工授精)': 1,
36         'IVF (试管婴儿)': 2
37     }
38     processed_df['IVF妊娠_编码'] = processed_df['IVF妊娠'].map(ivf_mapping)
39     # 改为二进制形式 (1和0的数值格式)
40     ivf_dummies = pd.get_dummies(processed_df['IVF妊娠'], prefix='IVF', dtype=int)
41     processed_df = pd.concat([processed_df, ivf_dummies], axis=1)
42
43     # 将检测孕周转换为检测时怀孕天数
44     def parse_gestational_weeks(week_str):
45         try:
46             if pd.isna(week_str):
47                 return np.nan
48             week_str = str(week_str).strip()
49             if 'w' in week_str:
50                 if '+' in week_str:
51                     weeks_part, days_part = week_str.split('+')
52                     weeks = int(weeks_part.replace('w', ''))
53                     days = int(days_part)
54                 else:
55                     weeks = int(week_str.replace('w', ''))
56                     days = 0
57                 total_days = weeks * 7 + days
58                 return total_days
59             else:
60                 return np.nan
61         except:
62             return np.nan

```

```

62
63 processed_df['检测时怀孕天数'] = processed_df['检测孕周'].apply(
    parse_gestational_weeks)
64
65 # 将染色体非整倍体信息处理为T13、T18、T21异常标记
66 def parse_chromosome_abnormality(abnormality_str):
67     # 初始化结果
68     result = {'T13': 0, 'T18': 0, 'T21': 0}
69     if pd.isna(abnormality_str) or abnormality_str == '' or abnormality_str is
    None:
70         return result
71     abnormality_str = str(abnormality_str).strip()
72     # 检查是否包含各种异常标记
73     if 'T13' in abnormality_str:
74         result['T13'] = 1
75     if 'T18' in abnormality_str:
76         result['T18'] = 1
77     if 'T21' in abnormality_str:
78         result['T21'] = 1
79     return result
80 chromosome_results = processed_df['染色体的非整倍体'].apply(
    parse_chromosome_abnormality)
81 processed_df['T13_异常'] = [result['T13'] for result in chromosome_results]
82 processed_df['T18_异常'] = [result['T18'] for result in chromosome_results]
83 processed_df['T21_异常'] = [result['T21'] for result in chromosome_results]
84
85 #怀孕次数转换为分类变量，创建数值编码和二进制形式
86 def categorize_pregnancy_count(count):
87     if pd.isna(count):
88         return np.nan
89     if count == 1:
90         return '初产妇'
91     elif count == 2:
92         return '经产妇_2次'
93     else:
94         return '经产妇_多次'
95
96 processed_df['怀孕次数_分类'] = processed_df['怀孕次数'].apply(
    categorize_pregnancy_count)
97 pregnancy_mapping = {
98     '初产妇': 0,
99     '经产妇_2次': 1,
100     '经产妇_多次': 2
101 }
102 processed_df['怀孕次数_编码'] = processed_df['怀孕次数_分类'].map(
    pregnancy_mapping)
103 pregnancy_dummies = pd.get_dummies(processed_df['怀孕次数_分类'], prefix='怀孕

```



```

次数', dtype=int)
1104 processed_df = pd.concat([processed_df, pregnancy_dummies], axis=1)
1105 # 胎儿健康状况转换为二分类变量, 1表示健康, 0表示不健康
1106 health_mapping = {
1107     '是': 1,
1108     '否': 0
1109 }
1110 processed_df['胎儿健康_编码'] = processed_df['胎儿是否健康'].map(health_mapping)
1111 # 检查新生成字段的缺失值情况
1112 new_columns = ['末次月经距检测日天数', 'IVF妊娠_编码', '检测时怀孕天数',
1113                'T13_异常', 'T18_异常', 'T21_异常', '怀孕次数_编码', '胎儿健康_
1114 编码']
1115 missing_info = {}
1116 for col in new_columns:
1117     if col in processed_df.columns:
1118         missing_count = processed_df[col].isnull().sum()
1119         missing_rate = missing_count / len(processed_df)
1120         missing_info[col] = {'缺失数量': missing_count, '缺失比例': f"{
1121 missing_rate:.3f}"}
1122
1123 missing_df = pd.DataFrame(missing_info).T
1124 # [8] AI协助添加过程可视化使其更具可解释性
1125 if enable_visualization:
1126     # 1. 数据概览可视化
1127     create_data_overview_plots(processed_df)
1128
1129     # 2. 分类变量分布可视化
1130     create_categorical_distribution_plots(processed_df)
1131
1132     # 3. 连续变量分布可视化
1133     create_continuous_distribution_plots(processed_df)
1134
1135     # 4. 数据质量分析可视化
1136     create_data_quality_plots(processed_df, missing_df)
1137
1138     # 5. 相关性初步分析可视化
1139     create_preliminary_correlation_plots(processed_df)
1140 # 选择关键字段用于建模
1141 key_columns = [
1142     # 原始标识字段
1143     '序号', '孕妇代码', '年龄', '身高', '体重', '孕妇BMI',
1144
1145     # 预处理后的字段
1146     '末次月经距检测日天数', 'IVF妊娠_编码', '检测时怀孕天数',
1147     'T13_异常', 'T18_异常', 'T21_异常',
1148     '怀孕次数_编码', '胎儿健康_编码',

```

```

147
148     # 重要的检测指标
149     'Y染色体浓度', 'Y染色体的Z值',
150     '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
151     'X染色体的Z值', 'GC含量',
152
153     # 哑变量
154     'IVF_自然受孕', 'IVF_IUI (人工授精)', 'IVF_IVF (试管婴儿)',
155     '怀孕次数_初产妇', '怀孕次数_经产妇_2次', '怀孕次数_经产妇_多次'
156 ]
157
158 # 只保留存在的列
159 available_columns = [col for col in key_columns if col in processed_df.columns]
160 final_df = processed_df[available_columns].copy()
161 return final_df, processed_df
162 # 数据概览图表
163 def create_data_overview_plots(df):
164     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
165     fig.suptitle('数据预处理概览', fontsize=16, fontweight='bold')
166     # 1. 样本量统计
167     sample_info = {
168         '总样本数': len(df),
169         '完整样本': len(df.dropna()),
170         '有Y染色体数据': len(df.dropna(subset=['Y染色体浓度'])) if 'Y染色体浓度' in
171         df.columns else 0
172     }
173     bars1 = axes[0, 0].bar(sample_info.keys(), sample_info.values(),
174                             color=['skyblue', 'lightgreen', 'orange'], alpha=0.8)
175     axes[0, 0].set_title('样本量统计')
176     axes[0, 0].set_ylabel('样本数')
177     for bar, value in zip(bars1, sample_info.values()):
178         axes[0, 0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
179                         f'{value}', ha='center', va='bottom', fontweight='bold')
180     # 2. 年龄分布
181     if '年龄' in df.columns:
182         age_data = df['年龄'].dropna()
183         axes[0, 1].hist(age_data, bins=20, color='lightcoral', alpha=0.7, edgecolor
184                         ='black')
185         axes[0, 1].axvline(age_data.mean(), color='red', linestyle='--', linewidth
186                             =2,
187                             label=f'均值: {age_data.mean():.1f}岁')
188         axes[0, 1].set_title('孕妇年龄分布')
189         axes[0, 1].set_xlabel('年龄 (岁)')
190         axes[0, 1].set_ylabel('频数')
191         axes[0, 1].legend()
192         axes[0, 1].grid(True, alpha=0.3)

```

```

191 # 3. BMI分布
192 if '孕妇BMI' in df.columns:
193     bmi_data = df['孕妇BMI'].dropna()
194     axes[1, 0].hist(bmi_data, bins=20, color='lightblue', alpha=0.7, edgecolor=
'black')
195     # 添加BMI分类线
196     axes[1, 0].axvline(18.5, color='blue', linestyle=':', label='偏瘦')
197     axes[1, 0].axvline(24, color='green', linestyle=':', label='正常')
198     axes[1, 0].axvline(28, color='orange', linestyle=':', label='超重')
199     axes[1, 0].axvline(bmi_data.mean(), color='red', linestyle='--', linewidth
=2,
200                        label=f'均值: {bmi_data.mean():.1f}')
201     axes[1, 0].set_title('孕妇BMI分布')
202     axes[1, 0].set_xlabel('BMI')
203     axes[1, 0].set_ylabel('频数')
204     axes[1, 0].legend()
205     axes[1, 0].grid(True, alpha=0.3)
206 # 4. Y染色体浓度分布
207 if 'Y染色体浓度' in df.columns:
208     y_data = df['Y染色体浓度'].dropna()
209     axes[1, 1].hist(y_data, bins=30, color='gold', alpha=0.7, edgecolor='black'
)
210     axes[1, 1].axvline(y_data.mean(), color='red', linestyle='--', linewidth=2,
211                        label=f'均值: {y_data.mean():.4f}')
212     axes[1, 1].set_title('Y染色体浓度分布')
213     axes[1, 1].set_xlabel('Y染色体浓度')
214     axes[1, 1].set_ylabel('频数')
215     axes[1, 1].legend()
216     axes[1, 1].grid(True, alpha=0.3)
217 else:
218     axes[1, 1].text(0.5, 0.5, 'Y染色体浓度数据\n不可用', ha='center', va='
center',
219                    transform=axes[1, 1].transAxes, fontsize=14)
220     axes[1, 1].set_title('Y染色体浓度分布')
221 plt.tight_layout()
222 plt.savefig('数据预处理_概览分析.png', dpi=300, bbox_inches='tight', facecolor=
'white')
223 plt.show()
224 #分类变量分布图表
225 def create_categorical_distribution_plots(df):
226     # 定义分类变量
227     categorical_vars = {
228         'IVF妊娠': 'IVF妊娠方式',
229         '怀孕次数_分类': '怀孕次数分类',
230         '胎儿是否健康': '胎儿健康状况'
231     }
232     # 添加染色体异常变量

```

```

233 chromosome_vars = ['T13_异常', 'T18_异常', 'T21_异常']
234 available_vars = {k: v for k, v in categorical_vars.items() if k in df.columns}
235 available_chromosomes = [var for var in chromosome_vars if var in df.columns]
236
237 fig, axes = plt.subplots(2, 2, figsize=(15, 12))
238 fig.suptitle('分类变量分布分析', fontsize=16, fontweight='bold')
239 axes = axes.flatten()
240 plot_idx = 0
241 # 绘制常规分类变量
242 for var, title in available_vars.items():
243     if plot_idx >= 4:
244         break
245     value_counts = df[var].value_counts(dropna=False)
246     # 饼图
247     colors = plt.cm.Set3(np.linspace(0, 1, len(value_counts)))
248     wedges, texts, autotexts = axes[plot_idx].pie(value_counts.values,
249                                                    labels=value_counts.index,
250                                                    autopct='%1.1f%%',
251                                                    colors=colors,
252                                                    startangle=90)
253
254     axes[plot_idx].set_title(title)
255     # 添加数量标注
256     for i, (label, count) in enumerate(value_counts.items()):
257         label_str = str(label) if not pd.isna(label) else '缺失'
258         axes[plot_idx].text(0.8, 0.8-i*0.1, f'{label_str}: {count}例',
259                             transform=axes[plot_idx].transAxes, fontsize=9,
260                             bbox=dict(boxstyle='round,pad=0.3', facecolor='white
261 ', alpha=0.8))
262     plot_idx += 1
263 # 绘制染色体异常汇总
264 if available_chromosomes and plot_idx < 4:
265     chromosome_data = {}
266     for var in available_chromosomes:
267         abnormal_count = df[var].sum() if var in df.columns else 0
268         normal_count = len(df) - abnormal_count
269         chromosome_name = var.replace('_异常', '')
270         chromosome_data[f'{chromosome_name}异常'] = abnormal_count
271         chromosome_data[f'{chromosome_name}正常'] = normal_count
272     # 堆叠条形图
273     chromosomes = ['T13', 'T18', 'T21']
274     abnormal_counts = [chromosome_data.get(f'{chr}异常', 0) for chr in
275 chromosomes]
276     normal_counts = [chromosome_data.get(f'{chr}正常', 0) for chr in
277 chromosomes]
278     x = np.arange(len(chromosomes))
279     width = 0.35

```

```

277     bars1 = axes[plot_idx].bar(x, abnormal_counts, width, label='异常', color='
red', alpha=0.7)
278     bars2 = axes[plot_idx].bar(x, normal_counts, width, bottom=abnormal_counts,
279                               label='正常', color='green', alpha=0.7)
280     axes[plot_idx].set_title('染色体异常分布')
281     axes[plot_idx].set_xlabel('染色体类型')
282     axes[plot_idx].set_ylabel('样本数')
283     axes[plot_idx].set_xticks(x)
284     axes[plot_idx].set_xticklabels(chromosomes)
285     axes[plot_idx].legend()
286     # 添加数值标签
287     for i, (bar1, bar2) in enumerate(zip(bars1, bars2)):
288         if abnormal_counts[i] > 0:
289             axes[plot_idx].text(bar1.get_x() + bar1.get_width()/2,
290                                bar1.get_height()/2, f'{abnormal_counts[i]}',
291                                ha='center', va='center', fontweight='bold',
color='white')
292             axes[plot_idx].text(bar2.get_x() + bar2.get_width()/2,
293                                abnormal_counts[i] + bar2.get_height()/2, f'{
normal_counts[i]}',
294                                ha='center', va='center', fontweight='bold', color='
white')
295             plot_idx += 1
296
297     # 隐藏多余的子图
298     for i in range(plot_idx, 4):
299         axes[i].set_visible(False)
300     plt.tight_layout()
301     plt.savefig('数据预处理_分类变量分布.png', dpi=300, bbox_inches='tight',
facecolor='white')
302     plt.show()
303 # 连续变量分布图表
304 def create_continuous_distribution_plots(df):
305     # 定义连续变量
306     continuous_vars = [
307         '年龄', '身高', '体重', '孕妇BMI',
308         '末次月经距检测日天数', '检测时怀孕天数',
309         'Y染色体浓度', 'Y染色体的Z值', 'GC含量'
310     ]
311
312     available_vars = [var for var in continuous_vars if var in df.columns]
313
314     if not available_vars:
315         print("□ 未找到连续变量，跳过连续变量分布图")
316         return
317
318     n_vars = len(available_vars)

```

```

319     n_cols = 3
320     n_rows = (n_vars + n_cols - 1) // n_cols
321
322     fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 5*n_rows))
323     fig.suptitle('连续变量分布分析', fontsize=16, fontweight='bold')
324
325     if n_rows == 1:
326         axes = axes.reshape(1, -1) if n_vars > 1 else [axes]
327     axes = axes.flatten() if n_vars > 1 else [axes]
328
329     for i, var in enumerate(available_vars):
330         if i >= len(axes):
331             break
332
333         data = df[var].dropna()
334
335         if len(data) == 0:
336             axes[i].text(0.5, 0.5, f'{var}\n无数据', ha='center', va='center',
337                          transform=axes[i].transAxes, fontsize=12)
338             axes[i].set_title(var)
339             continue
340
341         # 直方图 + 核密度估计
342         axes[i].hist(data, bins=30, density=True, alpha=0.7, color='skyblue',
343                      edgecolor='black')
344
345         # 核密度估计
346         try:
347             from scipy.stats import gaussian_kde
348             kde = gaussian_kde(data)
349             x_range = np.linspace(data.min(), data.max(), 100)
350             axes[i].plot(x_range, kde(x_range), 'r-', linewidth=2, label='KDE')
351         except:
352             pass
353
354         # 添加统计信息
355         mean_val = data.mean()
356         median_val = data.median()
357         axes[i].axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f
358                          '均值: {mean_val:.3f}')
359         axes[i].axvline(median_val, color='orange', linestyle='--', linewidth=2,
360                          label=f'中位数: {median_val:.3f}')
361
362         axes[i].set_title(var)
363         axes[i].set_ylabel('密度')
364         axes[i].legend()
365         axes[i].grid(True, alpha=0.3)

```

```

363
364 # 隐藏多余的子图
365 for j in range(len(available_vars), len(axes)):
366     axes[j].set_visible(False)
367
368 plt.tight_layout()
369 plt.savefig('数据预处理_连续变量分布.png', dpi=300, bbox_inches='tight',
370           facecolor='white')
371 plt.show()
372 # 数据质量分析图表
373 def create_data_quality_plots(df, missing_df):
374     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
375     fig.suptitle('数据质量分析', fontsize=16, fontweight='bold')
376     # 1. 缺失值分析
377     if not missing_df.empty:
378         missing_counts = missing_df['缺失数量'].astype(int)
379         missing_counts = missing_counts[missing_counts > 0] # 只显示有缺失值的字段
380         if len(missing_counts) > 0:
381             bars = axes[0, 0].barh(range(len(missing_counts)), missing_counts.
382                                   values,
383                                   color='red', alpha=0.7)
384             axes[0, 0].set_yticks(range(len(missing_counts)))
385             axes[0, 0].set_yticklabels(missing_counts.index)
386             axes[0, 0].set_xlabel('缺失值数量')
387             axes[0, 0].set_title('字段缺失值统计')
388             axes[0, 0].grid(True, alpha=0.3, axis='x')
389             # 添加数值标签
390             for i, (bar, value) in enumerate(zip(bars, missing_counts.values)):
391                 axes[0, 0].text(bar.get_width() + 5, i, f'{value}',
392                               va='center', fontweight='bold')
393             else:
394                 axes[0, 0].text(0.5, 0.5,
395                               ha='center', va='center', transform=axes[0, 0].transAxes
396                               ,
397                               fontsize=14, fontweight='bold', color='green')
398             axes[0, 0].set_title('字段缺失值统计')
399     # 2. 数值变量的异常值检测
400     if 'Y染色体浓度' in df.columns:
401         y_data = df['Y染色体浓度'].dropna()
402         if len(y_data) > 0:
403             # 箱线图检测异常值
404             bp = axes[0, 1].boxplot(y_data, patch_artist=True)
405             bp['boxes'][0].set_facecolor('lightgreen')
406             bp['boxes'][0].set_alpha(0.7)
407             # 计算异常值
408             Q1 = y_data.quantile(0.25)
409             Q3 = y_data.quantile(0.75)

```

```

407         IQR = Q3 - Q1
408         lower_bound = Q1 - 1.5 * IQR
409         upper_bound = Q3 + 1.5 * IQR
410         outliers = y_data[(y_data < lower_bound) | (y_data > upper_bound)]
411         axes[0, 1].set_title(f'Y染色体浓度异常值检测\n异常值: {len(outliers)}
个 ({len(outliers)/len(y_data)*100:.1f}%)')
412         axes[0, 1].set_ylabel('Y染色体浓度')
413         axes[0, 1].grid(True, alpha=0.3)
414     else:
415         axes[0, 1].text(0.5, 0.5, 'Y染色体浓度\n数据不可用',
416                        ha='center', va='center', transform=axes[0, 1].transAxes,
fontSize=14)
417         axes[0, 1].set_title('异常值检测')
418
419     # 3. 样本完整性分析
420     complete_data = df.dropna()
421     total_samples = len(df)
422     complete_samples = len(complete_data)
423     incomplete_samples = total_samples - complete_samples
424
425     sizes = [complete_samples, incomplete_samples]
426     labels = [f'完整样本\n{complete_samples}例', f'不完整样本\n{incomplete_samples}
例']
427     colors = ['lightgreen', 'lightcoral']
428
429     wedges, texts, autotexts = axes[1, 0].pie(sizes, labels=labels, autopct='%1.1f
%%',
430                                                colors=colors, startangle=90)
431     axes[1, 0].set_title('样本完整性分析')
432
433     # 4. 关键变量可用性
434     key_vars = ['年龄', '孕妇BMI', 'Y染色体浓度', '胎儿健康_编码', 'IVF妊娠_编码']
435     available_vars = [var for var in key_vars if var in df.columns]
436
437     if available_vars:
438         availability_rates = []
439         var_labels = []
440
441         for var in available_vars:
442             non_null_count = df[var].count()
443             rate = non_null_count / len(df) * 100
444             availability_rates.append(rate)
445             var_labels.append(f'{var}\n({non_null_count}/{len(df)})')
446
447         bars = axes[1, 1].bar(range(len(available_vars)), availability_rates,
448                               color='steelblue', alpha=0.7)
449         axes[1, 1].set_xticks(range(len(available_vars)))

```



```

450     axes[1, 1].set_xticklabels(var_labels, rotation=45, ha='right')
451     axes[1, 1].set_ylabel('数据可用率 (%)')
452     axes[1, 1].set_title('关键变量数据可用性')
453     axes[1, 1].set_ylim(0, 105)
454     axes[1, 1].grid(True, alpha=0.3, axis='y')
455
456     # 添加百分比标签
457     for bar, rate in zip(bars, availability_rates):
458         axes[1, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
459                         f'{rate:.1f}%', ha='center', va='bottom', fontweight='
bold')
460     else:
461         axes[1, 1].text(0.5, 0.5, '关键变量\n不可用',
462                         ha='center', va='center', transform=axes[1, 1].transAxes,
fontsize=14)
463         axes[1, 1].set_title('关键变量数据可用性')
464
465     plt.tight_layout()
466     plt.savefig('数据预处理_数据质量分析.png', dpi=300, bbox_inches='tight',
facecolor='white')
467     plt.show()
468 #初步相关性分析图表
469 def create_preliminary_correlation_plots(df):
470     # 选择数值变量进行相关性分析
471     numeric_vars = [
472         '年龄', '身高', '体重', '孕妇BMI',
473         '末次月经距检测日天数', '检测时怀孕天数',
474         'Y染色体浓度', 'Y染色体的Z值', 'GC含量',
475         'IVF妊娠_编码', '怀孕次数_编码', '胎儿健康_编码'
476     ]
477     available_vars = [var for var in numeric_vars if var in df.columns]
478     if len(available_vars) < 3:
479         print("可用数值变量不足3个，跳过相关性分析")
480         return
481     # 创建相关性数据
482     corr_data = df[available_vars].select_dtypes(include=[np.number]).dropna()
483
484     if len(corr_data) == 0:
485         print("没有足够的数值数据进行相关性分析")
486         return
487
488     correlation_matrix = corr_data.corr()
489
490     fig, axes = plt.subplots(1, 2, figsize=(16, 6))
491     fig.suptitle('初步相关性分析', fontsize=16, fontweight='bold')
492
493     # 1. 相关性热图

```

```

494 mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
495 sns.heatmap(correlation_matrix, mask=mask, annot=False, cmap='RdBu_r', center
=0,
496             square=True, fmt='.3f', cbar_kws={"shrink": .8}, ax=axes[0])
497 axes[0].set_title('变量间相关性热图')
498
499 # 2. 与Y染色体浓度的相关性
500 if 'Y染色体浓度' in correlation_matrix.columns:
501     y_correlations = correlation_matrix['Y染色体浓度'].drop('Y染色体浓度')
502     y_correlations = y_correlations.sort_values(key=abs, ascending=False)
503
504     colors = ['red' if x < 0 else 'blue' for x in y_correlations.values]
505     bars = axes[1].barh(range(len(y_correlations)), y_correlations.values,
506                        color=colors, alpha=0.7)
507     axes[1].set_yticks(range(len(y_correlations)))
508     axes[1].set_yticklabels(y_correlations.index)
509     axes[1].set_xlabel('相关系数')
510     axes[1].set_title('与Y染色体浓度的相关性')
511     axes[1].axvline(x=0, color='black', linestyle='-', linewidth=1)
512     axes[1].grid(True, alpha=0.3, axis='x')
513
514     # 添加相关系数标签
515     for i, (bar, value) in enumerate(zip(bars, y_correlations.values)):
516         x_pos = value + (0.01 if value > 0 else -0.01)
517         ha_pos = 'left' if value > 0 else 'right'
518         axes[1].text(x_pos, i, f'{value:.3f}', va='center', ha=ha_pos,
519                    fontweight='bold', fontsize=9)
520 else:
521     var_correlations = {}
522     for var in correlation_matrix.columns:
523         other_correlations = correlation_matrix[var].drop(var)
524         avg_correlation = other_correlations.abs().mean()
525         var_correlations[var] = avg_correlation
526
527     sorted_vars = sorted(var_correlations.items(), key=lambda x: x[1], reverse=
True)
528
529     variables = [item[0] for item in sorted_vars]
530     correlations = [item[1] for item in sorted_vars]
531
532     bars = axes[1].barh(range(len(variables)), correlations,
533                        color='steelblue', alpha=0.7)
534     axes[1].set_yticks(range(len(variables)))
535     axes[1].set_yticklabels(variables)
536     axes[1].set_xlabel('平均相关性强度')
537     axes[1].set_title('变量相关性强度排序')
538     axes[1].grid(True, alpha=0.3, axis='x')

```

```

539
540     # 添加数值标签
541     for i, (bar, value) in enumerate(zip(bars, correlations)):
542         axes[1].text(bar.get_width() + 0.005, i, f'{value:.3f}',
543                     va='center', fontweight='bold', fontsize=9)
544
545     plt.tight_layout()
546     plt.savefig('数据预处理_初步相关性分析.png', dpi=300, bbox_inches='tight',
547               facecolor='white')
548     plt.show()
549
550 if __name__ == "__main__":
551     current_dir = os.path.dirname(os.path.abspath(__file__))
552     file_path = os.path.join(current_dir, "附件.xlsx")
553     # 执行数据预处理 (启用可视化)
554     final_data, full_processed_data = preprocess_male_nipt_data(
555         file_path,
556         enable_visualization=True
557     )
558     # 保存处理后的数据
559     final_data.to_csv("男胎_预处理数据_精简版.csv",
560                     index=False, encoding='utf-8-sig')
561     full_processed_data.to_csv("男胎_预处理数据_完整版.csv",
562                              index=False, encoding='utf-8-sig')

```

问题 1 数据预处理.py

```

1  import pandas as pd
2  import numpy as np
3  from datetime import datetime, timedelta
4  import re
5  import os
6  from scipy import stats
7  from scipy.stats import normaltest, shapiro, pearsonr, spearmanr, f_oneway, kruskal
8  # 文件读取和修改变量
9  #[13]AI构建了一个包含核心功能模块的初始代码框架,再自行根据实际问题要求进行修改和完善
10 def preprocess_male_nipt_data(file_path):
11     df = pd.read_excel(file_path, sheet_name='男胎检测数据')
12     processed_df = df.copy()
13     # 将末次月经日期+检测日期转化变量为末次月经距检测日天数
14     def calculate_days_from_lmp(row):
15         try:
16             lmp_date = pd.to_datetime(row['末次月经'])
17             detection_date_str = str(int(row['检测日期']))
18             detection_date = pd.to_datetime(detection_date_str, format='%Y%m%d')
19             days_diff = (detection_date - lmp_date).days
20             return days_diff

```

```

21     except:
22         return np.nan
23     processed_df['末次月经距检测日天数'] = processed_df.apply(
24         calculate_days_from_lmp, axis=1)
25
26     # IVF妊娠转换为分类变量，创建数值编码和标签映射
27     ivf_mapping = {
28         '自然受孕': 0,
29         'IUI (人工授精)': 1,
30         'IVF (试管婴儿)': 2
31     }
32     processed_df['IVF妊娠_编码'] = processed_df['IVF妊娠'].map(ivf_mapping)
33     # 改为二进制形式 (1和0的数值格式)
34     ivf_dummies = pd.get_dummies(processed_df['IVF妊娠'], prefix='IVF', dtype=int)
35     processed_df = pd.concat([processed_df, ivf_dummies], axis=1)
36
37     # 将检测孕周转换为检测时怀孕天数
38     def parse_gestational_weeks(week_str):
39         try:
40             if pd.isna(week_str):
41                 return np.nan
42             week_str = str(week_str).strip()
43             if 'w' in week_str:
44                 if '+' in week_str:
45                     weeks_part, days_part = week_str.split('+')
46                     weeks = int(weeks_part.replace('w', ''))
47                     days = int(days_part)
48                 else:
49                     weeks = int(week_str.replace('w', ''))
50                     days = 0
51             total_days = weeks * 7 + days
52             return total_days
53         except:
54             return np.nan
55
56     processed_df['检测时怀孕天数'] = processed_df['检测孕周'].apply(
57         parse_gestational_weeks)
58
59     # 将染色体非整倍体信息处理为T13、T18、T21异常标记
60     def parse_chromosome_abnormality(abnormality_str):
61         # 初始化结果
62         result = {'T13': 0, 'T18': 0, 'T21': 0}
63         if pd.isna(abnormality_str) or abnormality_str == '' or abnormality_str is
64             None:
65             return result

```

```

65     abnormality_str = str(abnormality_str).strip()
66     # 检查是否包含各种异常标记
67     if 'T13' in abnormality_str:
68         result['T13'] = 1
69     if 'T18' in abnormality_str:
70         result['T18'] = 1
71     if 'T21' in abnormality_str:
72         result['T21'] = 1
73     return result
74 chromosome_results = processed_df['染色体的非整倍体'].apply(
parse_chromosome_abnormality)
75 processed_df['T13_异常'] = [result['T13'] for result in chromosome_results]
76 processed_df['T18_异常'] = [result['T18'] for result in chromosome_results]
77 processed_df['T21_异常'] = [result['T21'] for result in chromosome_results]
78
79 #怀孕次数转换为分类变量，创建数值编码和二进制形式
80 def categorize_pregnancy_count(count):
81     if pd.isna(count):
82         return np.nan
83     if count == 1:
84         return '初产妇'
85     elif count == 2:
86         return '经产妇_2次'
87     else:
88         return '经产妇_多次'
89
90 processed_df['怀孕次数_分类'] = processed_df['怀孕次数'].apply(
categorize_pregnancy_count)
91 pregnancy_mapping = {
92     '初产妇': 0,
93     '经产妇_2次': 1,
94     '经产妇_多次': 2
95 }
96 processed_df['怀孕次数_编码'] = processed_df['怀孕次数_分类'].map(
pregnancy_mapping)
97 pregnancy_dummies = pd.get_dummies(processed_df['怀孕次数_分类'], prefix='怀孕
次数', dtype=int)
98 processed_df = pd.concat([processed_df, pregnancy_dummies], axis=1)
99 # 胎儿健康状况转换为二分类变量，1表示健康，0表示不健康
100 health_mapping = {
101     '是': 1,
102     '否': 0
103 }
104 processed_df['胎儿健康_编码'] = processed_df['胎儿是否健康'].map(health_mapping
)
105 key_columns = [
106     # 原始标识字段

```

```

107     '序号', '孕妇代码', '年龄', '身高', '体重', '孕妇BMI',
108     # 预处理后的字段
109     '末次月经距检测日天数', 'IVF妊娠_编码', '检测时怀孕天数',
110     'T13_异常', 'T18_异常', 'T21_异常',
111     '怀孕次数_编码', '胎儿健康_编码',
112     # 重要的检测指标
113     'Y染色体浓度', 'Y染色体的Z值',
114     '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
115     'X染色体的Z值', 'GC含量',
116     # 哑变量
117     'IVF_自然受孕', 'IVF_IUI (人工授精)', 'IVF_IVF (试管婴儿)',
118     '怀孕次数_初产妇', '怀孕次数_经产妇_2次', '怀孕次数_经产妇_多次'
119 ]
120 available_columns = [col for col in key_columns if col in processed_df.columns]
121 final_df = processed_df[available_columns].copy()
122 return final_df, processed_df
123 def correlation_analysis_y_chromosome_with_csv(df, output_file="Y染色体浓度相关性检
124     验结果.csv"):
125     # 因变量: Y染色体浓度
126     y_var = 'Y染色体浓度'
127
128     if y_var not in df.columns:
129         print("错误: 数据中没有找到 {} 列".format(y_var))
130         return
131
132     # 去除Y染色体浓度的缺失值
133     df_clean = df.dropna(subset=[y_var]).copy()
134     print("有效样本数: {}".format(len(df_clean)))
135
136     # Y染色体浓度描述统计
137     y_stats = {
138         '均值': float(df_clean[y_var].mean()),
139         '标准差': float(df_clean[y_var].std()),
140         '中位数': float(df_clean[y_var].median()),
141         '最小值': float(df_clean[y_var].min()),
142         '最大值': float(df_clean[y_var].max())
143     }
144
145     print("Y染色体浓度描述统计:")
146     for key, value in y_stats.items():
147         print(" {}: {:.6f}".format(key, value))
148
149     # 检验Y染色体浓度的正态性
150     print("\n" + "-"*40)
151     print("Y染色体浓度正态性检验:")
152     print("-"*40)

```

```

153     if len(df_clean) < 5000:
154         y_shapiro_stat, y_shapiro_p = shapiro(df_clean[y_var])
155         print("Shapiro-Wilk 检验: 统计量={:.4f}, p值={:.6f}".format(y_shapiro_stat,
y_shapiro_p))
156         y_normal = y_shapiro_p > 0.05
157     else:
158         y_dagostino_stat, y_dagostino_p = normaltest(df_clean[y_var])
159         print("D'Agostino 检验: 统计量={:.4f}, p值={:.6f}".format(y_dagostino_stat,
y_dagostino_p))
160         y_normal = y_dagostino_p > 0.05
161
162     print("    结论: Y染色体浓度{}符合正态分布".format(" " if y_normal else "不"))
163
164     # 定义变量类型
165     continuous_vars = [
166         '年龄', '身高', '体重', '孕妇BMI',
167         '末次月经距检测日天数', '检测时怀孕天数',
168         '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
169         'X染色体的Z值', 'Y染色体的Z值',
170         'X染色体浓度', 'GC含量'
171     ]
172
173     # 有序分类变量
174     ordinal_vars = [
175         '怀孕次数_编码' # 0=初产妇, 1=经产妇_2次, 2=经产妇_多次
176     ]
177
178     # 无序分类变量
179     nominal_vars = [
180         'IVF妊娠_编码', # 0=自然受孕, 1=IUI, 2=IVF
181         'T13_异常', 'T18_异常', 'T21_异常',
182         '胎儿健康_编码'
183     ]
184
185     # 存储所有结果
186     all_results = []
187
188     # 连续变量相关性检验
189     print("\n" + "-"*40)
190     print("连续变量相关性检验:")
191     print("-"*40)
192
193     for var in continuous_vars:
194         if var in df_clean.columns:
195             # 去除该变量的缺失值
196             temp_df = df_clean[[y_var, var]].dropna()
197

```

```

198     if len(temp_df) < 3:
199         print("{:}: 样本量不足, 跳过".format(var))
200         continue
201
202     # 检验当前变量的正态性
203     if len(temp_df) < 5000:
204         var_shapiro_stat, var_shapiro_p = shapiro(temp_df[var])
205         var_normal = var_shapiro_p > 0.05
206         normality_test = "Shapiro-Wilk"
207         normality_stat = var_shapiro_stat
208         normality_p = var_shapiro_p
209     else:
210         var_dagostino_stat, var_dagostino_p = normaltest(temp_df[var])
211         var_normal = var_dagostino_p > 0.05
212         normality_test = "D'Agostino"
213         normality_stat = var_dagostino_stat
214         normality_p = var_dagostino_p
215
216     # 选择相关性检验方法: 两个变量都正态才用皮尔逊
217     if y_normal and var_normal:
218         corr_coef, p_value = pearsonr(temp_df[var], temp_df[y_var])
219         method = "皮尔逊相关"
220     else:
221         corr_coef, p_value = spearmanr(temp_df[var], temp_df[y_var])
222         method = "斯皮尔曼相关"
223
224     # 判断显著性
225     significance = ""
226     if p_value < 0.001:
227         significance = "****"
228     elif p_value < 0.01:
229         significance = "***"
230     elif p_value < 0.05:
231         significance = "**"
232     else:
233         significance = "n.s."
234
235     print("{:25s}: {}={:8.4f}, p值={:.6f} {}".format(
236         var, method, corr_coef, p_value, significance))
237     print("{:25s} (变量正态性: {}, 统计量={:.4f}, p={:.6f})".format(
238         "", "是" if var_normal else "否", normality_stat, normality_p))
239
240     # 存储结果
241     result = {
242         '变量名': var,
243         '变量类型': '连续变量',
244         '样本量': len(temp_df),

```



```

245         '检验方法': method,
246         '相关系数/统计量': corr_coef,
247         'p值': p_value,
248         '显著性': significance,
249         '变量正态性': "是" if var_normal else "否",
250         '正态性检验': normality_test,
251         '正态性统计量': normality_stat,
252         '正态性p值': normality_p,
253         '备注': ""
254     }
255     all_results.append(result)
256
257 # ===== 2. 有序分类变量相关性检验 =====
258 print("\n" + "-"*40)
259 print("有序分类变量相关性检验 (斯皮尔曼):")
260 print("-"*40)
261
262 for var in ordinal_vars:
263     if var in df_clean.columns:
264         temp_df = df_clean[[y_var, var]].dropna()
265
266         if len(temp_df) < 3:
267             print("{}: 样本量不足, 跳过".format(var))
268             continue
269
270         corr_coef, p_value = spearmanr(temp_df[var], temp_df[y_var])
271
272         # 判断显著性
273         significance = ""
274         if p_value < 0.001:
275             significance = "****"
276         elif p_value < 0.01:
277             significance = "***"
278         elif p_value < 0.05:
279             significance = "**"
280         else:
281             significance = "n.s."
282
283         print("{:25s}: 斯皮尔曼相关系数={:8.4f}, p值={:.6f} {}".format(
284             var, corr_coef, p_value, significance))
285
286         # 显示各组的描述统计
287         print("  各组统计:")
288         categories = ['初产妇', '经产妇_2次', '经产妇_多次']
289         unique_vals = sorted(temp_df[var].unique())
290
291         group_info = []

```

```

292         for i, val in enumerate(unique_vals):
293             group_data = temp_df[temp_df[var] == val][y_var]
294             cat_name = categories[i] if i < len(categories) else "类别{}".
format(i)
295
296             count_val = len(group_data)
297             mean_val = float(group_data.mean()) if len(group_data) > 0 else 0.0
298             median_val = float(group_data.median()) if len(group_data) > 0 else
0.0
299
300             print("{}: n={:3d}, 均值={:.6f}, 中位数={:.6f}".format(
301                 cat_name, count_val, mean_val, median_val))
302             group_info.append("{}(n={})".format(cat_name, count_val))
303
304         # 存储结果
305         result = {
306             '变量名': var,
307             '变量类型': '有序分类变量',
308             '样本量': len(temp_df),
309             '检验方法': '斯皮尔曼相关',
310             '相关系数/统计量': corr_coef,
311             'p值': p_value,
312             '显著性': significance,
313             '变量正态性': "N/A",
314             '正态性检验': "N/A",
315             '正态性统计量': np.nan,
316             '正态性p值': np.nan,
317             '备注': "; ".join(group_info)
318         }
319         all_results.append(result)
320     # 无序分类变量方差检验
321     print("\n" + "-"*40)
322     print("无序分类变量方差检验:")
323     print("-"*40)
324
325     for var in nominal_vars:
326         if var in df_clean.columns:
327             temp_df = df_clean[[y_var, var]].dropna()
328
329             if len(temp_df) < 3:
330                 print("{}: 样本量不足, 跳过".format(var))
331                 continue
332
333         # 获取各组数据
334         groups = []
335         group_names = []
336         for group_val in sorted(temp_df[var].unique()):

```

```

337         group_data = temp_df[temp_df[var] == group_val][y_var]
338         if len(group_data) > 0:
339             groups.append(group_data)
340             group_names.append(group_val)
341
342     if len(groups) < 2:
343         print("{:}: 分组数量不足, 跳过".format(var))
344         continue
345
346     # 检验方差齐性 (Levene检验)
347     levene_stat, levene_p = stats.levene(*groups)
348
349     if levene_p > 0.05:
350         # 方差齐性, 使用ANOVA
351         f_stat, anova_p = f_oneway(*groups)
352         test_method = "ANOVA F检验"
353         test_stat = f_stat
354         test_p = anova_p
355     else:
356         # 方差不齐, 使用Kruskal-Wallis检验
357         h_stat, kruskal_p = kruskal(*groups)
358         test_method = "Kruskal-Wallis H检验"
359         test_stat = h_stat
360         test_p = kruskal_p
361
362     # 判断显著性
363     significance = ""
364     if test_p < 0.001:
365         significance = "****"
366     elif test_p < 0.01:
367         significance = "***"
368     elif test_p < 0.05:
369         significance = "**"
370     else:
371         significance = "n.s."
372
373     print("{:25s}: {}".format(var, test_method))
374     print("{:25s}   统计量={:8.4f}, p值={:.6f} {}".format(
375         "", test_stat, test_p, significance))
376     print("{:25s}   Levene检验p值={:.6f} (方差{})".format(
377         "", levene_p, '齐性' if levene_p > 0.05 else '不齐'))
378
379     print("  各组统计:")
380     group_info = []
381     for i, (group_val, group_data) in enumerate(zip(group_names, groups)):
382         # 根据变量类型给出组名
383         if var == 'IVF妊娠_编码':

```

```

384         group_label = ['自然受孕', 'IUI', 'IVF'][int(group_val)] if
group_val < 3 else "组{}".format(group_val)
385     elif var in ['T13_异常', 'T18_异常', 'T21_异常']:
386         group_label = '异常' if group_val == 1 else '正常'
387     elif var == '胎儿健康_编码':
388         group_label = '健康' if group_val == 1 else '不健康'
389     else:
390         group_label = "组{}".format(group_val)
391
392     mean_val = float(group_data.mean()) if pd.notna(group_data.mean())
else 0.0
393     median_val = float(group_data.median()) if pd.notna(group_data.
median()) else 0.0
394
395     print("    {}: n={:3d}, 均值={:.6f}, 中位数={:.6f}".format(
396         group_label, len(group_data), mean_val, median_val))
397     group_info.append("{}(n={})".format(group_label, len(group_data)))
398     result = {
399         '变量名': var,
400         '变量类型': '无序分类变量',
401         '样本量': len(temp_df),
402         '检验方法': test_method,
403         '相关系数/统计量': test_stat,
404         'p值': test_p,
405         '显著性': significance,
406         '变量正态性': "N/A",
407         '正态性检验': "N/A",
408         '正态性统计量': np.nan,
409         '正态性p值': np.nan,
410         '备注': "Levene检验p={:.6f}; {}".format(levene_p, "; ").join(
group_info))
411     }
412     all_results.append(result)
413
414     #保存结果到CSV
415     print("\n" + "-"*60)
416     print("保存结果到CSV文件:")
417     print("-"*60)
418
419     results_df = pd.DataFrame(all_results)
420
421     # 添加Y染色体浓度的基本信息
422     y_info = pd.DataFrame([
423         '变量名': 'Y染色体浓度(因变量)',
424         '变量类型': '连续变量',
425         '样本量': len(df_clean),
426         '检验方法': 'Shapiro-Wilk' if len(df_clean) < 5000 else "D'Agostino",

```

```

427     '相关系数/统计量': y_shapiro_stat if len(df_clean) < 5000 else
y_dagostino_stat,
428     'p值': y_shapiro_p if len(df_clean) < 5000 else y_dagostino_p,
429     '显著性': "正态" if y_normal else "非正态",
430     '变量正态性': "是" if y_normal else "否",
431     '正态性检验': 'Shapiro-Wilk' if len(df_clean) < 5000 else "D'Agostino",
432     '正态性统计量': y_shapiro_stat if len(df_clean) < 5000 else
y_dagostino_stat,
433     '正态性p值': y_shapiro_p if len(df_clean) < 5000 else y_dagostino_p,
434     '备注': "因变量基本信息: 均值={:.6f}, 标准差={:.6f}".format(y_stats['均值'],
y_stats['标准差'])
435     })
436
437     # 合并所有结果
438     final_results = pd.concat([y_info, results_df], ignore_index=True)
439     final_results.to_csv(output_file, index=False, encoding='utf-8-sig')
440     print("结果已保存到: {}".format(output_file))
441     return final_results
442
443 if __name__ == "__main__":
444     current_dir = os.path.dirname(os.path.abspath(__file__))
445     file_path = os.path.join(current_dir, "附件.xlsx")
446     final_data, full_processed_data = preprocess_male_nipt_data(file_path)
447     final_csv_path = os.path.join(current_dir, "男胎_预处理数据_精简版.csv")
448     full_csv_path = os.path.join(current_dir, "男胎_预处理数据_完整版.csv")
449     final_data.to_csv(final_csv_path, index=False, encoding='utf-8-sig')
450     full_processed_data.to_csv(full_csv_path, index=False, encoding='utf-8-sig')
451     results = correlation_analysis_y_chromosome_with_csv(full_processed_data, "Y染色
体浓度相关性检验结果.csv")

```

问题1 相关性分析与假设检验.py

```

1 import matplotlib
2 matplotlib.use('Agg') #设置图形字体，避免冲突
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import numpy as np
6 import pandas as pd
7 from scipy import stats
8 import warnings
9 warnings.filterwarnings('ignore')
10 #设置中文字体，以便区分；
11 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong', 'DejaVu
Sans']
12 plt.rcParams['axes.unicode_minus'] = False
13 plt.rcParams['font.size'] = 10
14 #保持中文字体
15 sns.set_style("whitegrid")

```

```

16 sns.set_palette("husl")
17 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong']
18
19 #定义函数，使得最后每个图都是中文字体
20 def ensure_chinese_font(func):
21     def wrapper(*args, **kwargs):
22         plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong']
23         plt.rcParams['axes.unicode_minus'] = False
24         return func(*args, **kwargs)
25     return wrapper
26 @ensure_chinese_font
27 def plot_distribution_analysis(df, y_var='Y染色体浓度'):
28     print("1. Y染色体浓度分布可视化分析")
29     #再次设置中文字体
30     plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong']
31     plt.rcParams['axes.unicode_minus'] = False
32     #引入数据
33     y_data = df[y_var].dropna()
34     #[8]AI协助添加过程可视化使其更具可解释性
35     #准备画图
36     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
37     #设置标题
38     fig.suptitle('Y染色体浓度分布分析', fontsize=16, fontweight='bold',
39                 fontproperties='Microsoft YaHei')
40     #绘制直方图与密度曲线
41     axes[0, 0].hist(y_data, bins=40, density=True, alpha=0.7, color='skyblue',
42                    edgecolor='black')
43     axes[0, 0].axvline(y_data.mean(), color='red', linestyle='--', linewidth=2,
44                       label=f'均值: {y_data.mean():.4f}')
45     axes[0, 0].axvline(y_data.median(), color='orange', linestyle='--', linewidth
46                       =2, label=f'中位数: {y_data.median():.4f}')
47
48     #绘制理论正态分布曲线，并设置字体
49     x = np.linspace(y_data.min(), y_data.max(), 100)
50     normal_curve = stats.norm.pdf(x, y_data.mean(), y_data.std())
51     axes[0, 0].plot(x, normal_curve, 'g-', linewidth=2, label='理论正态分布')
52     axes[0, 0].set_title('直方图与理论正态分布对比', fontproperties='Microsoft
53     YaHei')
54     axes[0, 0].set_xlabel('Y染色体浓度', fontproperties='Microsoft YaHei')
55     axes[0, 0].set_ylabel('密度', fontproperties='Microsoft YaHei')
56     axes[0, 0].legend(prop={'family': 'Microsoft YaHei'})
57     axes[0, 0].grid(True, alpha=0.3)
58
59     #绘制核密度估计图
60     sns.kdeplot(data=y_data, ax=axes[0, 1], fill=True, color='lightcoral')
61     axes[0, 1].axvline(y_data.mean(), color='red', linestyle='--', linewidth=2,
62                       label='均值')

```

```

58     axes[0, 1].axvline(y_data.median(), color='orange', linestyle='--', linewidth
=2, label='中位数')
59     axes[0, 1].set_title('核密度估计')
60     axes[0, 1].set_xlabel('Y染色体浓度')
61     axes[0, 1].set_ylabel('密度')
62     axes[0, 1].legend()
63     axes[0, 1].grid(True, alpha=0.3)
64
65     #绘制QQ图, 用于正态性检验
66     stats.probplot(y_data, dist="norm", plot=axes[0, 2])
67     axes[0, 2].set_title('Q-Q图 (正态性检验)')
68     axes[0, 2].grid(True, alpha=0.3)
69
70     #绘制箱线图
71     box_plot = axes[1, 0].boxplot(y_data, patch_artist=True, labels=['Y染色体浓度'
])
72     box_plot['boxes'][0].set_facecolor('lightblue')
73     axes[1, 0].set_title('箱线图')
74     axes[1, 0].set_ylabel('Y染色体浓度')
75     axes[1, 0].grid(True, alpha=0.3)
76     #添加新的统计信息
77     Q1 = y_data.quantile(0.25)
78     Q3 = y_data.quantile(0.75)
79     IQR = Q3 - Q1
80     lower_bound = Q1 - 1.5 * IQR
81     upper_bound = Q3 + 1.5 * IQR
82     outliers = y_data[(y_data < lower_bound) | (y_data > upper_bound)]
83
84     axes[1, 0].text(0.02, 0.98, f'异常值数量: {len(outliers)} ({len(outliers)/len(
y_data)*100:.1f}%)',
85                     transform=axes[1, 0].transAxes, verticalalignment='top',
86                     bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))
87     #经验累积分布函数
88     sorted_data = np.sort(y_data)
89     y_ecdf = np.arange(1, len(sorted_data) + 1) / len(sorted_data)
90     axes[1, 1].plot(sorted_data, y_ecdf, marker='.', linestyle='-', alpha=0.7,
color='purple')
91     #添加理论正态分布CDF
92     theoretical_cdf = stats.norm.cdf(sorted_data, y_data.mean(), y_data.std())
93     axes[1, 1].plot(sorted_data, theoretical_cdf, 'r--', linewidth=2, label='理论正
态CDF')
94
95     axes[1, 1].set_title('经验累积分布函数 vs 理论正态CDF')
96     axes[1, 1].set_xlabel('Y染色体浓度')
97     axes[1, 1].set_ylabel('累积概率')
98     axes[1, 1].legend()
99     axes[1, 1].grid(True, alpha=0.3)

```

```

100
101 #统计描述信息表
102 axes[1, 2].axis('off')
103 #对数据进行正态性检验
104 shapiro_stat, shapiro_p = stats.shapiro(y_data) if len(y_data) <= 5000 else
stats.normaltest(y_data)
105 test_name = "Shapiro-Wilk" if len(y_data) <= 5000 else "D'Agostino"
106 #偏度和峰度
107 skewness = stats.skew(y_data)
108 kurtosis = stats.kurtosis(y_data)
109 stats_text = f'''统计描述信息：
110 样本量：{len(y_data)}
111 均值：{y_data.mean():.6f}
112 标准差：{y_data.std():.6f}
113 中位数：{y_data.median():.6f}
114 最小值：{y_data.min():.6f}
115 最大值：{y_data.max():.6f}
116 分位数：
117 25%：{Q1:.6f}
118 75%：{Q3:.6f}
119 四分位数间距：{IQR:.6f}
120 形状参数：
121 偏度：{skewness:.4f}
122 峰度：{kurtosis:.4f}
123 正态性检验 ({test_name}):
124 统计量：{shapiro_stat:.4f}
125 p值：{shapiro_p:.6f}
126 结论：{'符合正态分布' if shapiro_p > 0.05 else '不符合正态分布'}
127 '''
128 axes[1, 2].text(0.05, 0.95, stats_text, transform=axes[1, 2].transAxes,
129                 verticalalignment='top', fontsize=10,
130                 bbox=dict(boxstyle='round', facecolor='lightgray', alpha=0.8))
131 plt.tight_layout()
132 plt.savefig('Y染色体浓度分布分析.png', dpi=300, bbox_inches='tight')
133 plt.show()
134
135 #输出关键发现
136 print("\n关键发现:")
137 print(f"• Y染色体浓度均值为 {y_data.mean():.6f}, 标准差为 {y_data.std():.6f}")
138 print(f"• 数据分布{'符合' if shapiro_p > 0.05 else '不符合'}正态分布 (p={
shapiro_p:.6f})")
139 print(f"• 偏度为 {skewness:.4f} ({'右偏' if skewness > 0 else '左偏' if
skewness < 0 else '对称'})")
140 print(f"• 峰度为 {kurtosis:.4f} ({'尖峰' if kurtosis > 0 else '平峰' if
kurtosis < 0 else '正常峰度'})")
141 print(f"• 发现 {len(outliers)} 个异常值 ({len(outliers)/len(y_data)*100:.1f}%)
")

```



```

142
143     return {
144         'stats': {
145             'mean': y_data.mean(),
146             'std': y_data.std(),
147             'median': y_data.median(),
148             'min': y_data.min(),
149             'max': y_data.max(),
150             'q25': Q1,
151             'q75': Q3,
152             'iqr': IQR,
153             'skewness': skewness,
154             'kurtosis': kurtosis,
155             'normality_test': test_name,
156             'normality_stat': shapiro_stat,
157             'normality_p': shapiro_p,
158             'outliers_count': len(outliers)
159         },
160         'outliers': outliers
161     }
162
163 def plot_correlation_heatmap(df):
164     print("2. 相关性热图可视化分析")
165     #选择表格中的数值型变量
166     numeric_vars = [
167         'Y染色体浓度', '年龄', '身高', '体重', '孕妇BMI',
168         '末次月经距检测日天数', '检测时怀孕天数',
169         '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
170         'X染色体的Z值', 'Y染色体的Z值',
171         'X染色体浓度', 'GC含量'
172     ]
173     #对已经存在的变量进行筛选
174     available_vars = [var for var in numeric_vars if var in df.columns]
175     correlation_data = df[available_vars].dropna()
176     #算相关系数矩阵
177     correlation_matrix = correlation_data.corr()
178
179     #绘制热力图
180     fig, axes = plt.subplots(1, 2, figsize=(20, 8))
181     mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
182     sns.heatmap(correlation_matrix, mask=mask, annot=True, cmap='RdBu_r', center=0,
183                 square=True, fmt='.3f', cbar_kws={"shrink": .8}, ax=axes[0])
184     axes[0].set_title('变量间相关性热图 (下三角)', fontsize=14)
185
186     #Y染色体浓度与其他变量的相关性条形图
187     y_correlations = correlation_matrix['Y染色体浓度'].drop('Y染色体浓度').
    sort_values(key=abs, ascending=False)

```

```

188     colors = ['red' if x < 0 else 'blue' for x in y_correlations.values]
189     bars = axes[1].barh(range(len(y_correlations)), y_correlations.values, color=
190     colors, alpha=0.7)
191     axes[1].set_yticks(range(len(y_correlations)))
192     axes[1].set_yticklabels(y_correlations.index, fontsize=10)
193     axes[1].set_xlabel('相关系数')
194     axes[1].set_title('Y染色体浓度与其他变量的相关性', fontsize=14)
195     axes[1].axvline(x=0, color='black', linestyle='--', linewidth=0.5)
196     axes[1].grid(True, alpha=0.3, axis='x')
197
198     #设置标签
199     for i, (bar, value) in enumerate(zip(bars, y_correlations.values)):
200         axes[1].text(value + (0.01 if value > 0 else -0.01), i, f'{value:.3f}',
201                     va='center', ha='left' if value > 0 else 'right', fontsize=9)
202     plt.tight_layout()
203     plt.savefig('相关性分析热图.png', dpi=300, bbox_inches='tight')
204     plt.show()
205
206     #输出重要发现
207     print(f"相关性分析基于 {len(correlation_data)} 个完整样本")
208     print(f"\n与Y染色体浓度相关性最强的前5个变量:")
209     for i, (var, corr) in enumerate(y_correlations.head(5).items(), 1):
210         print(f" {i}. {var}: {corr:.4f}")
211     return {
212         'correlation_matrix': correlation_matrix,
213         'y_correlations': y_correlations,
214         'sample_size': len(correlation_data)
215     }
216
217 def plot_group_comparisons(df):
218     print("3. 分组对比可视化分析")
219     y_var = 'Y染色体浓度'
220     # 定义分类变量和其标签映射
221     categorical_vars = {
222         '怀孕次数_编码': {0: '初产妇', 1: '经产妇_2次', 2: '经产妇_多次'},
223         'IVF妊娠_编码': {0: '自然受孕', 1: 'IUI', 2: 'IVF'},
224         '胎儿健康_编码': {0: '不健康', 1: '健康'},
225         'T13_异常': {0: '正常', 1: '异常'},
226         'T18_异常': {0: '正常', 1: '异常'},
227         'T21_异常': {0: '正常', 1: '异常'}
228     }
229     available_cat_vars = {k: v for k, v in categorical_vars.items() if k in df.
230     columns}
231
232     #绘制子图
233     n_vars = len(available_cat_vars)
234     n_cols = 3
235     n_rows = (n_vars + n_cols - 1) // n_cols
236     fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 6*n_rows))

```

```

233     if n_rows == 1:
234         axes = axes.reshape(1, -1)
235     elif n_vars == 1:
236         axes = axes.reshape(1, 1)
237     plot_idx = 0
238     group_stats = {}
239     for var_name, label_map in available_cat_vars.items():
240         if plot_idx >= n_rows * n_cols:
241             break
242         row = plot_idx // n_cols
243         col = plot_idx % n_cols
244         ax = axes[row, col]
245         #输入必要数据
246         plot_data = df[[y_var, var_name]].dropna()
247         if len(plot_data) == 0:
248             ax.text(0.5, 0.5, f'{var_name}\n无可用数据', ha='center', va='center',
249                    transform=ax.transAxes, fontsize=12)
250             ax.set_title(var_name)
251             plot_idx += 1
252             continue
253         plot_data[f'{var_name}_label'] = plot_data[var_name].map(label_map)
254         #将箱线图+小提琴图绘制，并输出于同一张图中
255         parts = ax.violinplot([plot_data[plot_data[var_name] == val][y_var].values
256                               for val in sorted(plot_data[var_name].unique())],
257                               positions=range(len(label_map)), showmeans=True,
258                               showmedians=True)
259         #设置图形颜色
260         colors = plt.cm.Set3(np.linspace(0, 1, len(label_map)))
261         for pc, color in zip(parts['bodies'], colors):
262             pc.set_facecolor(color)
263             pc.set_alpha(0.7)
264         #绘制箱线图，并设置标题与字体
265         bp = ax.boxplot([plot_data[plot_data[var_name] == val][y_var].values
266                          for val in sorted(plot_data[var_name].unique())],
267                          positions=range(len(label_map)), widths=0.3, patch_artist=
268                          False,
269                          boxprops=dict(linewidth=2), medianprops=dict(linewidth=2,
270                                color='red'))
271         ax.set_xticks(range(len(label_map)))
272         ax.set_xticklabels([label_map[val] for val in sorted(plot_data[var_name].
273 unique())],
274                             rotation=45, ha='right')
275         ax.set_ylabel(y_var)
276         ax.set_title(f'{var_name} 分组对比')
277         ax.grid(True, alpha=0.3)
278
279         #计算组间统计并进行方差检验

```

```

276     group_data = []
277     for val in sorted(plot_data[var_name].unique()):
278         group_values = plot_data[plot_data[var_name] == val][y_var]
279         group_data.append(group_values)
280     if len(group_data) >= 2 and all(len(g) > 0 for g in group_data):
281         from scipy.stats import f_oneway, kruskal, levene
282         #使用Levene来检验方差齐性
283         levene_stat, levene_p = levene(*group_data)
284         if levene_p > 0.05:
285             #如果方差为齐性，使用ANOVA方法
286             f_stat, p_val = f_oneway(*group_data)
287             test_name = "ANOVA"
288         else:
289             #如果方差不齐，则用Kruskal-Wallis方法检验
290             f_stat, p_val = kruskal(*group_data)
291             test_name = "Kruskal-Wallis"
292             significance = "****" if p_val < 0.001 else "***" if p_val < 0.01 else "**"
293             " if p_val < 0.05 else "n.s."
294             #添加统计检验结果到图中
295             ax.text(0.02, 0.98, f'{test_name}: p={p_val:.4f} {significance}',
296                 transform=ax.transAxes, verticalalignment='top',
297                 bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.7))
298             group_stats[var_name] = {
299                 'test': test_name,
300                 'statistic': f_stat,
301                 'p_value': p_val,
302                 'significance': significance,
303                 'levene_p': levene_p
304             }
305             plot_idx += 1
306             #对于多余的子图，不显示
307             for i in range(plot_idx, n_rows * n_cols):
308                 row = i // n_cols
309                 col = i % n_cols
310                 axes[row, col].set_visible(False)
311             plt.tight_layout()
312             plt.savefig('分组对比分析.png', dpi=300, bbox_inches='tight')
313             plt.show()
314             #输出分组对比统计检验结果
315             print("分组对比统计检验结果:")
316             for var, stats_info in group_stats.items():
317                 print(f"• {var}: {stats_info['test']} p={stats_info['p_value']:.4f} {
318                     stats_info['significance']}")
319             return group_stats
320
321 def plot_scatter_relationships(df):
322     print("4. 散点图关系分析")

```

```

321
322
323 y_var = 'Y染色体浓度'
324 #对连续变量进行分析，选择表格中的连续变量
325 key_vars = ['孕妇BMI', '检测时怀孕天数', '年龄', 'X染色体浓度', 'Y染色体的Z值',
326             '18号染色体的Z值']
327 available_vars = [var for var in key_vars if var in df.columns]
328 if len(available_vars) < 2:
329     print("可用变量不足，跳过散点图分析")
330     return {}
331
332 #构建散点图矩阵
333 n_vars = len(available_vars)
334 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
335 axes = axes.flatten()
336 for i, var in enumerate(available_vars[:6]): # 最多显示6个
337     if i >= len(axes):
338         break
339     #输出已读取的数据
340     plot_data = df[[y_var, var]].dropna()
341     if len(plot_data) == 0:
342         axes[i].text(0.5, 0.5, f'{var}\n无数据', ha='center', va='center',
343                     transform=axes[i].transAxes)
344         continue
345     #绘制散点图
346     axes[i].scatter(plot_data[var], plot_data[y_var], alpha=0.6, s=20, color='
347     blue')
348
349     #在散点图的基础上添加回归线
350     try:
351         from sklearn.linear_model import LinearRegression
352         X = plot_data[var].values.reshape(-1, 1)
353         y = plot_data[y_var].values
354         reg = LinearRegression().fit(X, y)
355         x_line = np.linspace(plot_data[var].min(), plot_data[var].max(), 100)
356         y_line = reg.predict(x_line.reshape(-1, 1))
357         axes[i].plot(x_line, y_line, 'r-', linewidth=2, label=f'R2={reg.score(X
358         , y):.3f}')
359
360         #计算相关系数，用于后文图片绘制
361         corr_coef = plot_data[[var, y_var]].corr().iloc[0, 1]
362         axes[i].set_xlabel(var)
363         axes[i].set_ylabel(y_var)
364         axes[i].set_title(f'{var} vs {y_var}\n相关系数: {corr_coef:.4f}')
365         axes[i].legend()
366         axes[i].grid(True, alpha=0.3)
367     except Exception as e:

```

```

365         print(f"绘制 {var} 的回归线时出错: {e}")
366
367     #对于多余的子图, 隐藏不输出
368     for j in range(len(available_vars), len(axes)):
369         axes[j].set_visible(False)
370     plt.tight_layout()
371     plt.savefig('散点图关系分析.png', dpi=300, bbox_inches='tight')
372     plt.show()
373     return available_vars
374
375 def plot_statistical_results(correlation_results_file):
376     print("5. 统计检验结果可视化")
377
378     try:
379         #读取相关性检验结果
380         results_df = pd.read_csv(correlation_results_file, encoding='utf-8-sig')
381         #选出显著变量
382         significant_vars = results_df[
383             (results_df['p值'] < 0.05) &
384             (results_df['变量名'] != 'Y染色体浓度(因变量)')
385         ].copy()
386         if len(significant_vars) == 0:
387             print("没有发现显著变量")
388             return
389         #绘制森林图
390         fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 10))
391
392         #第一个, p值森林图
393         significant_vars_sorted = significant_vars.sort_values('p值')
394
395         #计算-log10, 用于后续可视化
396         neg_log_p = -np.log10(significant_vars_sorted['p值'])
397
398         colors = ['red' if p < 0.001 else 'orange' if p < 0.01 else 'yellow'
399                  for p in significant_vars_sorted['p值']]
400
401         bars1 = ax1.barh(range(len(significant_vars_sorted)), neg_log_p, color=
402             colors, alpha=0.7)
403         ax1.set_yticks(range(len(significant_vars_sorted)))
404         ax1.set_yticklabels(significant_vars_sorted['变量名'], fontsize=10)
405         ax1.set_xlabel('-log10(p值)')
406         ax1.set_title('显著性检验结果 (森林图)', fontsize=14)
407         #在图上添加显著性阈值线
408         ax1.axvline(-np.log10(0.05), color='green', linestyle='--', label='p=0.05')
409         ax1.axvline(-np.log10(0.01), color='blue', linestyle='--', label='p=0.01')
410         ax1.axvline(-np.log10(0.001), color='red', linestyle='--', label='p=0.001')
411         ax1.legend()

```

```

411     ax1.grid(True, alpha=0.3)
412     #在图片上添加p值标签
413     for i, (bar, p_val) in enumerate(zip(bars1, significant_vars_sorted['p值']))
414 ):
415         ax1.text(neg_log_p.iloc[i] + 0.1, i, f'p={p_val:.4f}',
416                 va='center', fontsize=8)
417
418     #第二歌，效应大小图
419     effect_sizes = significant_vars_sorted['相关系数/统计量'].abs()
420     colors2 = ['darkred' if abs(x) > 0.5 else 'red' if abs(x) > 0.3 else '
orange' if abs(x) > 0.1 else 'yellow'
421               for x in significant_vars_sorted['相关系数/统计量']]
422     bars2 = ax2.barh(range(len(significant_vars_sorted)), effect_sizes, color=
colors2, alpha=0.7)
423     ax2.set_yticks(range(len(significant_vars_sorted)))
424     ax2.set_yticklabels(significant_vars_sorted['变量名'], fontsize=10)
425     ax2.set_xlabel('效应大小 (|相关系数/统计量|)')
426     ax2.set_title('效应大小分析', fontsize=14)
427     ax2.grid(True, alpha=0.3)
428
429     #添加效应大小标签
430     for i, (bar, effect) in enumerate(zip(bars2, significant_vars_sorted['相关
系数/统计量'])):
431         ax2.text(abs(effect) + 0.01, i, f'{effect:.3f}',
432                 va='center', fontsize=8)
433     plt.tight_layout()
434     plt.savefig('统计检验结果可视化.png', dpi=300, bbox_inches='tight')
435     plt.show()
436     print(f"共发现 {len(significant_vars)} 个显著变量:")
437     for _, row in significant_vars_sorted.iterrows():
438         print(f"    • {row['变量名']}: {row['检验方法']}, p={row['p值']:.6f}")
439
440     return significant_vars_sorted
441 except Exception as e:
442     print(f"读取统计结果文件时出错: {e}")
443     return pd.DataFrame()
444
445 def comprehensive_visualization_analysis(df, correlation_results_file):
446     print("Y染色体浓度全面可视化分析报告")
447     results = {}
448
449     # 输出结果一，分布分析
450     results['distribution'] = plot_distribution_analysis(df)
451
452     #输出结果二，相关性热图
453     results['correlation'] = plot_correlation_heatmap(df)

```

```

454 #输出结果三，分组对比
455 results['group_comparison'] = plot_group_comparisons(df)
456
457 #输出结果四，散点图关系
458 results['scatter_relationships'] = plot_scatter_relationships(df)
459
460 #输出结果五，统计检验结果
461 results['statistical_results'] = plot_statistical_results(
correlation_results_file)
462
463 #总结，并生成综合报告
464 print("可视化分析综合报告")
465 print("\n数据概览:")
466 print(f" • 总样本数: {len(df)}")
467 print(f" • Y染色体浓度有效数据: {len(df['Y染色体浓度'].dropna())}")
468 print("\n分布特征:")
469 dist_stats = results['distribution']['stats']
470 print(f" • 均值: {dist_stats['mean']:.6f} ± {dist_stats['std']:.6f}")
471 print(f" • 正态性: {dist_stats['normality_test']} p={dist_stats['normality_p']:.6f}")
472 print(f" • 偏度: {dist_stats['skewness']:.4f}, 峰度: {dist_stats['kurtosis']:.4f}")
473 print(f" • 异常值: {dist_stats['outliers_count']} 个")
474 print("\n相关性发现:")
475 if 'y_correlations' in results['correlation']:
476     top_corrs = results['correlation']['y_correlations'].head(3)
477     for var, corr in top_corrs.items():
478         print(f" • {var}: r={corr:.4f}")
479 print("\n分组对比:")
480 if results['group_comparison']:
481     significant_groups = [k for k, v in results['group_comparison'].items() if
v['p_value'] < 0.05]
482     print(f" • 发现 {len(significant_groups)} 个显著分组差异")
483     for group in significant_groups:
484         print(f" - {group}: {results['group_comparison'][group]['
significance']}")
485 print("\n主要发现:")
486 print(" 1. Y染色体浓度数据显示非正态分布特征")
487 print(" 2. 存在多个显著相关的影响因素")
488 print(" 3. 不同分组间存在显著差异")
489 print(" 4. 建议使用非参数方法进行进一步分析")
490 return results
491
492
493 #代入数据使用
494 if __name__ == "__main__":
495     # 数据读取

```



```

496     df = pd.read_csv("男胎_预处理数据_完整版.csv")
497     correlation_file = "Y染色体浓度相关性检验结果.csv"
498     #综合可视化分析
499     analysis_results = comprehensive_visualization_analysis(df,
correlation_file)
500     #输出必要结果
501     print("生成以下图表文件：")
502     print(" • Y染色体浓度分布分析.png")
503     print(" • 相关性分析热图.png")
504     print(" • 分组对比分析.png")
505     print(" • 散点图关系分析.png")
506     print(" • 统计检验结果可视化.png")

```

多元线性回归详细解释和可视化.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5  from scipy import stats
6  from sklearn.model_selection import train_test_split, cross_val_score,
validation_curve
7  from sklearn.linear_model import LinearRegression, Ridge, RidgeCV
8  from sklearn.preprocessing import StandardScaler, PolynomialFeatures
9  from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
10 from sklearn.feature_selection import SelectKBest, f_regression, RFE
11 from statsmodels.stats.outliers_influence import variance_inflation_factor
12 import statsmodels.api as sm
13 from statsmodels.stats.diagnostic import het_breuschpagan, het_white
14 from statsmodels.stats.stattools import durbin_watson
15 import warnings
16 import os
17 warnings.filterwarnings('ignore')
18
19 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong']
20 plt.rcParams['axes.unicode_minus'] = False
21 plt.rcParams['font.size'] = 10
22
23 def enhanced_multiple_linear_regression(df):
24     # 因变量
25     y_var = 'Y染色体浓度'
26     # 基础特征集合
27     base_features = [
28         '孕妇BMI', '检测时怀孕天数', '年龄', '体重', '身高',
29         'X染色体浓度', '18号染色体的Z值', 'Y染色体的Z值',
30         '13号染色体的Z值', '21号染色体的Z值', 'X染色体的Z值',
31         'GC含量', '末次月经距检测日天数',
32         'IVF妊娠_编码', '怀孕次数_编码', '胎儿健康_编码',

```

```

33     'T13_异常', 'T18_异常', 'T21_异常'
34 ]
35
36 # 检查可用特征
37 available_features = [f for f in base_features if f in df.columns]
38 missing_features = [f for f in base_features if f not in df.columns]
39 # 数据清洗
40 model_data = df[[y_var] + available_features].dropna()
41 X = model_data[available_features]
42 y = model_data[y_var]
43 # 因变量基本统计
44 y_stats = {
45     '样本数': len(y),
46     '均值': y.mean(),
47     '中位数': y.median(),
48     '标准差': y.std(),
49     '最小值': y.min(),
50     '最大值': y.max(),
51     '偏度': stats.skew(y),
52     '峰度': stats.kurtosis(y)
53 }
54 for key, value in y_stats.items():
55     print(f" {key}: {value:.6f}")
56 # 因变量正态性检验
57 if len(y) > 5000:
58     normality_stat, normality_p = stats.normaltest(y)
59     normality_test = "D'Agostino-Pearson"
60 else:
61     normality_stat, normality_p = stats.shapiro(y)
62     normality_test = "Shapiro-Wilk"
63 print(f"\n因变量正态性检验 ({normality_test}):")
64 print(f" 统计量: {normality_stat:.4f}")
65 print(f" p值: {normality_p:.6f}")
66 print(f" 结论: {'符合正态分布' if normality_p > 0.05 else '不符合正态分布'}")
67 # 2. 特征工程和增强
68 X_engineered = X.copy()
69 new_features = []
70 # 交互特征
71 interaction_pairs = [
72     ('孕妇BMI', '检测时怀孕天数'),
73     ('年龄', '孕妇BMI'),
74     ('Y染色体的Z值', 'X染色体浓度'),
75     ('身高', '体重'),
76     ('18号染色体的Z值', 'Y染色体的Z值')
77 ]
78 for f1, f2 in interaction_pairs:
79     if f1 in X.columns and f2 in X.columns:

```

```

80         feature_name = f'{f1}_x_{f2}'
81         X_engineered[feature_name] = X[f1] * X[f2]
82         new_features.append(feature_name)
83         print(f" {feature_name}")
84 # 多项式特征 (二次项)
85 polynomial_vars = ['孕妇BMI', '检测时怀孕天数', '年龄', 'X染色体浓度']
86 for var in polynomial_vars:
87     if var in X.columns:
88         feature_name = f'{var}_平方'
89         X_engineered[feature_name] = X[var] ** 2
90         new_features.append(feature_name)
91         print(f" {feature_name}")
92 # 比率特征
93 ratio_pairs = [
94     ('体重', '身高', '体重身高比'),
95     ('Y染色体的Z值', 'X染色体的Z值', 'Y_X_Z值比率'),
96     ('末次月经距检测日天数', '检测时怀孕天数', '时间比率')
97 ]
98 for num_var, den_var, ratio_name in ratio_pairs:
99     if num_var in X.columns and den_var in X.columns:
100         X_engineered[ratio_name] = X[num_var] / (X[den_var] + 1e-8) # 避免除零
101         new_features.append(ratio_name)
102         print(f" {ratio_name} = {num_var} / {den_var}")
103 # 变换特征
104 transform_vars = ['孕妇BMI', 'X染色体浓度']
105 for var in transform_vars:
106     if var in X.columns:
107         # 对数变换
108         log_name = f'{var}_对数'
109         X_engineered[log_name] = np.log1p(X[var]) # log(1+x)避免负值问题
110         new_features.append(log_name)
111         print(f" {log_name}")
112         # 平方根变换
113         sqrt_name = f'{var}_平方根'
114         X_engineered[sqrt_name] = np.sqrt(X[var])
115         new_features.append(sqrt_name)
116         print(f" {sqrt_name}")
117 print(f"\n特征工程结果:")
118 print(f" 原始特征数: {len(available_features)}")
119 print(f" 新增特征数: {len(new_features)}")
120 print(f" 总特征数: {X_engineered.shape[1]}")
121 # 3. 特征相关性分析和选择
122 # 计算特征与因变量的相关性
123 correlations = []
124 for feature in X_engineered.columns:
125     # 使用Spearman相关系数 (对非正态分布更robust)
126     corr, p_value = stats.spearmanr(X_engineered[feature], y)

```

```

127     correlations.append({
128         '特征名': feature,
129         '相关系数': corr,
130         'p值': p_value,
131         '显著性': '***' if p_value < 0.001 else '**' if p_value < 0.01 else '*'
132         if p_value < 0.05 else 'n.s.',
133         '绝对相关系数': abs(corr)
134     })
135 corr_df = pd.DataFrame(correlations).sort_values('绝对相关系数', ascending=
False)
136 print("特征与因变量相关性分析 (Top 15):")
137 top_corr = corr_df.head(15)[['特征名', '相关系数', 'p值', '显著性']]
138 for _, row in top_corr.iterrows():
139     print(f"   {row['特征名']}: {row['相关系数']:.4f} (p={row['p值']:.6f}) {row
140     ['显著性']}")
141 # 选择显著相关的特征
142 significant_features = corr_df[
143     (corr_df['p值'] < 0.05) & (corr_df['绝对相关系数'] > 0.01)
144 ][['特征名']].tolist()
145 print(f"\n显著相关特征数: {len(significant_features)}")
146 # 4. 多重共线性诊断
147 # 选择前20个最相关的特征进行VIF分析
148 selected_for_vif = corr_df.head(20)['特征名'].tolist()
149 X_for_vif = X_engineered[selected_for_vif]
150 # 标准化数据
151 scaler_vif = StandardScaler()
152 X_vif_scaled = scaler_vif.fit_transform(X_for_vif)
153 X_vif_df = pd.DataFrame(X_vif_scaled, columns=X_for_vif.columns)
154 # 计算VIF
155 vif_data = pd.DataFrame()
156 vif_data["特征"] = X_for_vif.columns
157 vif_data["VIF"] = [variance_inflation_factor(X_vif_df.values, i)
158                     for i in range(len(X_for_vif.columns))]
159 vif_data = vif_data.sort_values('VIF', ascending=False)
160 print("方差膨胀因子 (VIF) 分析:")
161 for _, row in vif_data.iterrows():
162     if row['VIF'] > 10:
163         status = "严重共线性"
164     elif row['VIF'] > 5:
165         status = "中等共线性"
166     else:
167         status = "轻微共线性"
168     print(f"   {row['特征']}: {row['VIF']:.3f} ({status})")
169 # 移除高VIF特征
170 high_vif_features = vif_data[vif_data['VIF'] > 10]['特征'].tolist()
171 if high_vif_features:
172     print(f"\n移除高VIF特征: {high_vif_features}")

```

```

171     final_features = [f for f in selected_for_vif if f not in high_vif_features
172 ]
173 else:
174     final_features = selected_for_vif[:15] # 选择前15个特征
175     print(f"\n无需移除高VIF特征，选择前15个相关特征")
176 X_final = X_engineered[final_features]
177 print(f"最终建模特征数: {len(final_features)}")
178 # 5. 数据分割和标准化
179 # 分割数据
180 X_train, X_test, y_train, y_test = train_test_split(
181     X_final, y, test_size=0.2, random_state=42, stratify=None
182 )
183 print(f"训练集大小: {X_train.shape}")
184 print(f"测试集大小: {X_test.shape}")
185 # 标准化特征
186 scaler = StandardScaler()
187 X_train_scaled = scaler.fit_transform(X_train)
188 X_test_scaled = scaler.transform(X_test)
189 # 转换为DataFrame便于后续分析
190 X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns, index=
191 X_train.index)
192 X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_test.columns, index=
193 X_test.index)
194 print("特征标准化完成 (均值=0, 标准差=1)")
195 # 6. 模型建立和训练
196 # 使用RidgeCV自动选择最优正则化参数
197 alphas = np.logspace(-3, 3, 50)
198 ridge_cv = RidgeCV(alphas=alphas, cv=10, scoring='r2')
199 ridge_cv.fit(X_train_scaled, y_train)
200 optimal_alpha = ridge_cv.alpha_
201 print(f"最优正则化参数  $\alpha$ : {optimal_alpha:.6f}")
202 # 使用最优参数的Ridge回归
203 ridge_model = Ridge(alpha=optimal_alpha)
204 ridge_model.fit(X_train_scaled, y_train)
205 # 同时建立普通线性回归用于对比
206 lr_model = LinearRegression()
207 lr_model.fit(X_train_scaled, y_train)
208 # 使用statsmodels进行详细统计分析
209 X_train_const = sm.add_constant(X_train_scaled)
210 sm_model = sm.OLS(y_train, X_train_const).fit()
211 print("\nStatsmodels回归结果:")
212 print(sm_model.summary())
213 # 7. 模型预测和评估
214 # Ridge回归预测
215 y_train_pred_ridge = ridge_model.predict(X_train_scaled)
216 y_test_pred_ridge = ridge_model.predict(X_test_scaled)
217 # 普通线性回归预测

```

```

215 y_train_pred_lr = lr_model.predict(X_train_scaled)
216 y_test_pred_lr = lr_model.predict(X_test_scaled)
217 # 计算评估指标
218 def calculate_metrics(y_true, y_pred, model_name):
219     r2 = r2_score(y_true, y_pred)
220     rmse = np.sqrt(mean_squared_error(y_true, y_pred))
221     mae = mean_absolute_error(y_true, y_pred)
222     mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100 # 平均绝对百分比
误差
223     return {
224         'model': model_name,
225         'r2': r2,
226         'rmse': rmse,
227         'mae': mae,
228         'mape': mape
229     }
230 # 评估结果
231 metrics = []
232 metrics.append(calculate_metrics(y_train, y_train_pred_ridge, 'Ridge训练集'))
233 metrics.append(calculate_metrics(y_test, y_test_pred_ridge, 'Ridge测试集'))
234 metrics.append(calculate_metrics(y_train, y_train_pred_lr, '线性回归训练集'))
235 metrics.append(calculate_metrics(y_test, y_test_pred_lr, '线性回归测试集'))
236 print("模型性能比较:")
237 print(f"{'模型':<15} {'R²':<8} {'RMSE':<12} {'MAE':<12} {'MAPE(%)':<10}")
238 for m in metrics:
239     print(f"{m['model']:<15} {m['r2']:<8.4f} {m['rmse']:<12.6f} {m['mae']:<12.6f} {m['mape']:<10.2f}")
240 # 交叉验证评估
241 cv_scores_ridge = cross_val_score(ridge_model, X_train_scaled, y_train, cv=10,
scoring='r2')
242 cv_scores_lr = cross_val_score(lr_model, X_train_scaled, y_train, cv=10,
scoring='r2')
243 print(f"\n10折交叉验证结果:")
244 print(f"Ridge回归: R² = {cv_scores_ridge.mean():.4f} ± {cv_scores_ridge.std():.4f}")
245 print(f"线性回归: R² = {cv_scores_lr.mean():.4f} ± {cv_scores_lr.std():.4f}")
246 # 8. 模型系数详细分析
247 # 创建系数分析DataFrame
248 coef_analysis = pd.DataFrame({
249     '特征名': X_train.columns,
250     'Ridge系数': ridge_model.coef_,
251     '线性回归系数': lr_model.coef_,
252     '标准化系数_Ridge': ridge_model.coef_, # 已经是标准化的
253     '标准化系数_线性': lr_model.coef_,
254     '原始特征标准差': X_train.std().values,
255     'Ridge重要性': np.abs(ridge_model.coef_),
256     '线性回归重要性': np.abs(lr_model.coef_)

```

```

257     })
258     # 添加统计显著性 (从statsmodels结果)
259     if len(X_train.columns) <= len(sm_model.pvalues) - 1:
260         coef_analysis['p值'] = sm_model.pvalues[1:len(X_train.columns)+1].values
261         coef_analysis['置信区间下限'] = sm_model.conf_int()[0][1:len(X_train.
columns)+1].values
262         coef_analysis['置信区间上限'] = sm_model.conf_int()[1][1:len(X_train.
columns)+1].values
263         coef_analysis['显著性'] = coef_analysis['p值'].apply(
264             lambda p: '***' if p < 0.001 else '**' if p < 0.01 else '*' if p < 0.05
else 'n.s.'
265         )
266     # 按Ridge回归系数重要性排序
267     coef_analysis = coef_analysis.sort_values('Ridge重要性', ascending=False)
268     print("回归系数详细分析 (按Ridge重要性排序):")
269     print(f"{'特征':<20} {'Ridge系数':<10} {'线性系数':<10} {'p值':<10} {'显著性'
':<8}")
270     print("-" * 70)
271     for _, row in coef_analysis.iterrows():
272         p_val_str = f"{row.get('p值', np.nan):.6f}" if not pd.isna(row.get('p值',
np.nan)) else "N/A"
273         sig_str = row.get('显著性', 'N/A')
274         print(f"{row['特征名']:<20} {row['Ridge系数']:<10.6f} {row['线性回归系数'
']:<10.6f} {p_val_str:<10} {sig_str:<8}")
275
276     # 9. 模型诊断和假设检验
277     # 使用Ridge回归的残差进行诊断
278     residuals_train = y_train - y_train_pred_ride
279     residuals_test = y_test - y_test_pred_ride
280     # 1) 残差正态性检验
281     if len(residuals_train) > 5000:
282         res_stat, res_p = stats.normaltest(residuals_train)
283         res_test_name = "D'Agostino-Pearson"
284     else:
285         res_stat, res_p = stats.shapiro(residuals_train)
286         res_test_name = "Shapiro-Wilk"
287
288     print(f"残差正态性检验 ({res_test_name}):")
289     print(f"    统计量: {res_stat:.4f}")
290     print(f"    p值: {res_p:.6f}")
291     print(f"    结论: {'残差符合正态分布' if res_p > 0.05 else '残差不符合正态分布'}")
292
293     # 2) 残差同方差性检验 (Breusch-Pagan test)
294     try:
295         bp_stat, bp_p, _, _ = het_breuschpagan(residuals_train, X_train_const)
296         print(f"\n残差同方差性检验 (Breusch-Pagan):")

```

```

297     print(f"    统计量: {bp_stat:.4f}")
298     print(f"    p值: {bp_p:.6f}")
299     print(f"    结论: {'残差方差齐性' if bp_p > 0.05 else '存在异方差性'}")
300 except:
301     print("\n无法进行Breusch-Pagan检验")
302
303 # 3) 残差自相关检验 (Durbin-Watson)
304 dw_stat = durbin_watson(residuals_train)
305 print(f"\n残差自相关检验 (Durbin-Watson):")
306 print(f"    统计量: {dw_stat:.4f}")
307 if 1.5 <= dw_stat <= 2.5:
308     dw_conclusion = "无明显自相关"
309 elif dw_stat < 1.5:
310     dw_conclusion = "可能存在正自相关"
311 else:
312     dw_conclusion = "可能存在负自相关"
313 print(f"    结论: {dw_conclusion}")
314
315 # 4) 离群值检测
316 z_scores = np.abs(stats.zscore(residuals_train))
317 outliers_count = np.sum(z_scores > 3)
318 print(f"\n离群值检测 (|Z-score| > 3):")
319 print(f"    离群值数量: {outliers_count}")
320 print(f"    离群值比例: {outliers_count/len(residuals_train)*100:.2f}%")
321
322 # 5) Cook距离 (影响力分析)
323 try:
324     influence = sm_model.get_influence()
325     cook_d = influence.cooks_distance[0]
326     high_influence = np.sum(cook_d > 4/len(y_train))
327     print(f"\n影响力分析 (Cook距离 > 4/n):")
328     print(f"    高影响力点数量: {high_influence}")
329     print(f"    高影响力点比例: {high_influence/len(y_train)*100:.2f}%")
330 except:
331     print("\n无法计算Cook距离")
332
333 # 10. 模型可视化分析
334 # [8]AI协助添加过程可视化使其更具可解释性
335 # 创建综合可视化
336 fig = plt.figure(figsize=(20, 15))
337 # 子图1: 实际值 vs 预测值
338 ax1 = plt.subplot(3, 3, 1)
339 plt.scatter(y_test, y_test_pred_ridge, alpha=0.6, color='blue', s=30)
340 min_val = min(y_test.min(), y_test_pred_ridge.min())
341 max_val = max(y_test.max(), y_test_pred_ridge.max())
342 plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, label='完美预测线')

```



```

343 plt.xlabel('实际值')
344 plt.ylabel('预测值')
345 plt.title(f'预测效果 ( $R^2 = \{metrics[1]["r2"]:.3f\}$ )')
346 plt.legend()
347 plt.grid(True, alpha=0.3)
348 # 子图2: 残差图
349 ax2 = plt.subplot(3, 3, 2)
350 plt.scatter(y_test_pred_ridge, residuals_test, alpha=0.6)
351 plt.axhline(y=0, color='red', linestyle='--')
352 plt.xlabel('预测值')
353 plt.ylabel('残差')
354 plt.title('残差分布图')
355 plt.grid(True, alpha=0.3)
356 # 子图3: 残差直方图
357 ax3 = plt.subplot(3, 3, 3)
358 plt.hist(residuals_test, bins=20, alpha=0.7, color='skyblue', edgecolor='black'
359 )
360 plt.axvline(residuals_test.mean(), color='red', linestyle='--', label=f'均值: {
361 residuals_test.mean():.6f}')
362 plt.xlabel('残差')
363 plt.ylabel('频数')
364 plt.title('残差分布直方图')
365 plt.legend()
366 plt.grid(True, alpha=0.3)
367 # 子图4: Q-Q图
368 ax4 = plt.subplot(3, 3, 4)
369 stats.probplot(residuals_test, dist="norm", plot=plt)
370 plt.title('残差正态性Q-Q图')
371 plt.grid(True, alpha=0.3)
372 # 子图5: 系数重要性
373 ax5 = plt.subplot(3, 3, 5)
374 top_10_coef = coef_analysis.head(10)
375 colors = ['red' if x < 0 else 'blue' for x in top_10_coef['Ridge系数']]
376 bars = plt.barh(range(len(top_10_coef)), top_10_coef['Ridge系数'], color=colors
377 , alpha=0.7)
378 plt.yticks(range(len(top_10_coef)), top_10_coef['特征名'], fontsize=8)
379 plt.xlabel('标准化系数')
380 plt.title('Top 10 特征重要性')
381 plt.axvline(x=0, color='black', linestyle='-', alpha=0.3)
382 plt.grid(True, alpha=0.3)
383 # 子图6: 特征相关性热力图
384 ax6 = plt.subplot(3, 3, 6)
385 top_features = coef_analysis.head(8)['特征名'].tolist()
386 corr_matrix = X_train[top_features].corr()
387 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, square=True,
388             fmt='.2f', cbar_kws={'shrink': 0.8})
389 plt.title('Top 8 特征相关性矩阵')

```

```

387 # 子图7: 验证曲线 (正则化参数)
388 ax7 = plt.subplot(3, 3, 7)
389 if len(alphas) > 1:
390     train_scores, val_scores = validation_curve(
391         Ridge(), X_train_scaled, y_train,
392         param_name='alpha', param_range=alphas, cv=5, scoring='r2'
393     )
394     plt.semilogx(alphas, np.mean(train_scores, axis=1), 'b-', label='训练集 R2',
395                 linewidth=2)
396     plt.semilogx(alphas, np.mean(val_scores, axis=1), 'r-', label='验证集 R2',
397                 linewidth=2)
398     plt.axvline(x=optimal_alpha, color='green', linestyle='--', label=f'最优 $\alpha$ ={
399         optimal_alpha:.4f}')
400     plt.xlabel('正则化参数  $\alpha$ ')
401     plt.ylabel('R2 分数')
402     plt.title('Ridge 回归验证曲线')
403     plt.legend()
404     plt.grid(True, alpha=0.3)
405 # 子图8: 残差vs主要特征
406 if '孕妇BMI' in X_test.columns:
407     ax8 = plt.subplot(3, 3, 8)
408     plt.scatter(X_test['孕妇BMI'], residuals_test, alpha=0.6)
409     plt.axhline(y=0, color='red', linestyle='--')
410     plt.xlabel('孕妇BMI')
411     plt.ylabel('残差')
412     plt.title('残差 vs 孕妇BMI')
413     plt.grid(True, alpha=0.3)
414 # 子图9: 学习曲线 (样本量vs性能)
415 ax9 = plt.subplot(3, 3, 9)
416 from sklearn.model_selection import learning_curve
417 train_sizes, train_scores_lc, val_scores_lc = learning_curve(
418     ridge_model, X_train_scaled, y_train, cv=5,
419     train_sizes=np.linspace(0.1, 1.0, 10), scoring='r2'
420 )
421 plt.plot(train_sizes, np.mean(train_scores_lc, axis=1), 'o-', label='训练集')
422 plt.plot(train_sizes, np.mean(val_scores_lc, axis=1), 'o-', label='验证集')
423 plt.xlabel('训练样本数')
424 plt.ylabel('R2 分数')
425 plt.title('学习曲线')
426 plt.legend()
427 plt.grid(True, alpha=0.3)
428 plt.tight_layout()
429 plt.savefig('多元线性回归详细分析.png', dpi=300, bbox_inches='tight', facecolor
430             ='white')
431 plt.show()
432 # 11. 特征重要性深度分析
433 # 特征类型分类

```

```

430 def classify_feature_type(feature_name):
431     if any(marker in feature_name for marker in ['_x_', 'x']):
432         return '交互特征'
433     elif '平方' in feature_name:
434         return '多项式特征'
435     elif any(marker in feature_name for marker in ['比', '比率', '/']):
436         return '比率特征'
437     elif any(marker in feature_name for marker in ['对数', '平方根']):
438         return '变换特征'
439     elif feature_name in ['IVF妊娠_编码', '怀孕次数_编码', '胎儿健康_编码', 'T13_异常', 'T18_异常', 'T21_异常']:
440         return '分类特征'
441     else:
442         return '原始特征'
443
444 coef_analysis['特征类型'] = coef_analysis['特征名'].apply(classify_feature_type)
445
446 # 按特征类型汇总
447 feature_type_summary = coef_analysis.groupby('特征类型').agg({
448     'Ridge重要性': ['count', 'mean', 'sum'],
449     'Ridge系数': lambda x: np.sum(np.abs(x))
450 }).round(4)
451 print("按特征类型统计:")
452 print(feature_type_summary)
453 # 最重要的特征分析
454 top_5_features = coef_analysis.head(5)
455 print(f"\n最重要的5个特征:")
456 for i, (_, row) in enumerate(top_5_features.iterrows(), 1):
457     effect = "正向影响" if row['Ridge系数'] > 0 else "负向影响"
458     significance = row.get('显著性', 'N/A')
459     print(f" {i}. {row['特征名']} ({row['特征类型']})")
460     print(f"    系数: {row['Ridge系数']:.6f} ({effect})")
461     print(f"    统计显著性: {significance}")
462     print()
463 # 12. 模型解释和业务洞察
464 print("模型性能总结:")
465 print(f" • 模型类型: Ridge回归 (正则化参数 $\alpha$ ={optimal_alpha:.6f})")
466 print(f" • 特征数量: {len(final_features)}")
467 print(f" • 解释方差: {metrics[1]['r2']*100:.2f}%")
468 print(f" • 平均绝对误差: {metrics[1]['mae']:.6f}")
469 print(f" • 相对误差: {metrics[1]['mape']:.2f}%")
470
471 print("\n关键业务洞察:")
472
473 # 分析最重要的特征
474 most_important = coef_analysis.iloc[0]
475 print(f" 1. 最关键预测因子: {most_important['特征名']}")

```

```

475 print(f"      影响程度: {abs(most_important['Ridge系数']):.6f}")
476 print(f"      影响方向: {'增加' if most_important['Ridge系数'] > 0 else '降低'}Y
染色体浓度")
477
478 # 分析原始特征 vs 工程特征的贡献
479 original_features = coef_analysis[coef_analysis['特征类型'] == '原始特征']
480 engineered_features = coef_analysis[coef_analysis['特征类型'] != '原始特征']
481
482 if len(original_features) > 0 and len(engineered_features) > 0:
483     orig_contrib = original_features['Ridge重要性'].sum()
484     eng_contrib = engineered_features['Ridge重要性'].sum()
485     total_contrib = orig_contrib + eng_contrib
486
487     print(f"\n 2. 特征工程效果分析:")
488     print(f"      原始特征贡献: {orig_contrib/total_contrib*100:.1f}%")
489     print(f"      工程特征贡献: {eng_contrib/total_contrib*100:.1f}%")
490     print(f"      特征工程{'有效' if eng_contrib > orig_contrib*0.3 else '效果有
限'}")
491
492 # 模型稳定性分析
493 print(f"\n 3. 模型稳定性:")
494 train_test_diff = abs(metrics[0]['r2'] - metrics[1]['r2'])
495 if train_test_diff < 0.05:
496     stability = "非常稳定"
497 elif train_test_diff < 0.1:
498     stability = "稳定"
499 else:
500     stability = "可能过拟合"
501 print(f"      训练集R²: {metrics[0]['r2']:.4f}")
502 print(f"      测试集R²: {metrics[1]['r2']:.4f}")
503 print(f"      稳定性评估: {stability}")
504
505 # 13. 模型方程和预测公式
506 print("标准化回归方程 (用于理解特征相对重要性):")
507 intercept_scaled = ridge_model.intercept_
508 print(f"Y染色体浓度_预测 = {intercept_scaled:.6f}")
509
510 for i, (feature, coef) in enumerate(zip(X_train.columns, ridge_model.coef_)):
511     sign = "+" if coef >= 0 else "-"
512     print(f"      {sign} {coef:.6f} × {feature}_标准化")
513
514 print("\n实际预测公式 (用于实际预测):")
515 print("步骤1: 特征标准化")
516 print("      标准化特征 = (原始特征 - 特征均值) / 特征标准差")
517 print("步骤2: 应用回归方程")
518 print("步骤3: 得到预测结果")
519

```

```

520 # 14. 保存详细分析结果
521 # 保存系数分析
522 coef_analysis.to_csv("多元线性回归系数详细分析.csv", index=False, encoding='utf
523 -8-sig')
524
525 # 保存模型评估
526 metrics_df = pd.DataFrame(metrics)
527 metrics_df.to_csv("多元线性回归模型评估.csv", index=False, encoding='utf-8-sig'
528 )
529
530 # 保存预测结果
531 prediction_analysis = pd.DataFrame({
532     '样本序号': range(len(y_test)),
533     '实际值': y_test.values,
534     '预测值': y_test_pred_ridge,
535     '残差': residuals_test,
536     '绝对误差': np.abs(residuals_test),
537     '相对误差(%)': np.abs(residuals_test) / y_test.values * 100,
538     '误差等级': pd.cut(np.abs(residuals_test), bins=3, labels=['小误差', '中等
539 误差', '大误差'])
540 })
541 prediction_analysis.to_csv("多元线性回归预测详细分析.csv", index=False,
542 encoding='utf-8-sig')
543
544 # 保存特征相关性
545 corr_df.to_csv("特征相关性详细分析.csv", index=False, encoding='utf-8-sig')
546
547 # 保存诊断结果
548 diagnostics = pd.DataFrame([
549     {
550         '检验项目': '残差正态性',
551         '检验方法': res_test_name,
552         '统计量': res_stat,
553         'p值': res_p,
554         '结论': '符合正态分布' if res_p > 0.05 else '不符合正态分布'
555     }, {
556         '检验项目': '残差自相关',
557         '检验方法': 'Durbin-Watson',
558         '统计量': dw_stat,
559         'p值': np.nan,
560         '结论': dw_conclusion
561     }
562 ])
563
564 try:
565     diagnostics = pd.concat([diagnostics, pd.DataFrame([
566         {
567             '检验项目': '残差同方差性',
568             '检验方法': 'Breusch-Pagan',
569             '统计量': bp_stat,

```

```

563         'p值': bp_p,
564         '结论': '方差齐性' if bp_p > 0.05 else '存在异方差性'
565     ]]), ignore_index=True)
566 except:
567     pass
568
569     diagnostics.to_csv("模型诊断结果.csv", index=False, encoding='utf-8-sig')
570
571     print("已保存以下分析文件:")
572     print(" • 多元线性回归系数详细分析.csv - 所有系数和重要性分析")
573     print(" • 多元线性回归模型评估.csv - 模型性能指标")
574     print(" • 多元线性回归预测详细分析.csv - 每个样本的预测详情")
575     print(" • 特征相关性详细分析.csv - 特征与因变量相关性")
576     print(" • 模型诊断结果.csv - 模型假设检验结果")
577     print(" • 多元线性回归详细分析.png - 综合可视化图表")
578
579     return {
580         'ridge_model': ridge_model,
581         'lr_model': lr_model,
582         'sm_model': sm_model,
583         'scaler': scaler,
584         'coefficients': coef_analysis,
585         'metrics': metrics_df,
586         'predictions': prediction_analysis,
587         'correlations': corr_df,
588         'diagnostics': diagnostics,
589         'optimal_alpha': optimal_alpha,
590         'feature_names': final_features,
591         'X_train': X_train,
592         'X_test': X_test,
593         'y_train': y_train,
594         'y_test': y_test
595     }
596 if __name__ == "__main__":
597     current_dir = os.path.dirname(os.path.abspath(__file__))
598     file_path = os.path.join(current_dir, "男胎_预处理数据_完整版.csv")
599     df = pd.read_csv(file_path)
600     results = enhanced_multiple_linear_regression(df)

```

问题 1 岭回归.py

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6 from sklearn.model_selection import train_test_split, cross_val_score,
    validation_curve

```

```

7 from sklearn.linear_model import Ridge, RidgeCV
8 from sklearn.preprocessing import StandardScaler
9 from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong']
14 plt.rcParams['axes.unicode_minus'] = False
15 plt.rcParams['font.size'] = 10
16
17 def ridge_regression_detailed_analysis(df):
18     # 1. 数据准备和特征工程
19     # 因变量
20     y_var = 'Y染色体浓度'
21     # 基础特征
22     base_features = [
23         '孕妇BMI', '检测时怀孕天数', '年龄', '体重', '身高',
24         'X染色体浓度', '18号染色体的Z值', 'Y染色体的Z值',
25         '13号染色体的Z值', '21号染色体的Z值', 'X染色体的Z值',
26         'GC含量', '末次月经距检测日天数',
27         'IVF妊娠_编码', '怀孕次数_编码', '胎儿健康_编码',
28         'T13_异常', 'T18_异常', 'T21_异常'
29     ]
30     # 检查可用特征
31     available_features = [f for f in base_features if f in df.columns]
32     model_data = df[[y_var] + available_features].dropna()
33     print(f"建模样本数: {len(model_data)} (原始: {len(df)})")
34     print(f"使用特征数: {len(available_features)}")
35
36     X = model_data[available_features]
37     y = model_data[y_var]
38
39     # 特征工程
40     X_engineered = X.copy()
41
42     # 添加重要的交互特征
43     interaction_features = []
44     if '孕妇BMI' in X.columns and '检测时怀孕天数' in X.columns:
45         X_engineered['BMI_x_孕周'] = X['孕妇BMI'] * X['检测时怀孕天数']
46         interaction_features.append('BMI_x_孕周')
47
48     if '年龄' in X.columns and '孕妇BMI' in X.columns:
49         X_engineered['年龄_x_BMI'] = X['年龄'] * X['孕妇BMI']
50         interaction_features.append('年龄_x_BMI')
51
52     if 'Y染色体的Z值' in X.columns and 'X染色体浓度' in X.columns:
53         X_engineered['Y_Z值_x_X浓度'] = X['Y染色体的Z值'] * X['X染色体浓度']

```

```

54     interaction_features.append('Y_Z值_x_X浓度')
55
56     # 添加多项式特征
57     polynomial_features = []
58     poly_vars = ['孕妇BMI', '检测时怀孕天数', '年龄']
59     for var in poly_vars:
60         if var in X.columns:
61             X_engineered[f'{var}_平方'] = X[var] ** 2
62             polynomial_features.append(f'{var}_平方')
63
64     print(f"添加交互特征: {interaction_features}")
65     print(f"添加多项式特征: {polynomial_features}")
66     print(f"最终特征数: {X_engineered.shape[1]}")
67
68     # 2. 特征重要性预分析
69     # 计算所有特征与Y染色体浓度的相关系数
70     correlations = []
71     for col in X_engineered.columns:
72         corr, p_value = stats.spearmanr(X_engineered[col], y)
73         correlations.append({
74             '特征名': col,
75             '相关系数': corr,
76             'p值': p_value,
77             '显著性': '***' if p_value < 0.001 else '**' if p_value < 0.01 else '*'
78             if p_value < 0.05 else 'n.s.',
79             '绝对相关系数': abs(corr)
80         })
81
82     corr_df = pd.DataFrame(correlations).sort_values('绝对相关系数', ascending=
83     False)
84     print("特征相关性排序 (Top 15):")
85     print(corr_df.head(15)[['特征名', '相关系数', 'p值', '显著性']].to_string(index
86     =False, float_format='%.4f'))
87
88     # 3. 数据分割和标准化
89     X_train, X_test, y_train, y_test = train_test_split(
90         X_engineered, y, test_size=0.2, random_state=42
91     )
92
93     # 标准化
94     scaler = StandardScaler()
95     X_train_scaled = scaler.fit_transform(X_train)
96     X_test_scaled = scaler.transform(X_test)
97
98     # 转换为DataFrame保持列名
99     X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns, index
100     =X_train.index)

```



```

97 X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_test.columns, index=
X_test.index)
98
99 print(f"训练集样本数: {len(X_train)}")
100 print(f"测试集样本数: {len(X_test)}")
101 print("特征已标准化 (均值=0, 标准差=1)")
102
103 # 4. Ridge回归超参数优化
104 # 使用交叉验证选择最优alpha
105 alphas = np.logspace(-3, 3, 50) # 从0.001到1000
106 ridge_cv = RidgeCV(alphas=alphas, cv=5, scoring='r2')
107 ridge_cv.fit(X_train_scaled, y_train)
108
109 optimal_alpha = ridge_cv.alpha_
110 print(f"最优正则化参数  $\alpha$  = {optimal_alpha:.6f}")
111
112 # 绘制validation curve
113 train_scores, val_scores = validation_curve(
114     Ridge(), X_train_scaled, y_train,
115     param_name='alpha', param_range=alphas, cv=5, scoring='r2'
116 )
117
118 plt.figure(figsize=(12, 8))
119 # [8]AI协助添加过程可视化使其更具可解释性
120 # 子图1: 验证曲线
121 plt.subplot(2, 2, 1)
122 plt.semilogx(alphas, np.mean(train_scores, axis=1), 'b-', label='训练集R2',
linewidth=2)
123 plt.semilogx(alphas, np.mean(val_scores, axis=1), 'r-', label='验证集R2',
linewidth=2)
124 plt.axvline(x=optimal_alpha, color='green', linestyle='--', label=f'最优 $\alpha$ ={
optimal_alpha:.4f}')
125 plt.xlabel('正则化参数  $\alpha$ ')
126 plt.ylabel('R2 分数')
127 plt.title('Ridge回归超参数优化')
128 plt.legend()
129 plt.grid(True, alpha=0.3)
130
131 # 5. 构建最优Ridge模型
132 ridge_model = Ridge(alpha=optimal_alpha)
133 ridge_model.fit(X_train_scaled, y_train)
134
135 # 预测
136 y_train_pred = ridge_model.predict(X_train_scaled)
137 y_test_pred = ridge_model.predict(X_test_scaled)
138
139 # 计算指标

```

```

140 train_r2 = r2_score(y_train, y_train_pred)
141 test_r2 = r2_score(y_test, y_test_pred)
142 test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))
143 test_mae = mean_absolute_error(y_test, y_test_pred)
144
145 print(f"训练集 R2 = {train_r2:.4f}")
146 print(f"测试集 R2 = {test_r2:.4f}")
147 print(f"RMSE = {test_rmse:.6f}")
148 print(f"MAE = {test_mae:.6f}")
149
150 # 6. Ridge模型系数分析
151 print("\n6. Ridge模型系数详细分析")
152 print("-"*60)
153
154 # 获取系数
155 coefficients = ridge_model.coef_
156 feature_names = X_train.columns
157
158 # 创建系数DataFrame
159 coef_df = pd.DataFrame({
160     '特征名': feature_names,
161     '原始系数': coefficients,
162     '标准化系数': coefficients, # 因为输入已标准化
163     '系数绝对值': np.abs(coefficients)
164 }).sort_values('系数绝对值', ascending=False)
165
166 # 计算特征对标准差变化的影响
167 coef_df['标准差影响'] = coef_df['标准化系数'] * X_train.std()
168 coef_df['特征类型'] = coef_df['特征名'].apply(lambda x:
169     '交互特征' if any(inter in x for inter in ['_x_', 'x']) else
170     '多项式特征' if '平方' in x else
171     '分类特征' if x in ['IVF妊娠_编码', '怀孕次数_编码', '胎儿健康_编码', 'T13_
172     异常', 'T18_异常', 'T21_异常'] else
173     '连续特征'
174 )
175
176 print("Ridge回归系数排序 (按绝对值):")
177 print(coef_df[['特征名', '标准化系数', '系数绝对值', '特征类型']].head(15).
178 to_string(index=False, float_format='%.6f'))
179
180 # 子图2: 系数重要性条形图
181 plt.subplot(2, 2, 2)
182 top_features = coef_df.head(12)
183 colors = ['red' if x < 0 else 'blue' for x in top_features['标准化系数']]
184 bars = plt.barh(range(len(top_features)), top_features['标准化系数'], color=
185 colors, alpha=0.7)
186 plt.yticks(range(len(top_features)), top_features['特征名'], fontsize=8)

```

```

184 plt.xlabel('标准化系数')
185 plt.title('Top 12 特征系数')
186 plt.axvline(x=0, color='black', linestyle='--', alpha=0.3)
187 plt.grid(True, alpha=0.3)
188
189 # 添加数值标签
190 for i, (bar, value) in enumerate(zip(bars, top_features['标准化系数'])):
191     plt.text(value + (0.005 if value > 0 else -0.005), i, f'{value:.3f}',
192             va='center', ha='left' if value > 0 else 'right', fontsize=7)
193
194 # 7. 特征类型分析
195 feature_type_analysis = coef_df.groupby('特征类型').agg({
196     '系数绝对值': ['count', 'mean', 'sum'],
197     '标准化系数': lambda x: np.sum(np.abs(x))
198 }).round(4)
199
200 print("按特征类型统计:")
201 print(feature_type_analysis.to_string())
202
203 # 子图3: 特征类型贡献饼图
204 plt.subplot(2, 2, 3)
205 type_contributions = coef_df.groupby('特征类型')['系数绝对值'].sum()
206 plt.pie(type_contributions.values, labels=type_contributions.index, autopct=
207 '%1.1f%%', startangle=90)
208 plt.title('不同特征类型的贡献占比')
209
210 # 8. 回归方程展示
211 intercept = ridge_model.intercept_
212 print(f"Y染色体浓度 = {intercept:.6f}")
213
214 # 显示重要特征的回归方程
215 important_features = coef_df.head(10)
216 for _, row in important_features.iterrows():
217     feature_name = row['特征名']
218     coef = row['标准化系数']
219     sign = "+" if coef >= 0 else "-"
220     print(f"          {sign} {coef:.6f} × {feature_name}_标准化")
221
222 # 9. 模型预测效果可视化
223 # 子图4: 预测值 vs 实际值散点图
224 plt.subplot(2, 2, 4)
225 plt.scatter(y_test, y_test_pred, alpha=0.6, color='blue', s=30)
226
227 # 绘制完美预测线
228 min_val = min(y_test.min(), y_test_pred.min())
229 max_val = max(y_test.max(), y_test_pred.max())
230 plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, label='完

```

```

美预测线')
230
231 plt.xlabel('实际值')
232 plt.ylabel('预测值')
233 plt.title(f'预测效果 ( $R^2 = \{test\_r2:.3f\}$ )')
234 plt.legend()
235 plt.grid(True, alpha=0.3)
236
237 # 添加 $R^2$ 和RMSE标注
238 plt.text(0.05, 0.95, f' $R^2 = \{test\_r2:.4f\}$ \nRMSE =  $\{test\_rmse:.6f\}$ ',
239         transform=plt.gca().transAxes, verticalalignment='top',
240         bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))
241
242 plt.tight_layout()
243 plt.savefig('Ridge回归详细分析.png', dpi=300, bbox_inches='tight', facecolor='
white')
244 plt.show()
245
246 # 10. 残差分析
247 residuals = y_test - y_test_pred
248
249 plt.figure(figsize=(15, 10))
250
251 # 残差图1: 残差vs预测值
252 plt.subplot(2, 3, 1)
253 plt.scatter(y_test_pred, residuals, alpha=0.6)
254 plt.axhline(y=0, color='red', linestyle='--')
255 plt.xlabel('预测值')
256 plt.ylabel('残差')
257 plt.title('残差 vs 预测值')
258 plt.grid(True, alpha=0.3)
259
260 # 残差图2: 残差直方图
261 plt.subplot(2, 3, 2)
262 plt.hist(residuals, bins=20, alpha=0.7, color='skyblue', edgecolor='black')
263 plt.xlabel('残差')
264 plt.ylabel('频数')
265 plt.title('残差分布直方图')
266 plt.axvline(residuals.mean(), color='red', linestyle='--', label=f'均值: {
residuals.mean():.6f}')
267 plt.legend()
268 plt.grid(True, alpha=0.3)
269
270 # 残差图3: QQ图
271 plt.subplot(2, 3, 3)
272 stats.probplot(residuals, dist="norm", plot=plt)
273 plt.title('残差正态性QQ图')

```

```

274 plt.grid(True, alpha=0.3)
275
276 # 残差图4：残差vs主要特征
277 if 'X染色体浓度' in X_test.columns:
278     plt.subplot(2, 3, 4)
279     plt.scatter(X_test['X染色体浓度'], residuals, alpha=0.6)
280     plt.axhline(y=0, color='red', linestyle='--')
281     plt.xlabel('X染色体浓度')
282     plt.ylabel('残差')
283     plt.title('残差 vs X染色体浓度')
284     plt.grid(True, alpha=0.3)
285
286 # 残差图5：残差vs另一主要特征
287 if '检测时怀孕天数' in X_test.columns:
288     plt.subplot(2, 3, 5)
289     plt.scatter(X_test['检测时怀孕天数'], residuals, alpha=0.6)
290     plt.axhline(y=0, color='red', linestyle='--')
291     plt.xlabel('检测时怀孕天数')
292     plt.ylabel('残差')
293     plt.title('残差 vs 检测时怀孕天数')
294     plt.grid(True, alpha=0.3)
295
296 # 残差图6：残差绝对值vs预测值
297 plt.subplot(2, 3, 6)
298 plt.scatter(y_test_pred, np.abs(residuals), alpha=0.6)
299 plt.xlabel('预测值')
300 plt.ylabel('残差绝对值')
301 plt.title('残差绝对值 vs 预测值')
302 plt.grid(True, alpha=0.3)
303
304 plt.tight_layout()
305 plt.savefig('Ridge回归残差分析.png', dpi=300, bbox_inches='tight', facecolor='
white')
306 plt.show()
307
308 # 残差统计
309 shapiro_stat, shapiro_p = stats.shapiro(residuals)
310 print(f"残差正态性检验 (Shapiro-Wilk): 统计量={shapiro_stat:.4f}, p值={
shapiro_p:.6f}")
311 print(f"残差均值: {residuals.mean():.6f}")
312 print(f"残差标准差: {residuals.std():.6f}")
313 print(f"残差范围: [{residuals.min():.6f}, {residuals.max():.6f}]")
314
315 # 11. 特征重要性详细分析
316 # 创建特征重要性热力图
317 plt.figure(figsize=(12, 10))
318

```

```

319 # 选择top 20特征绘制热力图
320 top_20_features = coef_df.head(20)
321
322 # 准备数据 - 创建系数矩阵用于热力图
323 coef_matrix = top_20_features[['标准化系数']].T
324 coef_matrix.columns = top_20_features['特征名']
325
326 # 绘制热力图
327 plt.subplot(2, 1, 1)
328 sns.heatmap(coef_matrix, annot=True, cmap='RdBu_r', center=0,
329             fmt='.3f', cbar_kws={'label': '标准化系数'})
330 plt.title('Top 20 特征的Ridge回归系数热力图')
331 plt.xlabel('')
332 plt.ylabel('')
333
334 # 特征分类条形图
335 plt.subplot(2, 1, 2)
336 feature_types = coef_df['特征类型'].value_counts()
337 colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A']
338 bars = plt.bar(feature_types.index, feature_types.values, color=colors[:len(
339 feature_types)])
340 plt.title('不同类型特征的数量分布')
341 plt.ylabel('特征数量')
342 plt.xticks(rotation=45)
343
344 # 添加数值标签
345 for bar, value in zip(bars, feature_types.values):
346     plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.1,
347             str(value), ha='center', va='bottom', fontweight='bold')
348
349 plt.tight_layout()
350 plt.savefig('Ridge回归特征重要性分析.png', dpi=300, bbox_inches='tight',
351 facecolor='white')
352 plt.show()
353
354 # 12. 模型解释总结
355 print("\n12. Ridge回归模型解释总结")
356 print("-"*60)
357
358 # 最重要的5个正向特征
359 positive_features = coef_df[coef_df['标准化系数'] > 0].head(5)
360
361 # 最重要的5个负向特征
362 negative_features = coef_df[coef_df['标准化系数'] < 0].head(5)
363
364 print("最重要的正向影响因子 (增加Y染色体浓度):")
365 for i, (_, row) in enumerate(positive_features.iterrows(), 1):
366     print(f" {i}. {row['特征名']}: 系数={row['标准化系数']:.4f} ({row['特征类

```

```

    型 ']]')")
364
365 print("\n最重要的负向影响因子 (减少Y染色体浓度):")
366 for i, (_, row) in enumerate(negative_features.iterrows(), 1):
367     print(f"    {i}. {row['特征名']}: 系数={row['标准化系数']:.4f} ({row['特征类型']})")
368
369 print(f"\n模型性能总结:")
370 print(f"    • 解释方差: {test_r2*100:.2f}%")
371 print(f"    • 平均绝对误差: {test_mae:.6f}")
372 print(f"    • 相对误差: {test_mae/y.mean()*100:.2f}%")
373 print(f"    • 正则化参数:  $\alpha$  = {optimal_alpha:.6f}")
374
375 print(f"\n关键发现:")
376 most_important = coef_df.iloc[0]
377 print(f"    • 最关键因子: {most_important['特征名']} (系数={most_important['标准化系数']:.4f})")
378 print(f"    • 特征工程效果: 添加了{len(interaction_features + polynomial_features)}个衍生特征")
379 print(f"    • 模型复杂度: 使用{len(feature_names)}个特征, 正则化防止过拟合")
380
381 # 13. 保存详细结果
382 # 保存系数分析
383 coef_df.to_csv("Ridge回归系数详细分析.csv", index=False, encoding='utf-8-sig')
384
385 # 保存预测结果
386 prediction_analysis = pd.DataFrame({
387     '样本序号': range(len(y_test)),
388     '实际值': y_test.values,
389     '预测值': y_test_pred,
390     '残差': residuals,
391     '绝对误差': np.abs(residuals),
392     '相对误差(%)': np.abs(residuals) / y_test.values * 100,
393     '误差等级': pd.cut(np.abs(residuals), bins=3, labels=['小误差', '中等误差', '大误差'])
394 })
395 prediction_analysis.to_csv("Ridge回归预测详细分析.csv", index=False, encoding='utf-8-sig')
396
397 # 保存特征相关性分析
398 corr_df.to_csv("特征相关性详细分析.csv", index=False, encoding='utf-8-sig')
399
400 print("已保存以下分析文件:")
401 print("    • Ridge回归系数详细分析.csv - 所有特征系数和重要性")
402 print("    • Ridge回归预测详细分析.csv - 每个样本的预测详情")
403 print("    • 特征相关性详细分析.csv - 特征与因变量的相关性")
404 print("    • Ridge回归详细分析.png - 主要分析图表")

```

```

405     print(" • Ridge回归残差分析.png - 残差诊断图表")
406     print(" • Ridge回归特征重要性分析.png - 特征重要性图表")
407
408     return {
409         'model': ridge_model,
410         'scaler': scaler,
411         'coefficients': coef_df,
412         'predictions': prediction_analysis,
413         'correlations': corr_df,
414         'optimal_alpha': optimal_alpha,
415         'performance_metrics': {
416             'train_r2': train_r2,
417             'test_r2': test_r2,
418             'rmse': test_rmse,
419             'mae': test_mae
420         },
421         'feature_names': feature_names.tolist()
422     }
423 if __name__ == "__main__":
424     df = pd.read_csv("男胎_预处理数据_完整版.csv")
425     results = ridge_regression_detailed_analysis(df)

```

问题2 数据预处理.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5  from datetime import datetime, timedelta
6  import warnings
7  warnings.filterwarnings('ignore')
8  #设置字体
9  plt.rcParams['font.sans-serif'] = ['SimHei']
10 plt.rcParams['axes.unicode_minus'] = False
11 def problem2_preprocessing(df_file):
12     print("问题2数据预处理：风险指标与Y染色体浓度达标时间")
13     #读取由第一问预处理得到的数据
14     df = pd.read_csv(df_file, encoding='utf-8-sig')
15     print("原始数据形状：{}".format(df.shape))
16     print("原始数据列名：{}".format(list(df.columns)))
17     #创建一个为处理后的数据副本
18     processed_df = df.copy()
19     print("\n1. 创建风险指标")
20
21     def calculate_risk_level(pregnancy_days):
22         if pd.isna(pregnancy_days):
23             return np.nan
24         if pregnancy_days <= 84: #12周=84天

```



```

25         return 0 #定义为低风险
26     elif pregnancy_days <= 189: #27周=189天
27         return 1 #定义为高风险
28     else:
29         return 2 #定义为极高风险
30
31 #应用风险等级计算
32 processed_df['风险指标'] = processed_df['检测时怀孕天数'].apply(
    calculate_risk_level)
33 #统计风险等级分布
34 risk_counts = processed_df['风险指标'].value_counts().sort_index()
35 print("风险等级分布:")
36 risk_labels = {0: '低风险(≤12周)', 1: '高风险(13-27周)', 2: '极高风险(≥28周)'}
37 for level, count in risk_counts.items():
38     if not pd.isna(level):
39         percentage = count / len(processed_df) * 100
40         print(" {}: {} 例 {:.2f}%".format(
41             risk_labels[int(level)], count, percentage))
42 #检查缺失值
43 risk_missing = processed_df['风险指标'].isnull().sum()
44 if risk_missing > 0:
45     print(" 缺失值: {} 例 {:.2f}%".format(
46         risk_missing, risk_missing / len(processed_df) * 100))
47 print("\n2. 计算Y染色体浓度达标时间")
48 #Y染色体浓度达标的阈值
49 TARGET_CONCENTRATION = 0.04
50 print("Y染色体浓度达标阈值: {:.2f}% ({:.4f})".format(
51     TARGET_CONCENTRATION * 100, TARGET_CONCENTRATION))
52 #统计当前数据达标情况
53 current_qualified = (processed_df['Y染色体浓度'] >= TARGET_CONCENTRATION).sum()
54 current_qualified_rate = current_qualified / len(processed_df) * 100
55 print("当前检测中达标样本: {} 例 {:.2f}%".format(
56     current_qualified, current_qualified_rate))
57 #按孕妇分组, 计算每个孕妇的最早达标时间
58 print("\n按孕妇计算最早达标时间...")
59 #创建一个字典, 记录达标时间
60 earliest_qualified_time = {}
61 #根据孕妇代码, 分组
62 grouped = processed_df.groupby('孕妇代码')
63 total_women = len(grouped)
64 qualified_women = 0
65 print("总共孕妇数量: {}".format(total_women))
66 for woman_code, group in grouped:
67     #根据检测时怀孕天数排序
68     group_sorted = group.sort_values('检测时怀孕天数')
69     #找出第一次达标的时间
70     qualified_records = group_sorted[

```

```

71         group_sorted['Y染色体浓度'] >= TARGET_CONCENTRATION
72     ]
73     if len(qualified_records) > 0:
74         #得到最早达标时间
75         earliest_time = qualified_records['检测时怀孕天数'].iloc[0]
76         earliest_qualified_time[woman_code] = earliest_time
77         qualified_women += 1
78     else:
79         #如果未达标，则设为NaN
80         earliest_qualified_time[woman_code] = np.nan
81     print("达标孕妇数量: {} ({:.2f}%)".format(
82         qualified_women, qualified_women / total_women * 100))
83     #将得出的最早达标时间映射回原数据
84     processed_df['最早达标时间_天数'] = processed_df['孕妇代码'].map(
85         earliest_qualified_time)
86     #统计所有数据的最早达标时间
87     qualified_times = processed_df['最早达标时间_天数'].dropna()
88     if len(qualified_times) > 0:
89         print("\n最早达标时间统计 (天数):")
90         print("    均值: {:.1f} 天".format(qualified_times.mean()))
91         print("    中位数: {:.1f} 天".format(qualified_times.median()))
92         print("    标准差: {:.1f} 天".format(qualified_times.std()))
93         print("    范围: {:.1f} - {:.1f} 天".format(
94             qualified_times.min(), qualified_times.max()))
95     #根据划分的风险等级统计达标时间
96     print("\n按风险等级统计最早达标时间:")
97     for risk_level in [0, 1, 2]:
98         risk_data = processed_df[
99             (processed_df['风险指标'] == risk_level) &
100             (processed_df['最早达标时间_天数'].notna())
101         ][['最早达标时间_天数']]
102         if len(risk_data) > 0:
103             print("    {}: 均值 {:.1f} 天, n={}".format(
104                 risk_labels[risk_level], risk_data.mean(), len(risk_data)))
105     print("\n3. 创建达标状态指标")
106     #检验当前检测是否达标
107     processed_df['当前检测达标'] = (
108         processed_df['Y染色体浓度'] >= TARGET_CONCENTRATION
109     ).astype(int)
110     #得到该孕妇是否曾经达标过
111     processed_df['曾经达标'] = processed_df['最早达标时间_天数'].notna().astype(int)
112     #综合统计达标情况
113     current_qualified_count = processed_df['当前检测达标'].sum()
114     ever_qualified_count = processed_df['曾经达标'].sum()
115     print("当前检测达标: {} 例 ({:.2f}%)".format(
116         current_qualified_count, current_qualified_count / len(processed_df) * 100))

```

```

)
116 print("曾经达标: {} 例 ({:.2f}%)".format(
117     ever_qualified_count, ever_qualified_count / len(processed_df) * 100))
118 print("\n4. 按风险等级分析达标情况")
119 #根据按风险等级来划分达标情况
120 risk_analysis = processed_df.groupby('风险指标').agg({
121     '孕妇代码': 'count',
122     '曾经达标': 'mean',
123     '最早达标时间_天数': ['mean', 'median', 'std'],
124     'Y染色体浓度': 'mean',
125     '孕妇BMI': 'mean'
126 }).round(3)
127 print("风险等级达标分析:")
128 for risk_level in [0, 1, 2]:
129     if risk_level in risk_analysis.index:
130         data = risk_analysis.loc[risk_level]
131         print(" {}: 样本数={}, 达标率={:.1%}, 平均达标时间={:.1f}天".format(
132             risk_labels[risk_level],
133             int(data['孕妇代码', 'count']),
134             data[['曾经达标', 'mean']],
135             data[['最早达标时间_天数', 'mean']] if not pd.isna(data[['最早达标
136             时间_天数', 'mean']]) else 0
137         ))
138 print("\n5. 数据质量检查")
139 #检查新变量是否存在缺失值
140 new_vars = ['风险指标', '最早达标时间_天数', '当前检测达标', '曾经达标']
141 print("新变量缺失值统计:")
142 for var in new_vars:
143     if var in processed_df.columns:
144         missing_count = processed_df[var].isnull().sum()
145         missing_rate = missing_count / len(processed_df)
146         print(" {}: {} ({:.2f}%)".format(var, missing_count, missing_rate *
147         100))
148 #检查数据是否完整
149 complete_cases = processed_df[['风险指标', 'Y染色体浓度', '孕妇BMI']].dropna()
150 print("\n完整数据样本数: {} ({:.2f}%)".format(
151     len(complete_cases), len(complete_cases) / len(processed_df) * 100))
152 print("\n6. 保存处理后的数据")
153 #保存并输出完整的数据
154 output_file = "问题2_预处理数据_完整版.csv"
155 processed_df.to_csv(output_file, index=False, encoding='utf-8-sig')
156 #创建新的精简数据集
157 modeling_vars = [
158     #表格内基本信息
159     '序号', '孕妇代码', '年龄', '身高', '体重', '孕妇BMI',
160     #与时间相关的信息
161     '检测时怀孕天数', '末次月经距检测日天数',

```

```

160     #与检测结果相关
161     'Y染色体浓度', 'Y染色体的Z值', 'X染色体的Z值',
162     '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
163     'GC含量',
164     #分类变量
165     'IVF妊娠_编码', '怀孕次数_编码', '胎儿健康_编码',
166     'T13_异常', 'T18_异常', 'T21_异常',
167     #新增的变量
168     '风险指标', '最早达标时间_天数', '当前检测达标', '曾经达标'
169 ]
170 #只保留还存在的列
171 available_modeling_vars = [var for var in modeling_vars if var in processed_df.
columns]
172 modeling_df = processed_df[available_modeling_vars].copy()
173 modeling_file = "问题2_建模数据.csv"
174 modeling_df.to_csv(modeling_file, index=False, encoding='utf-8-sig')
175 print("已保存:")
176 print("    {}: 完整预处理数据 ({} 行 × {} 列)".format(
177     output_file, processed_df.shape[0], processed_df.shape[1]))
178 print("    {}: 建模用数据 ({} 行 × {} 列)".format(
179     modeling_file, modeling_df.shape[0], modeling_df.shape[1]))
180 print("\n7. 数据摘要报告")
181 summary_stats = {
182     '总样本数': len(processed_df),
183     '总孕妇数': len(processed_df['孕妇代码'].unique()),
184     '当前达标检测数': current_qualified_count,
185     '当前达标率(%)': round(current_qualified_count / len(processed_df) * 100,
2),
186     '曾经达标孕妇数': qualified_women,
187     '曾经达标率(%)': round(qualified_women / total_women * 100, 2),
188     '低风险检测数': risk_counts.get(0, 0),
189     '高风险检测数': risk_counts.get(1, 0),
190     '极高风险检测数': risk_counts.get(2, 0),
191     '平均最早达标时间(天)': round(qualified_times.mean(), 2) if len(
qualified_times) > 0 else np.nan
192 }
193 summary_df = pd.DataFrame(list(summary_stats.items()), columns=['指标', '数值'
])
194 summary_df.to_csv("问题2_数据摘要.csv", index=False, encoding='utf-8-sig')
195 print("    问题2_数据摘要.csv: 数据摘要统计")
196 print("    问题2数据预处理完成!")
197 print("新增关键变量:")
198 print("    1. 风险指标: 基于检测时间的风险等级 (0=低, 1=高, 2=极高)")
199 print("    2. 最早达标时间_天数: Y染色体浓度≥4%的最早时间(天)")
200 print("    3. 达标状态: 当前检测达标 & 曾经达标指标")
201 return processed_df, summary_stats
202 if __name__ == "__main__":

```

```

203 #使用之前第一问预处理后的数据
204 input_file = "男胎_预处理数据_精简版.csv"
205 #用于进行问题2预处理
206 processed_data, summary = problem2_preprocessing(input_file)
207 print("\n处理完成！ 可以进行问题2的建模分析。")

```

问题 2.py

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.cluster import DBSCAN, AgglomerativeClustering, SpectralClustering
4 from sklearn.mixture import GaussianMixture
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import silhouette_score, calinski_harabasz_score
7 from scipy.optimize import differential_evolution, minimize_scalar
8 from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
9 from scipy import stats
10 import matplotlib.pyplot as plt
11 import matplotlib.patches as patches
12 import seaborn as sns
13 from matplotlib.font_manager import FontProperties
14 import warnings
15 warnings.filterwarnings('ignore')
16 #[13]AI构建基础智能算法流程框架，我们根据实际问题情况修改和完善
17 #设置字体，便于后续利用
18 plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
19 plt.rcParams['axes.unicode_minus'] = False
20 class AdvancedBMIGroupingMethodsFixed:
21     #定义函数，修正重叠区间
22     def __init__(self, data_file):
23         self.df = pd.read_csv(data_file, encoding='utf-8-sig')
24         self.modeling_df = self.df.dropna(subset=['孕妇BMI', '最早达标时间_天数', '
风险指标']).copy()
25         self.scaler = StandardScaler()
26     def _fix_overlapping_intervals(self, bmi_groups, method_name="聚类"):
27         if not bmi_groups:
28             return bmi_groups
29         #检查是否存在原始重叠
30         original_overlaps = self._check_overlaps(bmi_groups, "原始")
31         #没有则不修正
32         if original_overlaps == 0:
33             print(" 无重叠，无需修正")
34             return bmi_groups
35         #按照下界排序
36         sorted_groups = sorted(bmi_groups, key=lambda x: x[0])
37         #使用中点分割法进行重叠修正
38         fixed_groups = [sorted_groups[0]]
39         for i in range(1, len(sorted_groups)):

```

```

40     prev_lower, prev_upper = fixed_groups[-1]
41     curr_lower, curr_upper = sorted_groups[i]
42     if curr_lower <= prev_upper:
43         #如果有重叠，计算新边界
44         new_boundary = (prev_upper + curr_lower) / 2
45         #更新前一个区间的上界
46         fixed_groups[-1] = (prev_lower, new_boundary)
47         #并且添加当前区间，下界为新边界
48         fixed_groups.append((new_boundary, curr_upper))
49     else:
50         #如果无重叠，则直接添加为边界
51         fixed_groups.append((curr_lower, curr_upper))
52     #验证修正后的结果
53     fixed_overlaps = self._check_overlaps(fixed_groups, "修正后")
54     if fixed_overlaps == 0:
55         print(" 重叠修正成功")
56     else:
57         print(f" 仍有 {fixed_overlaps} 个重叠，使用备用方法")
58         #另一个方法：用于强制无重叠数据分割
59         fixed_groups = self._force_non_overlapping(bmi_groups)
60         self._check_overlaps(fixed_groups, "强制修正后")
61     return fixed_groups
62
63 def _check_overlaps(self, bmi_groups, label=""):
64     overlap_count = 0
65     for i in range(len(bmi_groups)):
66         for j in range(i + 1, len(bmi_groups)):
67             lower1, upper1 = bmi_groups[i]
68             lower2, upper2 = bmi_groups[j]
69             #检查是否重叠
70             if (lower1 < upper2 and upper1 > lower2):
71                 overlap_count += 1
72                 if label == "原始":
73                     print(f" 重叠{overlap_count}: [{lower1:.1f}, {upper1:.1f}
74 ]] 与 [{lower2:.1f}, {upper2:.1f}]")
75             if overlap_count == 0 and label:
76                 print(f" {label}: 无重叠")
77             elif label and overlap_count > 0:
78                 print(f" {label}: {overlap_count}个重叠")
79     return overlap_count
80
81 def _force_non_overlapping(self, bmi_groups):
82     #获取所有的边界点
83     all_bounds = []
84     for lower, upper in bmi_groups:
85         all_bounds.extend([lower, upper])
86     bmi_min = min(all_bounds)

```

```

86     bmi_max = max(all_bounds)
87     n_groups = len(bmi_groups)
88     #将所有数据等间距分割
89     boundaries = np.linspace(bmi_min, bmi_max, n_groups + 1)
90     fixed_groups = []
91     for i in range(n_groups):
92         fixed_groups.append((boundaries[i], boundaries[i + 1]))
93     return fixed_groups
94
95     def method1_gaussian_mixture_clustering(self, n_components_range=(2, 6)):
96         print("方法1: 高斯混合模型(GMM)聚类 - 修正版")
97         #用BMI+最早达标时间进行聚类
98         features = self.modeling_df[['孕妇BMI', '最早达标时间_天数']].values
99         features_scaled = self.scaler.fit_transform(features)
100         best_score = -np.inf
101         best_model = None
102         best_n = 0
103         results = []
104         for n in range(n_components_range[0], n_components_range[1]+1):
105             gmm = GaussianMixture(n_components=n, random_state=42,
106                                   covariance_type='full')
107             gmm.fit(features_scaled)
108             #计算必要的评估指标
109             labels = gmm.predict(features_scaled)
110             aic = gmm.aic(features_scaled)
111             #确保聚类数量大于1
112             bic = gmm.bic(features_scaled)
113             if len(set(labels)) > 1:
114                 silhouette = silhouette_score(features_scaled, labels)
115                 calinski = calinski_harabasz_score(features_scaled, labels)
116             else:
117                 silhouette = -1
118                 calinski = 0
119             results.append({
120                 '聚类数': n,
121                 'AIC': aic,
122                 'BIC': bic,
123                 'Silhouette得分': silhouette,
124                 'Calinski-Harabasz指数': calinski
125             })
126             #选择BIC指标最小的模型
127             if -bic > best_score:
128                 best_score = -bic
129                 best_model = gmm
130                 best_n = n
131         results_df = pd.DataFrame(results)
132         print("GMM聚类评估结果:")

```

```

133     print(results_df.round(3))
134     #综合所有模型，并使用最佳模型进行分组
135     best_labels = best_model.predict(features_scaled)
136     #区间转化，转换为BMI区间
137     bmi_groups_original = self._labels_to_bmi_intervals(best_labels)
138     #再次修正重叠
139     bmi_groups_fixed = self._fix_overlapping_intervals(bmi_groups_original, "
GMM")
140     print(f"\n最佳聚类数: {best_n}")
141     print("GMM聚类结果 (修正后):")
142     group_stats = self._evaluate_groups(bmi_groups_fixed, "GMM_修正版")
143     print(group_stats.round(3))
144     return bmi_groups_fixed, group_stats
145
146 def method2_hierarchical_clustering(self):
147     print("方法2: 层次聚类 - 修正版")
148     #按照孕妇聚合数据
149     women_data = self.modeling_df.groupby('孕妇代码').agg({
150         '孕妇BMI': 'first',
151         '最早达标时间_天数': 'first',
152         '风险指标': 'first'
153     }).reset_index()
154     features = women_data[['孕妇BMI', '最早达标时间_天数']].values
155     features_scaled = self.scaler.fit_transform(features)
156     #根据所聚合的数据，计算链接矩阵
157     linkage_matrix = linkage(features_scaled, method='ward')
158     #尝试不同的聚类数，选择效果最好的聚类数
159     best_score = -1
160     best_n = 0
161     results = []
162     for n_clusters in range(2, 7):
163         labels = fcluster(linkage_matrix, n_clusters, criterion='maxclust') - 1
164         if len(set(labels)) > 1:
165             silhouette = silhouette_score(features_scaled, labels)
166             calinski = calinski_harabasz_score(features_scaled, labels)
167             results.append({
168                 '聚类数': n_clusters,
169                 'Silhouette得分': silhouette,
170                 'Calinski-Harabasz指数': calinski
171             })
172             if silhouette > best_score:
173                 best_score = silhouette
174                 best_n = n_clusters
175     results_df = pd.DataFrame(results)
176     print("层次聚类评估结果:")
177     print(results_df.round(3))
178     #综合所有聚类数效果，使用最佳的一个聚类数

```



```

179     best_labels = fcluster(linkage_matrix, best_n, criterion='maxclust') - 1
180     women_data['cluster'] = best_labels
181     #将结果映射回原数据
182     cluster_map = dict(zip(women_data['孕妇代码'], women_data['cluster']))
183     self.modeling_df['cluster'] = self.modeling_df['孕妇代码'].map(cluster_map)
184     bmi_groups_original = self._labels_to_bmi_intervals(self.modeling_df['
cluster'].values)
185     #对数据进行修正重叠
186     bmi_groups_fixed = self._fix_overlapping_intervals(bmi_groups_original, "层
次聚类")
187     print(f"\n最佳聚类数: {best_n} (Silhouette: {best_score:.3f})")
188     print("层次聚类结果 (修正后):")
189     group_stats = self._evaluate_groups(bmi_groups_fixed, "层次聚类_修正版")
190     print(group_stats.round(3))
191     return bmi_groups_fixed, group_stats
192
193 def method3_dbscan_clustering(self):
194     print("方法3: DBSCAN密度聚类 - 修正版")
195     features = self.modeling_df[['孕妇BMI', '最早达标时间_天数']].values
196     features_scaled = self.scaler.fit_transform(features)
197     #遍历网格以搜索最佳参数
198     eps_range = np.arange(0.3, 1.5, 0.1)
199     min_samples_range = [5, 10, 15, 20]
200     best_score = -1
201     best_params = None
202     results = []
203     for eps in eps_range:
204         for min_samples in min_samples_range:
205             dbscan = DBSCAN(eps=eps, min_samples=min_samples)
206             labels = dbscan.fit_predict(features_scaled)
207             n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
208             n_noise = list(labels).count(-1)
209             #确保至少要有2个聚类
210             if n_clusters >= 2:
211                 #只对非噪声点计算silhouette
212                 mask = labels != -1
213                 if np.sum(mask) > 1:
214                     silhouette = silhouette_score(features_scaled[mask], labels
[mask])
215                     results.append({
216                         'eps': eps,
217                         'min_samples': min_samples,
218                         '聚类数': n_clusters,
219                         '噪声点数': n_noise,
220                         'Silhouette得分': silhouette
221                     })
222                     if silhouette > best_score:

```

```

223         best_score = silhouette
224         best_params = (eps, min_samples)
225     if not results:
226         print("DBSCAN无法找到合适的聚类参数，使用默认分组")
227         return None, None
228     results_df = pd.DataFrame(results)
229     print("DBSCAN参数搜索结果(前10个):")
230     print(results_df.nlargest(10, 'Silhouette得分').round(3))
231     #综合所有聚类，使用最佳参数
232     best_eps, best_min_samples = best_params
233     dbscan = DBSCAN(eps=best_eps, min_samples=best_min_samples)
234     labels = dbscan.fit_predict(features_scaled)
235     #对噪声点进行处理
236     if -1 in labels:
237         #将所有噪声点随机分配到离其最近的聚类
238         from sklearn.neighbors import NearestNeighbors
239         noise_mask = labels == -1
240         if np.sum(noise_mask) > 0:
241             non_noise_mask = ~noise_mask
242             nn = NearestNeighbors(n_neighbors=1)
243             nn.fit(features_scaled[non_noise_mask])
244             distances, indices = nn.kneighbors(features_scaled[noise_mask])
245             noise_labels = labels[non_noise_mask][indices.flatten()]
246             labels[noise_mask] = noise_labels
247     bmi_groups_original = self._labels_to_bmi_intervals(labels)
248     #对重叠数据进行修正
249     bmi_groups_fixed = self._fix_overlapping_intervals(bmi_groups_original, "
DBSCAN")
250     print(f"\n最佳参数: eps={best_eps}, min_samples={best_min_samples}")
251     print("DBSCAN聚类结果 (修正后):")
252     group_stats = self._evaluate_groups(bmi_groups_fixed, "DBSCAN_修正版")
253     print(group_stats.round(3))
254     return bmi_groups_fixed, group_stats
255
256 def method4_optimal_binning(self):
257     print("方法4: 风险最优化分箱 - 修正版")
258     women_data = self.modeling_df.groupby('孕妇代码').agg({
259         '孕妇BMI': 'first',
260         '最早达标时间_天数': 'first',
261         '风险指标': 'first'
262     }).reset_index().sort_values('孕妇BMI')
263
264 def calculate_total_risk(cut_points):
265     if len(cut_points) == 0:
266         return np.inf
267     #对必要数据添加边界点
268     all_cuts = [women_data['孕妇BMI'].min()] + sorted(cut_points) + [

```

```

women_data['孕妇BMI'].max())
269     total_risk = 0
270     for i in range(len(all_cuts) - 1):
271         group_mask = (women_data['孕妇BMI'] >= all_cuts[i]) & (women_data['
孕妇BMI'] < all_cuts[i+1])
272         if i == len(all_cuts) - 2: # 最后一组包含右边界
273             group_mask = (women_data['孕妇BMI'] >= all_cuts[i]) & (
women_data['孕妇BMI'] <= all_cuts[i+1])
274             group_data = women_data[group_mask]
275             if len(group_data) == 0:
276                 continue
277             #计算该组数据的的风险指标
278             risk_weights = {0: 1, 1: 2, 2: 4}
279             group_risk = group_data['风险指标'].map(risk_weights).mean()
280             total_risk += group_risk * len(group_data)
281         return total_risk / len(women_data)
282
283     #为寻找最优分割点，使用差分进化算法
284     bmi_range = women_data['孕妇BMI'].max() - women_data['孕妇BMI'].min()
285     bmi_min = women_data['孕妇BMI'].min()
286     results = []
287     for n_groups in range(2, 6):
288         n_cuts = n_groups - 1
289         #定义数据搜索的边界
290         bounds = [(bmi_min + (i+1) * bmi_range / (n_cuts+1) - bmi_range/4,
291                    bmi_min + (i+1) * bmi_range / (n_cuts+1) + bmi_range/4)
292                   for i in range(n_cuts)]
293         def objective(x):
294             return calculate_total_risk(x)
295         try:
296             result = differential_evolution(objective, bounds, seed=42, maxiter
=100)
297             optimal_cuts = sorted(result.x)
298             optimal_risk = result.fun
299             results.append({
300                 '分组数': n_groups,
301                 '最优风险': optimal_risk,
302                 '分割点': optimal_cuts
303             })
304         except:
305             continue
306     if not results:
307         print("风险优化失败，使用等频分箱")
308         return None, None
309     #得出结果后，选择风险最小的分组
310     best_result = min(results, key=lambda x: x['最优风险'])
311     optimal_cuts = best_result['分割点']

```

```

312     # 根据结果构建分组区间, 由于方法本身是无重叠的, 无需再修正重叠区间
313     all_cuts = [women_data['孕妇BMI'].min()] + optimal_cuts + [women_data['孕妇
BMI'].max()]
314     bmi_groups = [(all_cuts[i], all_cuts[i+1]) for i in range(len(all_cuts)-1)]
315     #验证其无重叠区间
316     overlap_count = self._check_overlaps(bmi_groups, "风险优化")
317     print(f"最优分组数: {best_result['分组数']}")
318     print(f"最优风险得分: {best_result['最优风险']:.3f}")
319     print("风险最优化分组结果:")
320     group_stats = self._evaluate_groups(bmi_groups, "风险优化_修正版")
321     print(group_stats.round(3))
322     return bmi_groups, group_stats
323
324     def method5_decision_tree_binning(self):
325         print("方法5: 决策树分箱 - 修正版")
326         from sklearn.tree import DecisionTreeRegressor
327         women_data = self.modeling_df.groupby('孕妇代码').agg({
328             '孕妇BMI': 'first',
329             '最早达标时间_天数': 'first',
330             '风险指标': 'first'
331         }).reset_index()
332         #使用风险指标来作为目标因变量
333         X = women_data[['孕妇BMI']].values
334         y = women_data['风险指标'].values
335         best_score = np.inf
336         best_tree = None
337         results = []
338         for max_leaf_nodes in range(3, 8): #表示其有2-7个分组
339             tree = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes,
340                                         random_state=42,
341                                         min_samples_leaf=20)
342             tree.fit(X, y)
343             #获取分割点
344             tree_splits = []
345             def extract_splits(node_id, threshold_path):
346                 if tree.tree_.children_left[node_id] == tree.tree_.children_right[
node_id]:
347                     return #定义叶子节点
348                 threshold = tree.tree_.threshold[node_id]
349                 tree_splits.append(threshold)
350                 extract_splits(tree.tree_.children_left[node_id], threshold_path +
[threshold])
351                 extract_splits(tree.tree_.children_right[node_id], threshold_path +
[threshold])
352             extract_splits(0, [])
353             if len(tree_splits) > 0:
354                 #计算该树的风险得分

```

```

355         predictions = tree.predict(X)
356         mse = np.mean((y - predictions) ** 2)
357         results.append({
358             '叶子节点数': max_leaf_nodes,
359             'MSE': mse,
360             '分割点数': len(set(tree_splits)),
361             '分割点': sorted(set(tree_splits))
362         })
363         if mse < best_score:
364             best_score = mse
365             best_tree = tree
366     if not results:
367         print("决策树分箱失败")
368         return None, None
369     results_df = pd.DataFrame(results)
370     print("决策树分箱评估:")
371     print(results_df.round(3))
372     # 综合所有分割点, 使用最佳树的分割点
373     best_result = min(results, key=lambda x: x['MSE'])
374     splits = best_result['分割点']
375     # 构建分组区间, 由于此方法本身是无重叠的, 故不必再修正重叠区间
376     all_cuts = [women_data['孕妇BMI'].min()] + splits + [women_data['孕妇BMI'].
377 max()]
378     all_cuts = sorted(set(all_cuts))
379     bmi_groups = [(all_cuts[i], all_cuts[i+1]) for i in range(len(all_cuts)-1)]
380     # 验证是否有重叠区间
381     overlap_count = self._check_overlaps(bmi_groups, "决策树")
382     print(f"\n最佳叶子节点数: {best_result['叶子节点数']}")
383     print("决策树分组结果:")
384     group_stats = self._evaluate_groups(bmi_groups, "决策树_修正版")
385     print(group_stats.round(3))
386     return bmi_groups, group_stats
387
388 def optimize_detection_timepoints(self, bmi_groups, method_name="未知方法"):
389     print(f"\n{method_name} - 检测时点优化")
390     if not bmi_groups:
391         print("错误: BMI分组为空, 无法优化时点")
392         return None
393     optimal_timepoints = []
394     for i, (bmi_min, bmi_max) in enumerate(bmi_groups):
395         # 根据表格中BMI区间来筛选数据
396         if i == len(bmi_groups) - 1: # 对最后一个区间, 使用闭区间
397             group_data = self.modeling_df[
398                 (self.modeling_df['孕妇BMI'] >= bmi_min) & (self.modeling_df['
399 孕妇BMI'] <= bmi_max)
400             ]
401         else: # 而对于其他区间, 均使用左闭右开

```

```

400         group_data = self.modeling_df[
401             (self.modeling_df['孕妇BMI'] >= bmi_min) & (self.modeling_df['
孕妇BMI'] < bmi_max)
402         ]
403         if group_data.empty:
404             print(f" 警告：组别 {i+1} (BMI [{bmi_min:.2f}, {bmi_max:.2f}]) 内
没有样本数据")
405             continue
406         #以孕妇为主要因素，对数据进行聚合
407         women_stats = group_data.groupby('孕妇代码').agg({
408             '最早达标时间_天数': 'first',
409             '风险指标': 'first'
410         }).reset_index()
411         print(f"\n 组别 {i+1}: BMI [{bmi_min:.2f}, {bmi_max:.2f}] (孕妇数 n={
len(women_stats)})")
412
413         #定义目标函数：平衡的成功率与时间风险
414         def objective(t):
415             success_rate = (women_stats['最早达标时间_天数'] <= t).mean()
416             #时间风险，即越早检测风险越低，越晚检测风险越高
417             if t <= 12 * 7:          #12周内，风险较低
418                 time_risk = 1.0
419             elif t <= 20 * 7:        #12-20周，风险适中
420                 time_risk = 1.5
421             elif t <= 27 * 7:        #20-27周，风险较高
422                 time_risk = 2.5
423             else:                    #27周后，风险极高
424                 time_risk = 4.0
425             #若检测失败，则检测失败的代价为
426             failure_risk = (1 - success_rate) * 3.0
427             return time_risk + failure_risk
428         #使用标量最小化来求解改模型的最优时点
429         try:
430             result = minimize_scalar(objective, bounds=(10*7, 30*7), method='
bounded')
431             optimal_time = result.x
432             #评估该时点性能
433             success_rate = (women_stats['最早达标时间_天数'] <= optimal_time).
mean()
434             #对风险等级进行评估
435             if optimal_time <= 12*7:
436                 risk_level = '低风险'
437             elif optimal_time <= 20*7:
438                 risk_level = '中风险'
439             elif optimal_time <= 27*7:
440                 risk_level = '较高风险'
441             else:

```

```

442         risk_level = '高风险'
443         #计算平均达标时间，可用于后续参考
444         avg_detection_time = women_stats['最早达标时间_天数'].mean()
445         optimal_timepoints.append({
446             '组别': f"组{i+1}",
447             'BMI区间': f"[{bmi_min:.2f}, {bmi_max:.2f}]",
448             '孕妇数': len(women_stats),
449             '最佳时点_天数': optimal_time,
450             '最佳时点_周数': optimal_time / 7,
451             '预期成功率': success_rate,
452             '风险等级': risk_level,
453             '平均达标时间_天数': avg_detection_time,
454             '平均达标时间_周数': avg_detection_time / 7
455         })
456         print(f"        最佳检测时点: {optimal_time/7:.1f}周 ({optimal_time:.0f
}天)")
457         print(f"        预期成功率: {success_rate:.2%}")
458         print(f"        风险等级: {risk_level}")
459         except Exception as e:
460             print(f"        优化失败: {e}")
461             continue
462         if optimal_timepoints:
463             optimal_df = pd.DataFrame(optimal_timepoints)
464             print(f"\n{method_name} - 最优时点汇总:")
465             display_cols = ['组别', 'BMI区间', '孕妇数', '最佳时点_周数', '预期成功
率', '风险等级']
466             print(optimal_df[display_cols].round(2))
467             return optimal_df
468         else:
469             print("所有组别的时点优化均失败")
470             return None
471
472     def sensitivity_analysis(self, optimal_timepoints, method_name="未知方法"):
473         if optimal_timepoints is None or optimal_timepoints.empty:
474             print("错误: 需要先进行时点优化")
475             return None
476         print(f"\n{method_name} - 时点敏感性分析")
477         error_scenarios = [-7, -3, 0, 3, 7] #设置±7天误差，增加结果的准确性
478         sensitivity_results = []
479         for _, group_info in optimal_timepoints.iterrows():
480             bmi_range_str = group_info['BMI区间']
481             bmi_min, bmi_max = [float(x) for x in bmi_range_str.strip('[]').split('
, ')]
482             #获取数据
483             if group_info['组别'] == f"组{len(optimal_timepoints)}":
484                 group_data = self.modeling_df[
485                     (self.modeling_df['孕妇BMI'] >= bmi_min) & (self.modeling_df['

```

```

    孕妇BMI'] <= bmi_max)
486         ]
487     else:
488         group_data = self.modeling_df[
489             (self.modeling_df['孕妇BMI'] >= bmi_min) & (self.modeling_df['
孕妇BMI'] < bmi_max)
490         ]
491     if group_data.empty:
492         continue
493     women_stats = group_data.groupby('孕妇代码').agg({
494         '最早达标时间_天数': 'first'
495     }).reset_index()
496     optimal_time = group_info['最佳时点_天数']
497     for error in error_scenarios:
498         adjusted_time = optimal_time + error
499         success_rate = (women_stats['最早达标时间_天数'] <= adjusted_time).
mean()
500         sensitivity_results.append({
501             '组别': group_info['组别'],
502             '误差天数': error,
503             '调整后时点_周数': adjusted_time / 7,
504             '预期成功率': success_rate
505         })
506     sensitivity_df = pd.DataFrame(sensitivity_results)
507     #构建一个透视表，该透视表用于展示敏感性
508     pivot_df = sensitivity_df.pivot_table(
509         index='组别', columns='误差天数', values='预期成功率'
510     )
511     print("不同误差下的预期成功率变化:")
512     print(pivot_df.round(3))
513     #计算数据的敏感性等级
514     print("\n各组敏感性评估:")
515     for group_name in pivot_df.index:
516         success_rates = pivot_df.loc[group_name]
517         deviation = success_rates.max() - success_rates.min()
518         if deviation < 0.1:
519             level = "低敏感"
520         elif deviation < 0.2:
521             level = "中敏感"
522         else:
523             level = "高敏感"
524         print(f" {group_name}: 成功率波动范围 {deviation:.2%} - {level}")
525     return sensitivity_df, pivot_df
526
527 def visualize_groups_and_timepoints(self, bmi_groups, optimal_timepoints,
method_name="未知方法"):
528     if not bmi_groups or optimal_timepoints is None:

```



```

529         print("缺少可视化数据")
530         return
531     print(f"\n生成{method_name}可视化图表...")
532     # 创建一个新的图表
533     fig = plt.figure(figsize=(16, 12))
534     # 1. 将BMI的分布和分组可视化
535     ax1 = plt.subplot(2, 3, 1)
536     bmi_values = self.modeling_df['孕妇BMI'].values
537     plt.hist(bmi_values, bins=50, alpha=0.7, color='lightblue', edgecolor='
black')
538     # 对直方图进行标记, 标记其分组边界
539     colors = ['red', 'orange', 'green', 'purple', 'brown', 'pink']
540     for i, (bmi_min, bmi_max) in enumerate(bmi_groups):
541         color = colors[i % len(colors)]
542         plt.axvline(bmi_min, color=color, linestyle='--', alpha=0.8, linewidth
=2)
543         if i == len(bmi_groups) - 1: # 最后一个区间显示右边界
544             plt.axvline(bmi_max, color=color, linestyle='--', alpha=0.8,
linewidth=2)
545         # 在图上添加组别标签
546         mid_point = (bmi_min + bmi_max) / 2
547         plt.text(mid_point, plt.ylim()[1] * 0.8, f'组{i+1}',
548                 ha='center', va='center', fontsize=10, color=color, fontweight=
'bold')
549     plt.xlabel('孕妇BMI')
550     plt.ylabel('频次')
551     plt.title(f'{method_name} - BMI分布与分组')
552     plt.grid(True, alpha=0.3)
553     # 2. 输出各组最佳检测时点对比
554     ax2 = plt.subplot(2, 3, 2)
555     groups = optimal_timepoints['组别']
556     timepoints_weeks = optimal_timepoints['最佳时点_周数']
557     success_rates = optimal_timepoints['预期成功率']
558     bars = plt.bar(groups, timepoints_weeks, alpha=0.7, color='lightgreen')
559     # 在已生成的柱状图上标注成功率
560     for i, (bar, rate) in enumerate(zip(bars, success_rates)):
561         height = bar.get_height()
562         plt.text(bar.get_x() + bar.get_width()/2., height + 0.5,
563                 f'{rate:.1%}', ha='center', va='bottom', fontweight='bold')
564     plt.xlabel('BMI组别')
565     plt.ylabel('最佳检测时点 (周)')
566     plt.title('各组最佳检测时点')
567     plt.grid(True, alpha=0.3)
568     # 3. 绘制成功率-时点散点图
569     ax3 = plt.subplot(2, 3, 3)
570     plt.scatter(timepoints_weeks, success_rates, s=100, alpha=0.7, c=range(len(
groups)), cmap='viridis')

```

```

571     for i, (x, y, group) in enumerate(zip(timepoints_weeks, success_rates,
572     groups)):
573         plt.annotate(group, (x, y), xytext=(5, 5), textcoords='offset points',
574         fontsize=9)
575         plt.xlabel('最佳检测时点 (周)')
576         plt.ylabel('预期成功率')
577         plt.title('时点-成功率关系')
578         plt.grid(True, alpha=0.3)
579         #4.绘制各组风险分布，以明显的查看
580         ax4 = plt.subplot(2, 3, 4)
581         risk_levels = optimal_timepoints['风险等级'].value_counts()
582         colors_risk = ['green', 'yellow', 'orange', 'red']
583         plt.pie(risk_levels.values, labels=risk_levels.index, autopct='%1.1f%%',
584         colors=colors_risk[:len(risk_levels)])
585         plt.title('各组风险等级分布')
586         #5.绘制BMI-达标时间散点图
587         ax5 = plt.subplot(2, 3, 5)
588         #对绘制的图形，按组着色
589         for i, (bmi_min, bmi_max) in enumerate(bmi_groups):
590             if i == len(bmi_groups) - 1:
591                 mask = (self.modeling_df['孕妇BMI'] >= bmi_min) & (self.modeling_df
592                 ['孕妇BMI'] <= bmi_max)
593             else:
594                 mask = (self.modeling_df['孕妇BMI'] >= bmi_min) & (self.modeling_df
595                 ['孕妇BMI'] < bmi_max)
596             group_data = self.modeling_df[mask]
597             if not group_data.empty:
598                 color = colors[i % len(colors)]
599                 plt.scatter(group_data['孕妇BMI'], group_data['最早达标时间_天数
600                 ]/7,
601                 alpha=0.6, color=color, label=f'组{i+1}', s=30)
602                 plt.xlabel('孕妇BMI')
603                 plt.ylabel('最早达标时间 (周)')
604                 plt.title('BMI vs 达标时间分布')
605                 plt.legend()
606                 plt.grid(True, alpha=0.3)
607                 #6.输出的详细信息表格
608                 ax6 = plt.subplot(2, 3, 6)
609                 ax6.axis('tight')
610                 ax6.axis('off')
611                 #准备表格数据
612                 table_data = optimal_timepoints[['组别', 'BMI区间', '孕妇数', '最佳时点_周
613                 数', '预期成功率', '风险等级']].round(2)
614                 table_data['预期成功率'] = table_data['预期成功率'].apply(lambda x: f'{x
615                 :.1%}')
616                 table = ax6.table(cellText=table_data.values, colLabels=table_data.columns,
617                 cellLoc='center', loc='center', fontsize=9)

```

```

611     table.auto_set_font_size(False)
612     table.set_fontsize(8)
613     table.scale(1, 1.5)
614     ax6.set_title('详细结果汇总', pad=20)
615     plt.tight_layout()
616     #将输出结果保存为图片
617     filename = f'{method_name}_BMI分组与时点优化可视化.png'
618     plt.savefig(filename, dpi=300, bbox_inches='tight')
619     print(f"可视化图表已保存为: {filename}")
620     plt.show()
621     return fig
622
623 def _labels_to_bmi_intervals(self, labels):
624     unique_labels = sorted(set(labels))
625     bmi_groups = []
626     for label in unique_labels:
627         mask = np.array(labels) == label
628         group_bmis = self.modeling_df[mask]['孕妇BMI']
629         bmi_groups.append((group_bmis.min(), group_bmis.max()))
630     return sorted(bmi_groups, key=lambda x: x[0])
631
632 def _evaluate_groups(self, bmi_groups, method_name):
633     results = []
634     for i, (lower, upper) in enumerate(bmi_groups):
635         if i == len(bmi_groups) - 1:
636             group_data = self.modeling_df[(self.modeling_df['孕妇BMI'] >= lower
637 ) &
638                                     (self.modeling_df['孕妇BMI'] <= upper)]
639         else:
640             group_data = self.modeling_df[(self.modeling_df['孕妇BMI'] >= lower
641 ) &
642                                     (self.modeling_df['孕妇BMI'] < upper)]
643         if len(group_data) == 0:
644             continue
645         women_stats = group_data.groupby('孕妇代码').agg({
646             '最早达标时间_天数': 'first',
647             '风险指标': 'first'
648         }).reset_index()
649         risk_distribution = women_stats['风险指标'].value_counts().sort_index()
650         total_women = len(women_stats)
651         risk_weights = {0: 1, 1: 2, 2: 4}
652         weighted_risk = sum(risk_weights.get(risk, 2) * count
653                             for risk, count in risk_distribution.items()) /
654         total_women
655         results.append({
656             '组别': f"组{i+1}",
657             'BMI区间': f"[{lower:.1f}, {upper:.1f})",

```

```

655         '孕妇数': total_women,
656         '平均达标时间': women_stats['最早达标时间_天数'].mean(),
657         '风险得分': weighted_risk,
658         '低风险比例': risk_distribution.get(0, 0) / total_women,
659         '高风险比例': risk_distribution.get(1, 0) / total_women,
660         '极高风险比例': risk_distribution.get(2, 0) / total_women
661     })
662     return pd.DataFrame(results)
663
664     def compare_all_methods(self):
665         print("修正版高级建模方法综合对比 - 含时点优化与可视化")
666         methods_results = {}
667         all_optimal_timepoints = {}
668         #依次运行前面所建立是所有方法，并得出结果
669         try:
670             groups1, stats1 = self.method1_gaussian_mixture_clustering()
671             if stats1 is not None:
672                 methods_results['GMM_修正版'] = (groups1, stats1)
673                 #为该方法优化时点
674                 optimal_tp1 = self.optimize_detection_timepoints(groups1, "GMM_修正
版")
675                 if optimal_tp1 is not None:
676                     all_optimal_timepoints['GMM_修正版'] = optimal_tp1
677                     #进行敏感性分析
678                     self.sensitivity_analysis(optimal_tp1, "GMM_修正版")
679                     #可视化结果
680                     self.visualize_groups_and_timepoints(groups1, optimal_tp1, "
GMM_修正版")
681         except Exception as e:
682             print(f"GMM方法失败: {e}")
683         try:
684             groups2, stats2 = self.method2_hierarchical_clustering()
685             if stats2 is not None:
686                 methods_results['层次聚类_修正版'] = (groups2, stats2)
687                 optimal_tp2 = self.optimize_detection_timepoints(groups2, "层次聚类
_修正版")
688                 if optimal_tp2 is not None:
689                     all_optimal_timepoints['层次聚类_修正版'] = optimal_tp2
690                     self.sensitivity_analysis(optimal_tp2, "层次聚类_修正版")
691                     self.visualize_groups_and_timepoints(groups2, optimal_tp2, "层
次聚类_修正版")
692         except Exception as e:
693             print(f"层次聚类失败: {e}")
694         try:
695             groups3, stats3 = self.method3_dbscan_clustering()
696             if stats3 is not None:
697                 methods_results['DBSCAN_修正版'] = (groups3, stats3)

```

```

698         optimal_tp3 = self.optimize_detection_timepoints(groups3, "DBSCAN_
修正版")
699         if optimal_tp3 is not None:
700             all_optimal_timepoints['DBSCAN_修正版'] = optimal_tp3
701             self.sensitivity_analysis(optimal_tp3, "DBSCAN_修正版")
702             self.visualize_groups_and_timepoints(groups3, optimal_tp3, "
DBSCAN_修正版")
703         except Exception as e:
704             print(f"DBSCAN方法失败: {e}")
705         try:
706             groups4, stats4 = self.method4_optimal_binning()
707             if stats4 is not None:
708                 methods_results['风险优化_修正版'] = (groups4, stats4)
709                 optimal_tp4 = self.optimize_detection_timepoints(groups4, "风险优化
_修正版")
710                 if optimal_tp4 is not None:
711                     all_optimal_timepoints['风险优化_修正版'] = optimal_tp4
712                     self.sensitivity_analysis(optimal_tp4, "风险优化_修正版")
713                     self.visualize_groups_and_timepoints(groups4, optimal_tp4, "风
险优化_修正版")
714                 except Exception as e:
715                     print(f"风险优化方法失败: {e}")
716                 try:
717                     groups5, stats5 = self.method5_decision_tree_binning()
718                     if stats5 is not None:
719                         methods_results['决策树_修正版'] = (groups5, stats5)
720                         optimal_tp5 = self.optimize_detection_timepoints(groups5, "决策树_
修正版")
721                         if optimal_tp5 is not None:
722                             all_optimal_timepoints['决策树_修正版'] = optimal_tp5
723                             self.sensitivity_analysis(optimal_tp5, "决策树_修正版")
724                             self.visualize_groups_and_timepoints(groups5, optimal_tp5, "决
策树_修正版")
725                         except Exception as e:
726                             print(f"决策树方法失败: {e}")
727
728         #运行完所有方法后，综合对比，包括时点优化结果
729         if methods_results:
730             print("各方法综合评估结果")
731             comparison = []
732             for method_name, (groups, stats) in methods_results.items():
733                 #基础分组评估
734                 avg_risk = stats['风险得分'].mean()
735                 risk_std = stats['风险得分'].std()
736                 n_groups = len(stats)
737                 overlap_count = self._check_overlaps(groups, "")
738                 #时点优化评估

```

```

739         if method_name in all_optimal_timepoints:
740             optimal_tp = all_optimal_timepoints[method_name]
741             avg_success_rate = optimal_tp['预期成功率'].mean()
742             avg_timepoint = optimal_tp['最佳时点_周数'].mean()
743             low_risk_ratio = (optimal_tp['风险等级'] == '低风险').sum() /
len(optimal_tp)
744         else:
745             avg_success_rate = 0
746             avg_timepoint = 0
747             low_risk_ratio = 0
748             comparison.append({
749                 '方法': method_name,
750                 '分组数': n_groups,
751                 '平均风险得分': avg_risk,
752                 '风险得分标准差': risk_std,
753                 '平均成功率': avg_success_rate,
754                 '平均最佳时点_周': avg_timepoint,
755                 '低风险组比例': low_risk_ratio,
756                 '重叠检查': '无重叠' if overlap_count == 0 else f'{
overlap_count}个重叠'
757             })
758             comparison_df = pd.DataFrame(comparison)
759             #对所有方法进行评估后，按综合评分排序，根据前面定义，风险得分越小越好，
成功率越高越好
760             comparison_df['综合评分'] = comparison_df['平均风险得分'] * 2 -
comparison_df['平均成功率'] * 3 + (1 - comparison_df['低风险组比例']) * 1
761             comparison_df = comparison_df.sort_values('综合评分')
762             print("方法综合对比结果:")
763             display_cols = ['方法', '分组数', '平均风险得分', '平均成功率', '平均最
佳时点_周', '低风险组比例', '重叠检查']
764             print(comparison_df[display_cols].round(3))
765             #综合所有方法，得出的最佳方法
766             best_method = comparison_df.iloc[0]['方法']
767             print(f"\n综合推荐方法: {best_method}")
768             print(f"    平均风险得分: {comparison_df.iloc[0]['平均风险得分']:.3f}")
769             print(f"    平均成功率: {comparison_df.iloc[0]['平均成功率']:.3f}")
770             print(f"    平均最佳时点: {comparison_df.iloc[0]['平均最佳时点_周']:.1f}
周")
771             #将所有输出结果到Excel
772             self.save_comprehensive_results(methods_results, all_optimal_timepoints
, comparison_df)
773             return methods_results, all_optimal_timepoints
774         else:
775             print("所有方法都失败了")
776             return None, None
777
778     def save_comprehensive_results(self, methods_results, all_optimal_timepoints,

```

```

comparison_df):
779     print(f"\n保存综合分析结果...")
780     try:
781         filename = "问题2_综合BMI分组与时点优化结果.xlsx"
782         with pd.ExcelWriter(filename, engine='openpyxl') as writer:
783             #1.方法对比总表
784             comparison_df.to_excel(writer, sheet_name='方法综合对比', index=
False)
785             #2.各方法的分组统计
786             for method_name, (groups, stats) in methods_results.items():
787                 sheet_name = f"{method_name}_分组统计"[:31] # Excel sheet名称
限制
788                 stats.to_excel(writer, sheet_name=sheet_name, index=False)
789             #3.各方法的最优时点
790             for method_name, optimal_tp in all_optimal_timepoints.items():
791                 sheet_name = f"{method_name}_最优时点"[:31]
792                 optimal_tp.to_excel(writer, sheet_name=sheet_name, index=False)
793             #4.BMI分组区间总结
794             groups_summary = []
795             for method_name, (groups, _) in methods_results.items():
796                 for i, (bmi_min, bmi_max) in enumerate(groups):
797                     groups_summary.append({
798                         '方法': method_name,
799                         '组别': f'组{i+1}',
800                         'BMI下界': bmi_min,
801                         'BMI上界': bmi_max,
802                         '区间范围': bmi_max - bmi_min
803                     })
804             groups_summary_df = pd.DataFrame(groups_summary)
805             groups_summary_df.to_excel(writer, sheet_name='BMI分组区间总结',
index=False)
806             print(f"综合结果已保存到: {filename}")
807             except Exception as e:
808                 print(f"保存Excel文件失败: {e}")
809
810 #已准备好的所有方法，开始代入数据使用
811 def run_comprehensive_bmi_analysis():
812     try:
813         print("开始运行综合BMI分组与时点优化分析...")
814         analyzer = AdvancedBMIGroupingMethodsFixed("问题2_建模数据.csv")
815         methods_results, optimal_timepoints = analyzer.compare_all_methods()
816         if methods_results and optimal_timepoints:
817             print("综合分析完成!")
818             print("\n分析成果总结:")
819             print(f"成功测试了 {len(methods_results)} 种BMI分组方法")
820             print(f"为 {len(optimal_timepoints)} 种方法完成了时点优化")
821             print(f"生成了 {len(optimal_timepoints)} 套可视化图表")

```

```

822         print(f"完成了敏感性分析")
823         print(f"保存了综合分析报告")
824         print("\n各方法最佳时点概览:")
825         for method_name, optimal_tp in optimal_timepoints.items():
826             avg_timepoint = optimal_tp['最佳时点_周数'].mean()
827             avg_success = optimal_tp['预期成功率'].mean()
828             print(f"    {method_name}: 平均时点 {avg_timepoint:.1f}周, 平均成功率 {avg_success:.1%}")
829         return methods_results, optimal_timepoints
830     else:
831         print("分析失败")
832         return None, None
833 except Exception as e:
834     print(f"综合分析失败: {e}")
835     import traceback
836     traceback.print_exc()
837     return None, None
838 def run_fixed_advanced_modeling():
839     return run_comprehensive_bmi_analysis()
840 if __name__ == "__main__":
841     run_comprehensive_bmi_analysis()

```

问题 3.py

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.ensemble import RandomForestRegressor
4  from sklearn.cluster import KMeans, AgglomerativeClustering
5  from sklearn.mixture import GaussianMixture
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.metrics import silhouette_score
8  from scipy.optimize import minimize_scalar
9  from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
10 import matplotlib.pyplot as plt
11 import matplotlib.patches as patches
12 import seaborn as sns
13 from matplotlib.font_manager import FontProperties
14 import warnings
15 warnings.filterwarnings('ignore')
16 #设置中文字体和样式, 便于后续作图和输出展示
17 plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
18 plt.rcParams['axes.unicode_minus'] = False
19 #[13]AI构建基础算法流程框架, 再根据实际问题需要进行细化和完善
20 #[8]同时添加一些可视化过程使其更易于理解
21 class Problem3ComprehensiveAnalysis:
22     def __init__(self, data_file):
23         self.df = pd.read_csv(data_file, encoding='utf-8-sig')
24         self.scaler = StandardScaler()

```



```

25     print("问题三深度分析：多因素综合BMI分组与NIPT时点优化")
26     #首先对数据进行预处理
27     self._preprocess_data()
28
29     def _preprocess_data(self):
30         print("\n1. 数据预处理和特征工程")
31         print(f"原始数据shape: {self.df.shape}")
32         #遍历数据，移除有缺失关键数据的行
33         required_cols = ['孕妇BMI', '孕妇代码', '年龄', '身高', '体重']
34         self.df = self.df.dropna(subset=required_cols)
35         print(f"清理后数据shape: {self.df.shape}")
36         #以孕妇为依据，对数据进行聚合
37         agg_dict = {}
38         for col in self.df.columns:
39             if col == '孕妇代码':
40                 continue
41             elif col in ['Y染色体浓度', '当前检测达标']:
42                 agg_dict[col] = ['mean', 'std', 'count']
43             else:
44                 agg_dict[col] = 'first'
45         self.women_data = self.df.groupby('孕妇代码').agg(agg_dict).reset_index()
46         #对于多级列名，进行处理
47         if isinstance(self.women_data.columns, pd.MultiIndex):
48             new_cols = ['孕妇代码']
49             for col in self.women_data.columns[1:]:
50                 if isinstance(col, tuple):
51                     if col[1] == 'first':
52                         new_cols.append(col[0])
53                     else:
54                         new_cols.append(f'{col[0]}_{col[1]}')
55                 else:
56                     new_cols.append(col)
57             self.women_data.columns = new_cols
58         print(f"聚合后数据shape: {self.women_data.shape}")
59         #特征工程
60         self._create_derived_features()
61
62     def _create_derived_features(self):
63         print("  创建衍生特征...")
64         if '孕妇BMI' in self.women_data.columns:
65             self.women_data['BMI_平方'] = self.women_data['孕妇BMI'] ** 2
66         if '年龄' in self.women_data.columns:
67             self.women_data['年龄_平方'] = self.women_data['年龄'] ** 2
68             self.women_data['高龄产妇'] = (self.women_data['年龄'] >= 35).astype(
69                 int)
70             if 'Y染色体浓度_mean' in self.women_data.columns and 'Y染色体浓度_std' in
71                 self.women_data.columns:

```

```

70         self.women_data['Y浓度_变异系数'] = (
71             self.women_data['Y染色体浓度_std'] / self.women_data['Y染色体浓度
_mean']
72         ).fillna(0)
73         if '当前检测达标_mean' in self.women_data.columns:
74             self.women_data['达标率'] = self.women_data['当前检测达标_mean']
75         if '孕妇BMI' in self.women_data.columns and '年龄' in self.women_data.
columns:
76             self.women_data['BMI_年龄_乘积'] = self.women_data['孕妇BMI'] * self.
women_data['年龄']
77         print(f"    特征工程后 shape: {self.women_data.shape}")
78
79     def analyze_feature_importance(self):
80         print("\n2. 特征重要性分析")
81         numeric_cols = self.women_data.select_dtypes(include=[np.number]).columns.
tolist()
82         feature_cols = [col for col in numeric_cols if col not in ['孕妇代码', '最
早达标时间_天数']]
83         if '最早达标时间_天数' not in self.women_data.columns or len(feature_cols)
< 2:
84             print("缺少目标变量或可用特征不足，跳过特征重要性分析")
85             self.feature_importance = None
86             return None
87         X = self.women_data[feature_cols].fillna(self.women_data[feature_cols].
median())
88         y = self.women_data['最早达标时间_天数'].fillna(self.women_data['最早达标时
间_天数'].median())
89         #增强数据的可解释性：并且显示数据准备过程
90         print(f"分析特征数: {len(feature_cols)}")
91         print(f"样本数: {len(X)}")
92         print(f"缺失值填充策略: 中位数填充")
93         rf = RandomForestRegressor(n_estimators=100, random_state=42)
94         rf.fit(X, y)
95         self.feature_importance= pd.DataFrame({
96             '特征': feature_cols,
97             '重要性': rf.feature_importances_
98         }).sort_values('重要性', ascending=False)
99         print("对'最早达标时间'影响最大的特征排序:")
100        print(self.feature_importance.head(10).round(4))
101        #绘制特征重要性图，可视化展示结果
102        self._visualize_feature_importance()
103        #增强代码的可解释性，用于解释关键特征
104        self._explain_key_features()
105        return self.feature_importance
106
107    def _visualize_feature_importance(self):
108        if self.feature_importance is None:

```

```

109         return
110     print(" 生成特征重要性可视化图表...")
111     #提取前10个最重要的特征
112     top_features = self.feature_importance.head(10)
113     #创建一个新的图表，用于后续绘图
114     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
115     #1.绘制横向条形图
116     colors = plt.cm.viridis(np.linspace(0, 1, len(top_features)))
117     bars = ax1.barh(range(len(top_features)), top_features['重要性'], color=
colors)
118     ax1.set_yticks(range(len(top_features)))
119     ax1.set_yticklabels(top_features['特征'], fontsize=10)
120     ax1.set_xlabel('特征重要性')
121     ax1.set_title('前10个最重要特征 (随机森林)', fontsize=12, fontweight='bold'
)
122     ax1.grid(True, alpha=0.3)
123     #在已绘制的条形图上，标注数据的数值
124     for i, (bar, importance) in enumerate(zip(bars, top_features['重要性'])):
125         width = bar.get_width()
126         ax1.text(width + 0.001, bar.get_y() + bar.get_height()/2,
127                 f'{importance:.3f}', ha='left', va='center', fontsize=9)
128     #2.绘制累积重要性图
129     cumsum_importance = top_features['重要性'].cumsum()
130     ax2.plot(range(1, len(top_features)+1), cumsum_importance, 'o-', linewidth
=2, markersize=6)
131     ax2.fill_between(range(1, len(top_features)+1), cumsum_importance, alpha
=0.3)
132     ax2.set_xlabel('特征数量')
133     ax2.set_ylabel('累积重要性')
134     ax2.set_title('特征重要性累积分布', fontsize=12, fontweight='bold')
135     ax2.grid(True, alpha=0.3)
136     ax2.set_xticks(range(1, len(top_features)+1))
137     #在已绘制的累积重要性图上，添加80%重要性线
138     ax2.axhline(y=0.8, color='red', linestyle='--', alpha=0.7, label='80%重要性
线')
139     ax2.legend()
140     plt.tight_layout()
141     #将已绘制的图片保存在路径中
142     filename = '特征重要性分析可视化.png'
143     plt.savefig(filename, dpi=300, bbox_inches='tight')
144     print(f" 特征重要性图表已保存为: {filename}")
145     plt.show()
146
147     def _explain_key_features(self):
148         if self.feature_importance is None:
149             return
150         print(" 关键特征解释:")

```

```

151     top_3_features = self.feature_importance.head(3)
152     feature_explanations = {
153         '孕妇BMI': 'BMI是预测检测时点的核心因素，BMI高的孕妇可能需要更长时间达
154         标',
155         'BMI_平方': 'BMI的平方项可以捕捉非线性关系，反映BMI极值对检测时点的影响',
156         '年龄': '孕妇年龄影响生理状态，高龄产妇可能影响检测结果',
157         '年龄_平方': '年龄的平方项反映年龄极值的非线性影响',
158         '高龄产妇': '35岁以上高龄产妇标识，直接影响检测策略',
159         'Y浓度_变异系数': 'Y染色体浓度变异程度，反映检测稳定性',
160         'BMI_年龄_乘积': 'BMI和年龄的交互作用，复合影响因子',
161         '达标率': '历史达标情况，预测未来检测成功概率',
162         '身高': '孕妇身高影响BMI计算和生理特征',
163         '体重': '孕妇体重直接影响BMI和检测时点'
164     }
165     for _, row in top_3_features.iterrows():
166         feature_name = row['特征']
167         importance = row['重要性']
168         explanation = feature_explanations.get(feature_name, '该特征对检测时点
169         有重要影响')
170         print(f"    {feature_name} (重要性: {importance:.3f}): {explanation}")
171         #提取前3个特征，并计算其累积重要性占比
172         total_importance = self.feature_importance['重要性'].sum()
173         top3_importance = top_3_features['重要性'].sum()
174         coverage = top3_importance / total_importance
175         print(f"    前3个特征累计解释了 {coverage:.1%} 的预测能力")
176
177     def multi_factor_clustering(self):
178         print("\n3. 多因素聚类分组")
179
180         core_features = ['孕妇BMI', '年龄', '身高', '体重']
181         clustering_features = set(core_features)
182         if hasattr(self, 'feature_importance') and self.feature_importance is not
183         None:
184             additional_features = self.feature_importance.head(3)['特征'].tolist()
185             print(f"根据数据驱动分析，添加补充特征: {additional_features}")
186             for feature in additional_features:
187                 clustering_features.add(feature)
188             final_clustering_features = [f for f in list(clustering_features) if f in
189             self.women_data.columns]
190             print(f"最终用于聚类的综合特征: {final_clustering_features}")
191             X_cluster = self.women_data[final_clustering_features].fillna(self.
192             women_data[final_clustering_features].median())
193             X_scaled = self.scaler.fit_transform(X_cluster)
194             #为了增强代码的可解释性，将在此处显示数据预处理信息
195             print(f"聚类数据维度: {X_scaled.shape}")
196             print(f"数据标准化: 使用StandardScaler进行Z-score标准化")

```

```

192     clustering_results = {}
193     print("\n K-means聚类:")
194     clustering_results['KMeans'] = self._kmeans_clustering(X_scaled)
195     print("\n 层次聚类:")
196     clustering_results['Hierarchical'] = self._hierarchical_clustering(X_scaled
)
197     print("\n 高斯混合模型(GMM):")
198     clustering_results['GMM'] = self._gmm_clustering(X_scaled)
199     best_method = self._select_best_clustering(clustering_results)
200     self.best_clustering = clustering_results[best_method]
201     self.women_data['cluster'] = self.best_clustering['labels']
202     print(f"\n最佳聚类方法: {best_method}")
203     #将聚类过程可视化
204     self._visualize_clustering_process(X_scaled, final_clustering_features,
clustering_results, best_method)
205     #添加新的聚类步骤, 生成并修正BMI区间
206     self.fixed_bmi_groups = self._generate_and_fix_bmi_groups()
207     print("\n修正后的无重叠BMI分组区间:")
208     for i, grp in enumerate(self.fixed_bmi_groups):
209         print(f" 组 {i+1}: [{grp[0]:.2f}, {grp[1]:.2f}]")
210     # 将BMI分组结果可视化
211     self._visualize_bmi_grouping()
212     return clustering_results
213
214     def _visualize_clustering_process(self, X_scaled, feature_names,
clustering_results, best_method):
215         print(" 生成聚类过程可视化图表...")
216         fig = plt.figure(figsize=(20, 12))
217         #1.所有聚类方法进行对比
218         ax1 = plt.subplot(2, 4, 1)
219         methods = list(clustering_results.keys())
220         scores = [clustering_results[method]['score'] for method in methods]
221         n_clusters = [clustering_results[method]['n_clusters'] for method in
methods]
222         colors = ['red', 'blue', 'green']
223         bars = ax1.bar(methods, scores, color=colors)
224         ax1.set_ylabel('轮廓系数')
225         ax1.set_title('聚类方法对比', fontweight='bold')
226         ax1.grid(True, alpha=0.3)
227         #综合所有聚类方法后, 得出最佳方法后, 标注出来
228         best_idx = methods.index(best_method)
229         bars[best_idx].set_edgecolor('gold')
230         bars[best_idx].set_linewidth(3)
231         #在绘制的图片中, 标注聚类数和得分
232         for i, (bar, score, n_clus) in enumerate(zip(bars, scores, n_clusters)):
233             height = bar.get_height()
234             ax1.text(bar.get_x() + bar.get_width()/2., height + 0.01,

```

```

235         f'{n_clus}组\n{score:.3f}',
236         ha='center', va='bottom', fontsize=9, fontweight='bold')
237 #2.最佳聚类方法的聚类结果（前两个主要特征的散点图）
238 ax2 = plt.subplot(2, 4, 2)
239 if len(feature_names) >= 2:
240     # 使用前面两个重要特征进行可视化
241     feat1, feat2 = feature_names[0], feature_names[1]
242     feat1_idx = feature_names.index(feat1)
243     feat2_idx = feature_names.index(feat2)
244     best_labels = self.best_clustering['labels']
245     scatter = ax2.scatter(X_scaled[:, feat1_idx], X_scaled[:, feat2_idx],
246                          c=best_labels, cmap='viridis', alpha=0.6, s=30)
247     ax2.set_xlabel(f'{feat1} (标准化)')
248     ax2.set_ylabel(f'{feat2} (标准化)')
249     ax2.set_title(f'{best_method} 聚类结果', fontweight='bold')
250     plt.colorbar(scatter, ax=ax2)
251 #3.层次聚类树状图
252 ax3 = plt.subplot(2, 4, 3)
253 if 'Hierarchical' in clustering_results:
254     #重新计算层次聚类用于绘制树状图
255     linkage_matrix = linkage(X_scaled, method='ward')
256     dendrogram(linkage_matrix, ax=ax3, truncate_mode='level', p=5)
257     ax3.set_title('层次聚类树状图', fontweight='bold')
258     ax3.set_xlabel('样本索引')
259     ax3.set_ylabel('距离')
260 else:
261     ax3.text(0.5, 0.5, '仅在层次聚类时显示', ha='center', va='center',
transform=ax3.transAxes)
262     ax3.set_title('层次聚类树状图')
263 #4.绘制聚类数量选择过程图
264 ax4 = plt.subplot(2, 4, 4)
265 k_range = range(2, 7)
266 inertias = []
267 silhouettes = []
268 for k in k_range:
269     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
270     labels = kmeans.fit_predict(X_scaled)
271     inertias.append(kmeans.inertia_)
272     if len(set(labels)) > 1:
273         silhouettes.append(silhouette_score(X_scaled, labels))
274     else:
275         silhouettes.append(0)
276 ax4_twin = ax4.twinx()
277 line1 = ax4.plot(k_range, inertias, 'b-o', label='惯性 (左轴)')
278 line2 = ax4_twin.plot(k_range, silhouettes, 'r-s', label='轮廓系数 (右轴)')
279 ax4.set_xlabel('聚类数 K')
280 ax4.set_ylabel('惯性', color='b')

```

```

281     ax4_twin.set_ylabel('轮廓系数', color='r')
282     ax4.set_title('聚类数量选择 (肘部法则)', fontweight='bold')
283     ax4.grid(True, alpha=0.3)
284     #将生成的图例合并
285     lines = line1 + line2
286     labels = [l.get_label() for l in lines]
287     ax4.legend(lines, labels, loc='center right')
288     #5.绘制各组BMI分布对比图
289     ax5 = plt.subplot(2, 4, 5)
290     best_labels = self.best_clustering['labels']
291     bmi_data = self.women_data['孕妇BMI']
292     unique_labels = sorted(list(set(best_labels)))
293     colors = plt.cm.viridis(np.linspace(0, 1, len(unique_labels)))
294     for i, label in enumerate(unique_labels):
295         group_bmi = bmi_data[best_labels == label]
296         ax5.hist(group_bmi, alpha=0.6, label=f'组{label+1}', color=colors[i],
bins=15)
297     ax5.set_xlabel('孕妇BMI')
298     ax5.set_ylabel('频次')
299     ax5.set_title('各组BMI分布', fontweight='bold')
300     ax5.legend()
301     ax5.grid(True, alpha=0.3)
302     #6.各组统计信息表格
303     ax6 = plt.subplot(2, 4, 6)
304     ax6.axis('tight')
305     ax6.axis('off')
306     #计算各组的统计信息
307     group_stats = []
308     for label in unique_labels:
309         mask = best_labels == label
310         group_data = self.women_data[mask]
311         stats = {
312             '组别': f'组{label+1}',
313             '样本数': len(group_data),
314             'BMI均值': group_data['孕妇BMI'].mean(),
315             'BMI标准差': group_data['孕妇BMI'].std(),
316             '年龄均值': group_data['年龄'].mean() if '年龄' in group_data.
columns else 0
317         }
318         group_stats.append(stats)
319     stats_df = pd.DataFrame(group_stats).round(2)
320     table = ax6.table(cellText=stats_df.values, colLabels=stats_df.columns,
321                       cellLoc='center', loc='center', fontsize=9)
322     table.auto_set_font_size(False)
323     table.set_fontsize(8)
324     table.scale(1, 1.5)
325     ax6.set_title('各组统计信息', fontweight='bold', pad=20)

```

```

326     #7.绘制图形来描述特征重要性在聚类中的体现
327     ax7 = plt.subplot(2, 4, 7)
328     if hasattr(self, 'feature_importance') and self.feature_importance is not
None:
329         # 显示聚类特征的重要性排序
330         clustering_feature_importance = self.feature_importance[
331             self.feature_importance['特征'].isin(feature_names)
332         ].head(6)
333         bars = ax7.barh(range(len(clustering_feature_importance)),
334                         clustering_feature_importance['重要性'])
335         ax7.set_yticks(range(len(clustering_feature_importance)))
336         ax7.set_yticklabels(clustering_feature_importance['特征'], fontsize=9)
337         ax7.set_xlabel('特征重要性')
338         ax7.set_title('聚类特征重要性', fontweight='bold')
339         ax7.grid(True, alpha=0.3)
340     else:
341         ax7.text(0.5, 0.5, '需要先进行\n特征重要性分析', ha='center', va='
center', transform=ax7.transAxes)
342         ax7.set_title('聚类特征重要性')
343     #8.绘制图进行轮廓分析
344     ax8 = plt.subplot(2, 4, 8)
345     from sklearn.metrics import silhouette_samples
346     best_labels = self.best_clustering['labels']
347     sample_silhouette_values = silhouette_samples(X_scaled, best_labels)
348     y_lower = 10
349     for i, label in enumerate(unique_labels):
350         cluster_silhouette_values = sample_silhouette_values[best_labels ==
label]
351         cluster_silhouette_values.sort()
352         size_cluster_i = cluster_silhouette_values.shape[0]
353         y_upper = y_lower + size_cluster_i
354         color = colors[i]
355         ax8.fill_betweenx(np.arange(y_lower, y_upper), 0,
cluster_silhouette_values,
356                         facecolor=color, edgecolor=color, alpha=0.7)
357         ax8.text(-0.05, y_lower + 0.5 * size_cluster_i, str(label+1))
358         y_lower = y_upper + 10
359     ax8.set_xlabel('轮廓系数值')
360     ax8.set_ylabel('聚类标签')
361     ax8.set_title('各组轮廓分析', fontweight='bold')
362     #对绘制的图形添加平均轮廓系数线
363     silhouette_avg = silhouette_score(X_scaled, best_labels)
364     ax8.axvline(x=silhouette_avg, color="red", linestyle="--",
365                label=f'平均值: {silhouette_avg:.3f}')
366     ax8.legend()
367     plt.tight_layout()
368     #保存输出的图片于代码路径中

```



```

369     filename = '聚类过程综合分析可视化.png'
370     plt.savefig(filename, dpi=300, bbox_inches='tight')
371     print(f"  聚类过程分析图表已保存为: {filename}")
372     plt.show()
373
374     def _visualize_bmi_grouping(self):
375         if not hasattr(self, 'fixed_bmi_groups') or not self.fixed_bmi_groups:
376             return
377         print("  生成BMI分组可视化图表...")
378         fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
379         #1.对BMI分布直方图与分组边界
380         ax1.hist(self.women_data['孕妇BMI'], bins=50, alpha=0.7, color='lightblue',
381                 edgcolor='black', label='BMI分布')
382         #绘制分组边界
383         colors = ['red', 'orange', 'green', 'purple', 'brown', 'pink']
384         for i, (bmi_min, bmi_max) in enumerate(self.fixed_bmi_groups):
385             color = colors[i % len(colors)]
386             ax1.axvline(bmi_min, color=color, linestyle='--', linewidth=2, alpha
=0.8)
387             if i == len(self.fixed_bmi_groups) - 1: # 最后一个区间显示右边界
388                 ax1.axvline(bmi_max, color=color, linestyle='--', linewidth=2,
alpha=0.8)
389             #对绘制的图形, 添加组别标签
390             mid_point = (bmi_min + bmi_max) / 2
391             ax1.text(mid_point, ax1.get_ylim()[1] * 0.8, f'组{i+1}',
392                     ha='center', va='center', fontsize=12, color=color,
393                     bbox=dict(boxstyle="round,pad=0.3", facecolor='white', alpha
=0.8))
394             ax1.set_xlabel('孕妇BMI')
395             ax1.set_ylabel('频次')
396             ax1.set_title('BMI分布与分组边界', fontweight='bold', fontsize=14)
397             ax1.grid(True, alpha=0.3)
398             ax1.legend()
399         #2.获取各组样本数量和BMI范围
400         ax2.axis('tight')
401         ax2.axis('off')
402         #创建新的表格, 用于信息分组
403         group_info = []
404         for i, (bmi_min, bmi_max) in enumerate(self.fixed_bmi_groups):
405             #统计该组的所有样本数
406             if i == len(self.fixed_bmi_groups) - 1:
407                 mask = (self.women_data['孕妇BMI'] >= bmi_min) & (self.women_data['
孕妇BMI'] <= bmi_max)
408             else:
409                 mask = (self.women_data['孕妇BMI'] >= bmi_min) & (self.women_data['
孕妇BMI'] < bmi_max)
410             sample_count = mask.sum()

```

```

411         avg_bmi = self.women_data[mask]['孕妇BMI'].mean() if sample_count > 0
else 0
412         group_info.append({
413             '组别': f'组{i+1}',
414             'BMI范围': f'[{bmi_min:.2f}, {bmi_max:.2f}]',
415             '区间宽度': f'{bmi_max - bmi_min:.2f}',
416             '样本数': sample_count,
417             'BMI均值': f'{avg_bmi:.2f}' if sample_count > 0 else 'N/A'
418         })
419     info_df = pd.DataFrame(group_info)
420     table = ax2.table(cellText=info_df.values, colLabels=info_df.columns,
421                      cellLoc='center', loc='center', fontsize=10)
422     table.auto_set_font_size(False)
423     table.set_fontsize(10)
424     table.scale(1.2, 2)
425     #对输出的表格进行美化
426     for i in range(len(info_df.columns)):
427         table[(0, i)].set_facecolor('#4CAF50')
428         table[(0, i)].set_text_props(weight='bold', color='white')
429     for i in range(1, len(info_df) + 1):
430         for j in range(len(info_df.columns)):
431             table[(i, j)].set_facecolor('#f0f0f0')
432     ax2.set_title('BMI分组详细信息', fontweight='bold', fontsize=14, pad=20)
433     plt.tight_layout()
434     #将输出的图片保存在代码困境路径中
435     filename = 'BMI分组结果可视化.png'
436     plt.savefig(filename, dpi=300, bbox_inches='tight')
437     print(f"    BMI分组结果图表已保存为: {filename}")
438     plt.show()
439
440     def _kmeans_clustering(self, X_scaled):
441         best_score = -1; best_k = 0; best_labels = None
442         for k in range(2, 7):
443             kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
444             labels = kmeans.fit_predict(X_scaled)
445             if len(set(labels)) > 1:
446                 silhouette = silhouette_score(X_scaled, labels)
447                 if silhouette > best_score:
448                     best_score, best_k, best_labels = silhouette, k, labels
449         print(f"    最佳K: {best_k}, 轮廓系数: {best_score:.3f}")
450         return {'method': 'KMeans', 'n_clusters': best_k, 'labels': best_labels, '
score': best_score}
451
452     def _hierarchical_clustering(self, X_scaled):
453         linkage_matrix = linkage(X_scaled, method='ward')
454         best_score = -1; best_n = 0; best_labels = None
455         for n in range(2, 7):

```

```

456         labels = fcluster(linkage_matrix, n, criterion='maxclust') - 1
457         if len(set(labels)) > 1:
458             silhouette = silhouette_score(X_scaled, labels)
459             if silhouette > best_score:
460                 best_score, best_n, best_labels = silhouette, n, labels
461         print(f"    最佳聚类数: {best_n}, 轮廓系数: {best_score:.3f}")
462         return {'method': 'Hierarchical', 'n_clusters': best_n, 'labels':
best_labels, 'score': best_score}
463
464     def _gmm_clustering(self, X_scaled):
465         best_bic = np.inf; best_n = 0; best_labels = None
466         for n in range(2, 7):
467             gmm = GaussianMixture(n_components=n, random_state=42)
468             labels = gmm.fit_predict(X_scaled)
469             bic = gmm.bic(X_scaled)
470             if bic < best_bic:
471                 best_bic, best_n, best_labels = bic, n, labels
472             silhouette = silhouette_score(X_scaled, best_labels)
473             print(f"    最佳聚类数: {best_n}, BIC: {best_bic:.1f}, 轮廓系数: {
silhouette:.3f}")
474             return {'method': 'GMM', 'n_clusters': best_n, 'labels': best_labels, '
score': silhouette}
475
476     def _select_best_clustering(self, clustering_results):
477         comparison = [{'方法': name, '聚类数': res['n_clusters'], '评估得分(轮廓系
数)': res['score']}
478                         for name, res in clustering_results.items()]
479         # 将对比结果保存为类属性，以便后续输出
480         self.clustering_comparison_df = pd.DataFrame(comparison).sort_values('评估
得分(轮廓系数)', ascending=False)
481         print(f"\n    聚类方法对比:")
482         print(self.clustering_comparison_df.round(3))
483         return self.clustering_comparison_df.iloc[0]['方法']
484
485     # 新增核心函数，用于生成并修正重叠区间
486     def _generate_and_fix_bmi_groups(self):
487         if not hasattr(self, 'best_clustering') or 'labels' not in self.
best_clustering:
488             return []
489         labels = self.best_clustering['labels']
490         unique_labels = sorted(list(set(labels)))
491         # 1. 生成初始的、可能重叠的BMI分组
492         initial_groups = []
493         for label in unique_labels:
494             group_bmi = self.women_data[self.women_data['cluster'] == label]['孕
妇BMI']
495             if not group_bmi.empty:

```

```

496         initial_groups.append([group_bmi.min(), group_bmi.max()])
497     if not initial_groups:
498         return []
499     #2.将生成的区间按下界进行排序
500     sorted_groups = sorted(initial_groups, key=lambda x: x[0])
501     #3.迭代检查并修正重叠
502     fixed_groups = [sorted_groups[0]]
503     for i in range(1, len(sorted_groups)):
504         prev_lower, prev_upper = fixed_groups[-1]
505         curr_lower, curr_upper = sorted_groups[i]
506         # 如果检测到重叠
507         if curr_lower < prev_upper:
508             #计算重叠部分的中点作为新的分界线
509             mid_point = (prev_upper + curr_lower) / 2
510             fixed_groups[-1][1] = mid_point #修正前一个区间的上界
511             sorted_groups[i][0] = mid_point #修正当前区间的下界
512             fixed_groups.append(sorted_groups[i])
513         #如果没有重叠
514         else:
515             fixed_groups.append(sorted_groups[i])
516     return fixed_groups
517 #结束新增加的函数
518
519 def optimize_detection_timepoints(self):
520     print("\n4. 各分组检测时点优化")
521     #使用修正后的无重叠区间来进行优化
522     print(f"优化策略说明:")
523     print(f" - 目标函数: 最小化 [时间风险 + 失败风险]")
524     print(f" - 时间风险: 12周内(风险1.0) → 27周内(风险2.0) → 27周后(风险4.0)")
525     print(f" - 失败风险: (1-成功率) × 3.0")
526     print(f" - 搜索范围: 10-30周")
527     optimal_timepoints = []
528     optimization_details = [] #存储优化过程的详细信息
529     for i, (bmi_min, bmi_max) in enumerate(self.fixed_bmi_groups):
530         #根据修正后的清晰区间筛选数据
531         #对于最后一个区间, 使用闭开区间
532         if i == len(self.fixed_bmi_groups) - 1:
533             cluster_data = self.women_data[
534                 (self.women_data['孕妇BMI'] >= bmi_min) & (self.women_data['孕
535 妇BMI'] <= bmi_max)
536             ]
537         else: #而其他区间, 使用左闭右开区间
538             cluster_data = self.women_data[
539                 (self.women_data['孕妇BMI'] >= bmi_min) & (self.women_data['孕
540 妇BMI'] < bmi_max)
541             ]
542         if cluster_data.empty:

```

```

541         print(f"\n 警告：组别 {i + 1} (BMI [{bmi_min:.2f}, {bmi_max:.2f}])
           内没有样本数据，已跳过。")
542         continue
543         print(f"\n 分析组别 {i + 1}: BMI [{bmi_min:.2f}, {bmi_max:.2f}] (样本
           数 n={len(cluster_data)})")
544         #增强代码的可解释性，显示该组的基本统计信息
545         detection_times = cluster_data['最早达标时间_天数']
546         print(f"    该组达标时间分布：均值{detection_times.mean():.1f}天，中位
           数{detection_times.median():.1f}天")
547         print(f"    达标时间范围：{detection_times.min():.0f}-{detection_times.
           max():.0f}天")
548         #存储目标函数计算过程并用于可视化
549         time_points = np.linspace(10*7, 30*7, 100)
550         objective_values = []
551         success_rates = []
552         time_risks = []
553         failure_risks = []
554
555         def objective(t):
556             success_rate = (cluster_data['最早达标时间_天数'] <= t).mean()
557             if t <= 12 * 7: time_risk = 1.0
558             elif t <= 27 * 7: time_risk = 2.0
559             else: time_risk = 4.0
560             failure_risk = (1 - success_rate) * 3.0
561             return time_risk + failure_risk
562         #计算目标函数曲线并用于可视化
563         for t in time_points:
564             success_rate = (cluster_data['最早达标时间_天数'] <= t).mean()
565             if t <= 12 * 7: time_risk = 1.0
566             elif t <= 27 * 7: time_risk = 2.0
567             else: time_risk = 4.0
568             failure_risk = (1 - success_rate) * 3.0
569             objective_values.append(time_risk + failure_risk)
570             success_rates.append(success_rate)
571             time_risks.append(time_risk)
572             failure_risks.append(failure_risk)
573         result = minimize_scalar(objective, bounds=(10*7, 30*7), method='
           bounded')
574         optimal_time = result.x
575         performance = self._evaluate_timepoint(cluster_data, optimal_time)
576         #增强代码的可解释性，并解释优化结果
577         optimal_success_rate = (cluster_data['最早达标时间_天数'] <=
           optimal_time).mean()
578         print(f"    优化结果：最佳时点 {optimal_time/7:.1f}周 ({optimal_time:.0
           f}天)")
579         print(f"    在最佳时点的预期成功率：{optimal_success_rate:.1%}")
580         print(f"    风险评级：{performance['风险等级']}")

```

```

581         optimal_timepoints.append({
582             '组别': f"组 {i + 1}",
583             'BMI区间': f"[{bmi_min:.2f}, {bmi_max:.2f}]",
584             '样本数': len(cluster_data),
585             '最佳时点_天数': optimal_time,
586             '最佳时点_周数': optimal_time / 7,
587             **performance
588         })
589         # 存储优化详细信息，用于后续可视化
590         optimization_details.append({
591             'group_id': i + 1,
592             'bmi_range': f"[{bmi_min:.2f}, {bmi_max:.2f}]",
593             'time_points': time_points,
594             'objective_values': objective_values,
595             'success_rates': success_rates,
596             'time_risks': time_risks,
597             'failure_risks': failure_risks,
598             'optimal_time': optimal_time,
599             'detection_times': detection_times.values
600         })
601         self.optimal_timepoints = pd.DataFrame(optimal_timepoints)
602         self.optimization_details = optimization_details # 保存优化详情
603         print("\n检测时点优化结果汇总:")
604         print(self.optimal_timepoints[['组别', 'BMI区间', '样本数', '最佳时点_周数',
605             '预期成功率', '风险等级']].round(2))
606         # 对时点优化过程绘制，可视化
607         self._visualize_timepoint_optimization()
608         return self.optimal_timepoints
609
610     def _visualize_timepoint_optimization(self):
611         if not hasattr(self, 'optimization_details') or not self.
612             optimization_details:
613             return
614         print(" 生成时点优化可视化图表...")
615         n_groups = len(self.optimization_details)
616         fig = plt.figure(figsize=(20, 4 * n_groups))
617         for i, detail in enumerate(self.optimization_details):
618             # 对每组创建4个子图
619             base_idx = i * 4
620             # 1. 目标函数曲线
621             ax1 = plt.subplot(n_groups, 4, base_idx + 1)
622             time_weeks = np.array(detail['time_points']) / 7
623             ax1.plot(time_weeks, detail['objective_values'], 'b-', linewidth=2,
624                 label='目标函数')
625             ax1.axvline(detail['optimal_time']/7, color='red', linestyle='--',
626                 linewidth=2,
627                 label=f'最优时点: {detail["optimal_time"]/7:.1f}周')

```

```

624         ax1.set_xlabel('检测时点 (周)')
625         ax1.set_ylabel('目标函数值')
626         ax1.set_title(f'组{detail["group_id"]} - 目标函数优化曲线', fontweight=
'bold')
627         ax1.grid(True, alpha=0.3)
628         ax1.legend()
629         #2. 成功率曲线
630         ax2 = plt.subplot(n_groups, 4, base_idx + 2)
631         ax2.plot(time_weeks, detail['success_rates'], 'g-', linewidth=2, label=
'成功率')
632         ax2.axvline(detail['optimal_time']/7, color='red', linestyle='--',
linewidth=2, alpha=0.7)
633         ax2.set_xlabel('检测时点 (周)')
634         ax2.set_ylabel('预期成功率')
635         ax2.set_title(f'组{detail["group_id"]} - 成功率曲线', fontweight='bold'
)
636         ax2.grid(True, alpha=0.3)
637         ax2.set_ylim([0, 1])
638         ax2.legend()
639         #3. 风险分解
640         ax3 = plt.subplot(n_groups, 4, base_idx + 3)
641         ax3.plot(time_weeks, detail['time_risks'], 'orange', linewidth=2, label
='时间风险')
642         ax3.plot(time_weeks, detail['failure_risks'], 'purple', linewidth=2,
label='失败风险')
643         ax3.axvline(detail['optimal_time']/7, color='red', linestyle='--',
linewidth=2, alpha=0.7)
644         ax3.set_xlabel('检测时点 (周)')
645         ax3.set_ylabel('风险值')
646         ax3.set_title(f'组{detail["group_id"]} - 风险分解', fontweight='bold')
647         ax3.grid(True, alpha=0.3)
648         ax3.legend()
649         #4. 达标时间分布直方图
650         ax4 = plt.subplot(n_groups, 4, base_idx + 4)
651         detection_weeks = detail['detection_times'] / 7
652         ax4.hist(detection_weeks, bins=15, alpha=0.7, color='lightblue',
edgecolor='black')
653         ax4.axvline(detail['optimal_time']/7, color='red', linestyle='--',
linewidth=3,
654                     label=f'最优时点: {detail["optimal_time"]/7:.1f}周')
655         ax4.set_xlabel('达标时间 (周)')
656         ax4.set_ylabel('样本数')
657         ax4.set_title(f'组{detail["group_id"]} - 达标时间分布', fontweight='
bold')
658         ax4.grid(True, alpha=0.3)
659         ax4.legend()
660         plt.tight_layout()

```

```

661     #输出生成的图片，保存在代码路径中
662     filename = '时点优化过程可视化.png'
663     plt.savefig(filename, dpi=300, bbox_inches='tight')
664     print(f"  时点优化过程图表已保存为: {filename}")
665     plt.show()
666     #创建新的图片，用于优化结果综合对比
667     self._create_optimization_summary_plot()
668     def _create_optimization_summary_plot(self):
669         if not hasattr(self, 'optimal_timepoints'):
670             return
671         print("  生成优化结果综合对比图表...")
672         fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(16, 12))
673         #1.各组最佳时点对比
674         groups = self.optimal_timepoints['组别']
675         timepoints_weeks = self.optimal_timepoints['最佳时点_周数']
676         success_rates = self.optimal_timepoints['预期成功率']
677         bars1 = ax1.bar(groups, timepoints_weeks, alpha=0.7, color='steelblue')
678         #在生成的图片里标注成功率
679         for i, (bar, rate) in enumerate(zip(bars1, success_rates)):
680             height = bar.get_height()
681             ax1.text(bar.get_x() + bar.get_width()/2., height + 0.3,
682                     f'{rate:.1%}', ha='center', va='bottom', fontweight='bold')
683         ax1.set_xlabel('BMI组别')
684         ax1.set_ylabel('最佳检测时点 (周)')
685         ax1.set_title('各组最佳检测时点对比', fontweight='bold', fontsize=14)
686         ax1.grid(True, alpha=0.3)
687         #2.成功率vs时点散点图
688         colors = ['red' if level == '高风险' else 'orange' if level == '中风险'
689                  else 'green']
690         for level in self.optimal_timepoints['风险等级']
691         scatter = ax2.scatter(timepoints_weeks, success_rates, s=100, c=colors,
692                               alpha=0.7, edgecolors='black')
693         for i, group in enumerate(groups):
694             ax2.annotate(group, (timepoints_weeks.iloc[i], success_rates.iloc[i]),
695                           xytext=(5, 5), textcoords='offset points', fontsize=10)
696         ax2.set_xlabel('最佳检测时点 (周)')
697         ax2.set_ylabel('预期成功率')
698         ax2.set_title('时点-成功率关系图', fontweight='bold', fontsize=14)
699         ax2.grid(True, alpha=0.3)
700         #添加一些图例
701         from matplotlib.patches import Patch
702         legend_elements = [Patch(facecolor='green', label='低风险'),
703                             Patch(facecolor='orange', label='中风险'),
704                             Patch(facecolor='red', label='高风险')]
705         ax2.legend(handles=legend_elements)
706         #3.绘制风险等级分布图
707         risk_counts = self.optimal_timepoints['风险等级'].value_counts()

```



```

706     colors_pie = ['green' if risk == '低风险' else 'orange' if risk == '中风险'
707                  else 'red']
708     for risk in risk_counts.index]
709     wedges, texts, autotexts = ax3.pie(risk_counts.values, labels=risk_counts.
index,
710                                         autopct='%1.1f%%', colors=colors_pie,
startangle=90)
711     ax3.set_title('各组风险等级分布', fontweight='bold', fontsize=14)
712     #4.绘制详细信息表格
713     ax4.axis('tight')
714     ax4.axis('off')
715     table_data = self.optimal_timepoints[['组别', 'BMI区间', '样本数', '最佳时
点_周数', '预期成功率', '风险等级']].copy()
716     table_data['预期成功率'] = table_data['预期成功率'].apply(lambda x: f'{x
:.1%}')
717     table_data['最佳时点_周数'] = table_data['最佳时点_周数'].apply(lambda x: f
'{x:.1f}')
718     table = ax4.table(cellText=table_data.values, colLabels=table_data.columns,
719                      cellLoc='center', loc='center', fontsize=10)
720     table.auto_set_font_size(False)
721     table.set_fontsize(9)
722     table.scale(1.2, 1.8)
723     #对输出的表格进行美化
724     for i in range(len(table_data.columns)):
725         table[(0, i)].set_facecolor('#4CAF50')
726         table[(0, i)].set_text_props(weight='bold', color='white')
727     for i in range(1, len(table_data) + 1):
728         for j in range(len(table_data.columns)):
729             table[(i, j)].set_facecolor('#f9f9f9')
730     ax4.set_title('检测时点优化结果汇总', fontweight='bold', fontsize=14, pad
=20)
731     plt.tight_layout()
732     #输出生成的图片，保存在代码路径中
733     filename = '优化结果综合对比.png'
734     plt.savefig(filename, dpi=300, bbox_inches='tight')
735     print(f" 优化结果综合对比图表已保存为: {filename}")
736     plt.show()
737
738     def _evaluate_timepoint(self, group_data, test_time):
739         success_rate = (group_data['最早达标时间_天数'] <= test_time).mean()
740         risk_level = '低风险' if test_time <= 12*7 else '中风险' if test_time <=
27*7 else '高风险'
741         return {'预期成功率': success_rate, '风险等级': risk_level}
742
743     def sensitivity_analysis(self):
744         print("\n5. 检测误差敏感性分析")

```

```

745     #增强代码的可解释性，以解释敏感性分析的意义
746     print("敏感性分析说明:")
747     print(" - 目的：评估最佳时点在实操中的稳定性")
748     print(" - 方法：测试±7天误差范围内成功率的变化")
749     print(" - 意义：误差范围越小，时点选择越稳定可靠")
750     error_scenarios = [-7, -3, 0, 3, 7]
751     sensitivity_results = []
752     for _, group_info in self.optimal_timepoints.iterrows():
753         bmi_range_str = group_info['BMI区间']
754         bmi_min, bmi_max = [float(x) for x in bmi_range_str.strip('[]').split(
, ')]

755         #同样，需要注意最后一个区间的边界
756         if group_info['组别'] == f"组 {len(self.fixed_bmi_groups)}":
757             cluster_data = self.women_data[
758                 (self.women_data['孕妇BMI'] >= bmi_min) & (self.women_data['孕
妇BMI'] <= bmi_max)
759             ]
760         else:
761             cluster_data = self.women_data[
762                 (self.women_data['孕妇BMI'] >= bmi_min) & (self.women_data['孕
妇BMI'] < bmi_max)
763             ]
764         if cluster_data.empty: continue
765         optimal_time = group_info['最佳时点_天数']
766         group_name = group_info['组别']
767         print(f"\n {group_name} 敏感性测试:")
768         group_sensitivity = []
769         for error in error_scenarios:
770             adjusted_time = optimal_time + error
771             performance = self._evaluate_timepoint(cluster_data, adjusted_time)
772             success_rate = performance['预期成功率']
773             sensitivity_results.append({
774                 '组别': group_name, '误差天数': error,
775                 '调整后时点_周数': adjusted_time / 7, '预期成功率':
success_rate
776             })
777             group_sensitivity.append(success_rate)
778             #增强代码可解释性，显示每个误差场景的详细信息
779             print(f"    误差{error:+2d}天 -> 时点{adjusted_time/7:.1f}周 -> 成
功率{success_rate:.1%}")

780
781         #计算，并且，解释该组的敏感性
782         max_rate, min_rate = max(group_sensitivity), min(group_sensitivity)
783         deviation = max_rate - min_rate
784         print(f"    敏感性评估：成功率波动范围 {deviation:.2%}")
785         self.sensitivity_df = pd.DataFrame(sensitivity_results)
786         print("\n不同误差下的预期成功率变化:")

```

```

787     #将透视表结果保存为类属性
788     self.sensitivity_pivot_df = self.sensitivity_df.pivot_table(
789         index='组别', columns='误差天数', values='预期成功率'
790     )
791     print(self.sensitivity_pivot_df.round(3))
792     print("\n各组敏感性等级评估:")
793     for group_name in sorted(self.sensitivity_pivot_df.index):
794         success_rates = self.sensitivity_pivot_df.loc[group_name]
795         deviation = success_rates.max() - success_rates.min()
796         level = "低敏感" if deviation < 0.1 else "中敏感" if deviation < 0.2
else "高敏感"
797         print(f" {group_name}: 成功率波动范围 {deviation:.2%} - {level}")
798         #增强代码的可解释性, 解释敏感性等级的含义
799         if level == "低敏感":
800             interpretation = "时点选择非常稳定, 实际操作中的小幅偏差影响很小"
801         elif level == "中敏感":
802             interpretation = "时点选择较为稳定, 但需要注意控制操作误差"
803         else:
804             interpretation = "时点选择敏感度较高, 需要精确控制检测时机"
805         print(f"          解释: {interpretation}")
806     #输出敏感性分析结果, 用于可视化
807     self._visualize_sensitivity_analysis()
808     return self.sensitivity_df
809
810 def _visualize_sensitivity_analysis(self):
811     if not hasattr(self, 'sensitivity_pivot_df'):
812         return
813     print(" 生成敏感性分析可视化图表...")
814     fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(16, 12))
815     #1.绘制敏感性热力图
816     sns.heatmap(self.sensitivity_pivot_df, annot=True, fmt='.3f', cmap='RdYlGn'
,
817                 center=self.sensitivity_pivot_df.mean().mean(), ax=ax1)
818     ax1.set_title('各组敏感性分析热力图', fontweight='bold', fontsize=14)
819     ax1.set_xlabel('误差天数')
820     ax1.set_ylabel('BMI组别')
821     #2.绘制各组敏感性曲线图
822     colors = plt.cm.viridis(np.linspace(0, 1, len(self.sensitivity_pivot_df.
index)))
823     for i, group in enumerate(self.sensitivity_pivot_df.index):
824         ax2.plot(self.sensitivity_pivot_df.columns, self.sensitivity_pivot_df.
loc[group],
825                 'o-', linewidth=2, markersize=6, color=colors[i], label=group)
826     ax2.set_xlabel('误差天数')
827     ax2.set_ylabel('预期成功率')
828     ax2.set_title('各组敏感性曲线', fontweight='bold', fontsize=14)
829     ax2.grid(True, alpha=0.3)

```

```

830     ax2.legend()
831     ax2.set_ylim([0, 1])
832     #3. 绘制敏感性等级分布
833     deviations = []
834     sensitivity_levels = []
835     group_names = []
836     for group_name in self.sensitivity_pivot_df.index:
837         success_rates = self.sensitivity_pivot_df.loc[group_name]
838         deviation = success_rates.max() - success_rates.min()
839         level = "低敏感" if deviation < 0.1 else "中敏感" if deviation < 0.2
else "高敏感"
840         deviations.append(deviation)
841         sensitivity_levels.append(level)
842         group_names.append(group_name)
843     level_counts = pd.Series(sensitivity_levels).value_counts()
844     colors_pie = ['green' if level == '低敏感' else 'orange' if level == '中敏感' else 'red']
845         for level in level_counts.index]
846     wedges, texts, autotexts = ax3.pie(level_counts.values, labels=level_counts.index,
847                                         autopct='%1.1f%%', colors=colors_pie,
startangle=90)
848     ax3.set_title('敏感性等级分布', fontweight='bold', fontsize=14)
849     #4. 绘制敏感性偏差条形图
850     colors_bar = ['green' if level == '低敏感' else 'orange' if level == '中敏感' else 'red']
851         for level in sensitivity_levels]
852     bars = ax4.bar(group_names, deviations, color=colors_bar, alpha=0.7,
edgecolor='black')
853     ax4.set_xlabel('BMI组别')
854     ax4.set_ylabel('成功率波动范围')
855     ax4.set_title('各组敏感性偏差对比', fontweight='bold', fontsize=14)
856     ax4.grid(True, alpha=0.3)
857     #在绘制的图形上添加阈值线
858     ax4.axhline(y=0.1, color='orange', linestyle='--', alpha=0.7, label='中敏感阈值(10%)')
859     ax4.axhline(y=0.2, color='red', linestyle='--', alpha=0.7, label='高敏感阈值(20%)')
860     ax4.legend()
861     #在会知道图形上标注数值
862     for bar, deviation, level in zip(bars, deviations, sensitivity_levels):
863         height = bar.get_height()
864         ax4.text(bar.get_x() + bar.get_width()/2., height + 0.005,
865                 f'{deviation:.2%}\n{level}',
866                 ha='center', va='bottom', fontsize=9, fontweight='bold')
867     plt.tight_layout()
868     #输出生成的图片，保存在代码路径中

```

```

869     filename = '敏感性分析可视化.png'
870     plt.savefig(filename, dpi=300, bbox_inches='tight')
871     print(f" 敏感性分析图表已保存为: {filename}")
872     plt.show()
873
874 #结果输出功能
875 def save_results_to_excel(self, filename="综合分析结果.xlsx"):
876     print(f"6. 保存分析结果至Excel文件: {filename}")
877     try:
878         with pd.ExcelWriter(filename, engine='openpyxl') as writer:
879             #1.写入特征重要性
880             if hasattr(self, 'feature_importance') and self.feature_importance
is not None:
881                 self.feature_importance.to_excel(writer, sheet_name='特征重要性
排序', index=False)
882                 print(f"  - 已保存 '特征重要性排序' sheet。")
883             #2.写入聚类方法对比
884             if hasattr(self, 'clustering_comparison_df'):
885                 self.clustering_comparison_df.to_excel(writer, sheet_name='聚类
方法对比', index=False)
886                 print(f"  - 已保存 '聚类方法对比' sheet。")
887             #3.写入最终BMI分组区间
888             if hasattr(self, 'fixed_bmi_groups'):
889                 bmi_groups_df = pd.DataFrame(self.fixed_bmi_groups, columns=['
BMI下限', 'BMI上限'])
890                 bmi_groups_df.index = [f"组 {i+1}" for i in bmi_groups_df.index
]
891                 bmi_groups_df.to_excel(writer, sheet_name='最终BMI分组')
892                 print(f"  - 已保存 '最终BMI分组' sheet。")
893             #4.写入各组最佳检测时点
894             if hasattr(self, 'optimal_timepoints'):
895                 self.optimal_timepoints.to_excel(writer, sheet_name='各组最佳检
测时点', index=False)
896                 print(f"  - 已保存 '各组最佳检测时点' sheet。")
897             #5.写入时点敏感性分析
898             if hasattr(self, 'sensitivity_pivot_df'):
899                 self.sensitivity_pivot_df.to_excel(writer, sheet_name='时点敏感
性分析')
900                 print(f"  - 已保存 '时点敏感性分析' sheet。")
901             #6.写入带标签的完整数据
902             if 'cluster' in self.women_data.columns:
903                 self.women_data.to_excel(writer, sheet_name='带标签的聚合数据',
index=False)
904                 print(f"  - 已保存 '带标签的聚合数据' sheet。")
905                 print(f"\n所有结果已成功保存到 '{filename}'!")
906             except Exception as e:
907                 print(f"\n保存Excel文件时出错: {e}")

```

```

908         print("请确保您有写入文件的权限，并且没有其他程序正在打开该文件。")
909
910     def run_complete_analysis(self):
911         try:
912             print(f"\n开始运行问题三综合深度分析...")
913             print(f"分析阶段概览:")
914             print(f"  数据预处理和特征工程")
915             print(f"  特征重要性分析 + 可视化")
916             print(f"  多因素聚类分组 + 可视化")
917             print(f"  检测时点优化 + 可视化")
918             print(f"  敏感性分析 + 可视化")
919             print(f"  结果保存与综合汇总")
920             #运行并分析流程
921             self.analyze_feature_importance()
922             self.multi_factor_clustering()
923             self.optimize_detection_timepoints()
924             self.sensitivity_analysis()
925             #生成所有的综合分析汇总
926             self._create_comprehensive_summary()
927             #在所有分析结束后，调用保存结果的方法
928             self.save_results_to_excel() # 使用默认文件名 "综合分析结果.xlsx"
929             print("问题三深度分析全部流程成功完成!")
930             print(f"  生成的可视化文件:")
931             print(f"    - 特征重要性分析可视化.png")
932             print(f"    - 聚类过程综合分析可视化.png")
933             print(f"    - BMI分组结果可视化.png")
934             print(f"    - 时点优化过程可视化.png")
935             print(f"    - 优化结果综合对比.png")
936             print(f"    - 敏感性分析可视化.png")
937             print(f"    - 综合分析汇总Dashboard.png")
938             print(f"  分析报告: 综合分析结果.xlsx")
939             return True
940         except Exception as e:
941             print(f"分析过程中出错: {e}")
942             import traceback
943             traceback.print_exc()
944             return False
945
946     def _create_comprehensive_summary(self):
947         print(f"\n  生成综合分析汇总Dashboard...")
948         fig = plt.figure(figsize=(20, 16))
949         #创建新的主标题
950         fig.suptitle('问题三综合深度分析 - 多因素BMI分组与NIPT时点优化',
951                     fontsize=20, fontweight='bold', y=0.98)
952         #1.绘制特征重要性Top5
953         ax1 = plt.subplot(3, 4, 1)
954         if hasattr(self, 'feature_importance') and self.feature_importance is not

```

None:

```
955     top5_features = self.feature_importance.head(5)
956     bars = ax1.barh(range(len(top5_features)), top5_features['重要性'],
957                    color='skyblue', alpha=0.8)
958     ax1.set_yticks(range(len(top5_features)))
959     ax1.set_yticklabels(top5_features['特征'], fontsize=10)
960     ax1.set_xlabel('重要性')
961     ax1.set_title('Top5 关键特征', fontweight='bold')
962     ax1.grid(True, alpha=0.3)
963     #在图上标注数值
964     for i, (bar, importance) in enumerate(zip(bars, top5_features['重要性'
965 ]))):
966         width = bar.get_width()
967         ax1.text(width + 0.005, bar.get_y() + bar.get_height()/2,
968                f'{importance:.3f}', ha='left', va='center', fontsize=9)
969 #2. 绘制聚类方法对比
970 ax2 = plt.subplot(3, 4, 2)
971 if hasattr(self, 'clustering_comparison_df'):
972     methods = self.clustering_comparison_df['方法']
973     scores = self.clustering_comparison_df['评估得分(轮廓系数)']
974     colors = ['gold' if i == 0 else 'lightblue' for i in range(len(methods)
975 )]
976     bars = ax2.bar(methods, scores, color=colors, alpha=0.8, edgecolor='
977 black')
978     ax2.set_ylabel('轮廓系数')
979     ax2.set_title('聚类方法对比', fontweight='bold')
980     ax2.grid(True, alpha=0.3)
981     ax2.tick_params(axis='x', rotation=45)
982     #综合所有分析后，并标注最佳方法
983     for i, (bar, score) in enumerate(zip(bars, scores)):
984         height = bar.get_height()
985         label = f'{score:.3f}\n最佳' if i == 0 else f'{score:.3f}'
986         ax2.text(bar.get_x() + bar.get_width()/2., height + 0.01,
987                label, ha='center', va='bottom', fontsize=9, fontweight='
988 bold')
989 #3. 绘制BMI分组区间展示
990 ax3 = plt.subplot(3, 4, 3)
991 if hasattr(self, 'fixed_bmi_groups'):
992     #绘制BMI分布直方图
993     ax3.hist(self.women_data['孕妇BMI'], bins=30, alpha=0.6, color='
994 lightblue', edgecolor='black')
995     #标记分组边界
996     colors = ['red', 'orange', 'green', 'purple', 'brown']
997     for i, (bmi_min, bmi_max) in enumerate(self.fixed_bmi_groups):
998         color = colors[i % len(colors)]
999         ax3.axvline(bmi_min, color=color, linestyle='--', linewidth=2,
1000 alpha=0.8)
```

```

995         if i == len(self.fixed_bmi_groups) - 1:
996             ax3.axvline(bmi_max, color=color, linestyle='--', linewidth=2,
alpha=0.8)
997             #添加组别标签
998             mid_point = (bmi_min + bmi_max) / 2
999             ax3.text(mid_point, ax3.get_ylim()[1] * 0.8, f'组{i+1}',
1000                     ha='center', va='center', fontsize=10, color=color,
fontweight='bold')
1001             ax3.set_xlabel('孕妇BMI')
1002             ax3.set_ylabel('频次')
1003             ax3.set_title('BMI分组结果', fontweight='bold')
1004             #4.绘制最佳时点对比图
1005             ax4 = plt.subplot(3, 4, 4)
1006             if hasattr(self, 'optimal_timepoints'):
1007                 groups = self.optimal_timepoints['组别']
1008                 timepoints = self.optimal_timepoints['最佳时点_周数']
1009                 success_rates = self.optimal_timepoints['预期成功率']
1010                 bars = ax4.bar(groups, timepoints, color='lightgreen', alpha=0.8,
edgecolor='black')
1011                 ax4.set_ylabel('最佳时点 (周)')
1012                 ax4.set_title('各组最佳检测时点', fontweight='bold')
1013                 ax4.grid(True, alpha=0.3)
1014                 #在绘制的图形中标注成功率
1015                 for bar, rate in zip(bars, success_rates):
1016                     height = bar.get_height()
1017                     ax4.text(bar.get_x() + bar.get_width()/2., height + 0.3,
1018                             f'{rate:.1%}', ha='center', va='bottom', fontsize=9,
fontweight='bold')
1019             #5.绘制敏感性分析热力图
1020             ax5 = plt.subplot(3, 4, 5)
1021             if hasattr(self, 'sensitivity_pivot_df'):
1022                 sns.heatmap(self.sensitivity_pivot_df, annot=True, fmt='.2f',
1023                             cmap='RdYlGn', center=0.9, ax=ax5, cbar_kws={'shrink': 0.8})
1024             ax5.set_title('敏感性分析热力图', fontweight='bold')
1025             ax5.set_xlabel('误差天数')
1026             ax5.set_ylabel('组别')
1027             #6.绘制风险等级分布图
1028             ax6 = plt.subplot(3, 4, 6)
1029             if hasattr(self, 'optimal_timepoints'):
1030                 risk_counts = self.optimal_timepoints['风险等级'].value_counts()
1031                 colors_pie = ['green' if risk == '低风险' else 'orange' if risk == '中
风险' else 'red'
1032                                for risk in risk_counts.index]
1033                 wedges, texts, autotexts = ax6.pie(risk_counts.values, labels=
risk_counts.index,
1034                                                     autopct='%1.1f%%', colors=colors_pie,
startangle=90)

```



```

1035         ax6.set_title('风险等级分布', fontweight='bold')
1036     #7.绘制数据概览信息表
1037     ax7 = plt.subplot(3, 4, (7, 8))
1038     ax7.axis('tight')
1039     ax7.axis('off')
1040     #创建新的数据概览表
1041     overview_data = []
1042     if hasattr(self, 'women_data'):
1043         overview_data.append(['总样本数', f'{len(self.women_data)}'])
1044         overview_data.append(['BMI范围', f'{self.women_data["孕妇BMI"].min():.1f} - {self.women_data["孕妇BMI"].max():.1f}'])
1045         overview_data.append(['年龄范围', f'{self.women_data["年龄"].min():.0f} - {self.women_data["年龄"].max():.0f}岁'])
1046     if hasattr(self, 'fixed_bmi_groups'):
1047         overview_data.append(['分组数量', f'{len(self.fixed_bmi_groups)}组'])
1048     if hasattr(self, 'optimal_timepoints'):
1049         avg_timepoint = self.optimal_timepoints['最佳时点_周数'].mean()
1050         avg_success = self.optimal_timepoints['预期成功率'].mean()
1051         overview_data.append(['平均最佳时点', f'{avg_timepoint:.1f}周'])
1052         overview_data.append(['平均成功率', f'{avg_success:.1%}'])
1053     if hasattr(self, 'feature_importance'):
1054         top_feature = self.feature_importance.iloc[0]['特征']
1055         top_importance = self.feature_importance.iloc[0]['重要性']
1056         overview_data.append(['最重要特征', f'{top_feature}'])
1057         overview_data.append(['特征重要性', f'{top_importance:.3f}'])
1058     overview_df = pd.DataFrame(overview_data, columns=['指标', '数值'])
1059     table = ax7.table(cellText=overview_df.values, colLabels=overview_df.
columns,
1060                      cellLoc='center', loc='center', fontsize=12)
1061     table.auto_set_font_size(False)
1062     table.set_fontsize(11)
1063     table.scale(1.2, 2)
1064     #对生成的表格进行美化
1065     for i in range(len(overview_df.columns)):
1066         table[(0, i)].set_facecolor('#4CAF50')
1067         table[(0, i)].set_text_props(weight='bold', color='white')
1068     for i in range(1, len(overview_df) + 1):
1069         for j in range(len(overview_df.columns)):
1070             table[(i, j)].set_facecolor('#f0f8ff')
1071     ax7.set_title('数据概览', fontweight='bold', fontsize=14, pad=20)
1072     #8.绘制关键结论与建议图
1073     ax8 = plt.subplot(3, 4, (9, 10))
1074     ax8.axis('off')
1075     conclusions = []
1076     if hasattr(self, 'optimal_timepoints'):
1077         best_group = self.optimal_timepoints.loc[self.optimal_timepoints['预期
成功率'].idxmax()]

```

```

1078     worst_group = self.optimal_timepoints.loc[self.optimal_timepoints['预期
1079     成功率'].idxmin()]
1080     conclusions.extend([
1081         "  关键发现:",
1082         f"• 最佳表现: {best_group['组别']} (成功率{best_group['预期成功率
1083         ']:.1%})",
1084         f"• 需要关注: {worst_group['组别']} (成功率{worst_group['预期成功率
1085         ']:.1%})",
1086         "",
1087         "  临床建议:",
1088         "• 实施个性化检测时点策略",
1089         "• 重点关注高BMI孕妇群体",
1090         "• 建立时点偏差控制机制",
1091         "",
1092         "  注意事项:",
1093         "• 考虑实际操作中的时间误差",
1094         "• 定期评估和调整分组策略",
1095         "• 监控检测质量指标变化"
1096     ])
1097     conclusion_text = '\n'.join(conclusions)
1098     ax8.text(0.05, 0.95, conclusion_text, transform=ax8.transAxes, fontsize=11,
1099             verticalalignment='top', horizontalalignment='left',
1100             bbox=dict(boxstyle="round,pad=0.5", facecolor='lightyellow', alpha
1101             =0.8))
1102     ax8.set_title('分析结论与建议', fontweight='bold', fontsize=14)
1103     #9. 绘制分析流程图示
1104     ax9 = plt.subplot(3, 4, (11, 12))
1105     ax9.axis('off')
1106     #绘制流程图
1107     steps = [
1108         "1. 数据预处理\n特征工程",
1109         "2. 特征重要性\n随机森林分析",
1110         "3. 多因素聚类\n3种方法对比",
1111         "4. BMI分组\n区间修正",
1112         "5. 时点优化\n目标函数最小化",
1113         "6. 敏感性分析\n稳定性评估"
1114     ]
1115     y_positions = np.linspace(0.9, 0.1, len(steps))
1116     colors_flow = ['lightblue', 'lightgreen', 'lightyellow', 'lightpink', '
1117     lightcoral', 'lightgray']
1118     for i, (step, y_pos, color) in enumerate(zip(steps, y_positions,
1119     colors_flow)):
1120         #绘制步骤框
1121         bbox = dict(boxstyle="round,pad=0.3", facecolor=color, alpha=0.8,
1122         edgecolor='black')
1123         ax9.text(0.5, y_pos, step, transform=ax9.transAxes, fontsize=10,
1124                 ha='center', va='center', bbox=bbox)

```

```

1118         #绘制箭头
1119         if i < len(steps) - 1:
1120             ax9.annotate('', xy=(0.5, y_positions[i+1] + 0.05), xytext=(0.5,
y_pos - 0.05),
1121                           xycoords='axes fraction', textcoords='axes fraction',
1122                           arrowprops=dict(arrowstyle='->', lw=2, color='darkblue')
)
1123         ax9.set_title('分析流程', fontweight='bold', fontsize=14)
1124         plt.tight_layout()
1125         filename = '综合分析汇总Dashboard.png'
1126         plt.savefig(filename, dpi=300, bbox_inches='tight')
1127         print(f" 综合分析汇总Dashboard已保存为: {filename}")
1128         plt.show()
1129 if __name__ == "__main__":
1130     analyzer = Problem3ComprehensiveAnalysis("问题2_建模数据.csv")
1131     analyzer.run_complete_analysis()

```

问题 4 数据预处理与可视化.py

```

1 import pandas as pd
2 import numpy as np
3 from datetime import datetime
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import warnings
7 warnings.filterwarnings('ignore')
8
9 # 设置中文字体
10 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'STSong']
11 plt.rcParams['axes.unicode_minus'] = False
12 plt.rcParams['font.size'] = 10
13 #女胎数据处理
14 def preprocess_female_data_simple(excel_file_path, enable_visualization=True):
15     df = pd.read_excel(excel_file_path, sheet_name='女胎检测数据')
16     processed_df = df.copy()
17     #末次月经日期+检测日期转化为新变量末次月经距离检测日的天数
18     def calc_days_diff(row):
19         try:
20             check_date_str = str(int(row['检测日期']))
21             check_date = datetime.strptime(check_date_str, '%Y%m%d')
22             if isinstance(row['末次月经'], (int, float)):
23                 last_period = pd.Timestamp('1899-12-30') + pd.Timedelta(days=row['
末次月经'])
24             else:
25                 last_period = pd.to_datetime(row['末次月经'])
26             # 计算天数差
27             return (check_date - last_period).days
28     except:

```

```

29         return np.nan
30
31     processed_df['末次月经距离检测日天数'] = processed_df.apply(calc_days_diff,
32 axis=1)
33     valid_days = processed_df['末次月经距离检测日天数'].dropna()
34     print(f"    计算完成，有效样本：{len(valid_days)}，范围：{valid_days.min():.0f}
35     {valid_days.max():.0f}天")
36
37     # 编码 IVF 妊娠状态
38     ivf_mapping = {'自然受孕': 0, 'IVF (试管婴儿)': 1}
39     processed_df['IVF 妊娠_编码'] = processed_df['IVF 妊娠'].map(ivf_mapping)
40
41     ivf_counts = processed_df['IVF 妊娠_编码'].value_counts()
42     print(f"    自然受孕：{ivf_counts.get(0, 0)} 例")
43     print(f"    IVF 试管：{ivf_counts.get(1, 0)} 例")
44
45     # 转换孕周为天数
46     def weeks_to_days(week_str):
47         try:
48             if pd.isna(week_str):
49                 return np.nan
50
51             week_str = str(week_str)
52             if 'w' in week_str:
53                 parts = week_str.replace('w', '').split('+')
54                 weeks = int(parts[0])
55                 days = int(parts[1]) if len(parts) > 1 and parts[1] else 0
56                 return weeks * 7 + days
57             else:
58                 return float(week_str) * 7
59         except:
60             return np.nan
61
62     processed_df['检测时怀孕天数'] = processed_df['检测孕周'].apply(weeks_to_days)
63     valid_preg_days = processed_df['检测时怀孕天数'].dropna()
64     print(f"    转换完成，范围：{valid_preg_days.min():.0f}-{valid_preg_days.max()
65     :.0f}天")
66
67     # 解析染色体异常
68     def parse_chromosome(abnormality):
69         if pd.isna(abnormality):
70             return {'T13': 0, 'T18': 0, 'T21': 0}
71
72         abnormality_str = str(abnormality)
73         return {
74             'T13': 1 if 'T13' in abnormality_str else 0,
75             'T18': 1 if 'T18' in abnormality_str else 0,

```

```

73         'T21': 1 if 'T21' in abnormality_str else 0
74     }
75
76     # 应用解析函数
77     chromosome_results = processed_df['染色体的非整倍体'].apply(parse_chromosome)
78     processed_df['T13_异常'] = [result['T13'] for result in chromosome_results]
79     processed_df['T18_异常'] = [result['T18'] for result in chromosome_results]
80     processed_df['T21_异常'] = [result['T21'] for result in chromosome_results]
81
82     # 统计染色体异常
83     t13_count = processed_df['T13_异常'].sum()
84     t18_count = processed_df['T18_异常'].sum()
85     t21_count = processed_df['T21_异常'].sum()
86     total_abnormal = (processed_df['T13_异常'] | processed_df['T18_异常'] |
87 processed_df['T21_异常']).sum()
88
89     print(f"    T13异常: {t13_count} 例")
90     print(f"    T18异常: {t18_count} 例")
91     print(f"    T21异常: {t21_count} 例")
92     print(f"    总计异常: {total_abnormal} 例, 正常: {len(processed_df) -
93 total_abnormal} 例")
94
95     #根据染色体异常编码胎儿健康状态
96     processed_df['胎儿健康_编码'] = np.where(
97         (processed_df['T13_异常'] == 0) &
98         (processed_df['T18_异常'] == 0) &
99         (processed_df['T21_异常'] == 0),
100         1, # 健康
101         0 # 不健康
102     )
103
104     health_counts = processed_df['胎儿健康_编码'].value_counts().sort_index()
105     print(f"    基于染色体异常的健康状态分布:")
106     for code, count in health_counts.items():
107         status = "健康" if code == 1 else "不健康"
108         print(f"        {status} (编码={code}): {count} 例")
109
110     # 验证逻辑的正确性
111     healthy_samples = processed_df[processed_df['胎儿健康_编码'] == 1]
112     unhealthy_samples = processed_df[processed_df['胎儿健康_编码'] == 0]
113
114     print(f"    验证:")
115     print(f"        健康样本中染色体异常数: T13={healthy_samples['T13_异常'].sum()},
116 T18={healthy_samples['T18_异常'].sum()}, T21={healthy_samples['T21_异常'].sum()}")
117     print(f"        不健康样本中染色体异常数: T13={unhealthy_samples['T13_异常'].sum()}
118 , T18={unhealthy_samples['T18_异常'].sum()}, T21={unhealthy_samples['T21_异常

```

```

115     '].sum()}")
116
117 #数据预处理可视化分析
118 if enable_visualization:
119     # 1. 数据概览可视化
120     create_female_data_overview_plots(processed_df)
121
122     # 2. 分类变量分布可视化
123     create_female_categorical_distribution_plots(processed_df)
124
125     # 3. 连续变量分布可视化
126     create_female_continuous_distribution_plots(processed_df)
127
128     # 4. 数据质量分析可视化
129     create_female_data_quality_plots(processed_df)
130
131     # 5. 相关性初步分析可视化
132     create_female_preliminary_correlation_plots(processed_df)
133
134     print("女胎数据预处理可视化分析完成！")
135
136 #选择最终特征
137 # 定义最终使用的特征列
138 final_features = [
139     # 基本信息
140     '序号', '孕妇代码', '年龄', '身高', '体重', '孕妇BMI',
141     '怀孕次数', '生产次数', '检测抽血次数',
142
143     # 预处理后的新特征
144     '末次月经距离检测日天数',
145     'IVF妊娠_编码',
146     '检测时怀孕天数',
147     'T13_异常', 'T18_异常', 'T21_异常',
148     '胎儿健康_编码',
149
150     # 生化检测特征
151     '原始读段数', '在参考基因组上比对的比例', '重复读段的比例',
152     '唯一比对的读段数', 'GC含量', '被过滤掉读段数的比例',
153
154     # 染色体Z值特征
155     '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值', 'X染色体的Z值',
156
157     # 染色体GC含量特征
158     '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC含量',
159     'X染色体浓度'
160 ]

```

```

161     available_features = [feat for feat in final_features if feat in processed_df.
162                            columns]
163
164     missing_features = [feat for feat in final_features if feat not in processed_df
165                          .columns]
166
167     if missing_features:
168         print(f"    缺失特征: {missing_features}")
169
170     # 创建最终数据集
171     final_df = processed_df[available_features].copy()
172
173     print(f"预处理完成!")
174     print(f"最终数据形状: {final_df.shape}")
175     print(f"包含特征: {len(available_features)} 个")
176
177     return final_df
178
179 def create_female_data_overview_plots(df):
180     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
181     fig.suptitle('女胎数据预处理概览', fontsize=16, fontweight='bold')
182
183     # 1. 样本量统计
184     sample_info = {
185         '总样本数': len(df),
186         '完整样本': len(df.dropna()),
187         '有效染色体数据': len(df.dropna(subset=['T13_异常', 'T18_异常', 'T21_异常'
188         ]))
189     }
190
191     bars1 = axes[0, 0].bar(sample_info.keys(), sample_info.values(),
192                            color=['skyblue', 'lightgreen', 'orange'], alpha=0.8)
193     axes[0, 0].set_title('女胎样本量统计')
194     axes[0, 0].set_ylabel('样本数')
195     for bar, value in zip(bars1, sample_info.values()):
196         axes[0, 0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
197                          f'{value}', ha='center', va='bottom', fontweight='bold')
198
199     # 2. 年龄分布
200     if '年龄' in df.columns:
201         age_data = df['年龄'].dropna()
202         axes[0, 1].hist(age_data, bins=20, color='lightcoral', alpha=0.7, edgecolor
203                         ='black')
204         axes[0, 1].axvline(age_data.mean(), color='red', linestyle='--', linewidth
205                             =2,
206                             label=f'均值: {age_data.mean():.1f}岁')
207         axes[0, 1].set_title('女胎孕妇年龄分布')
208         axes[0, 1].set_xlabel('年龄 (岁)')

```

```

203     axes[0, 1].set_ylabel('频数')
204     axes[0, 1].legend()
205     axes[0, 1].grid(True, alpha=0.3)
206
207 # 3. BMI分布
208 if '孕妇BMI' in df.columns:
209     bmi_data = df['孕妇BMI'].dropna()
210     axes[1, 0].hist(bmi_data, bins=20, color='lightblue', alpha=0.7, edgecolor=
211 'black')
212     axes[1, 0].axvline(18.5, color='blue', linestyle=':', label='偏瘦')
213     axes[1, 0].axvline(24, color='green', linestyle=':', label='正常')
214     axes[1, 0].axvline(28, color='orange', linestyle=':', label='超重')
215     axes[1, 0].axvline(bmi_data.mean(), color='red', linestyle='--', linewidth
216 =2,
217                        label=f'均值: {bmi_data.mean():.1f}')
218     axes[1, 0].set_title('女胎孕妇BMI分布')
219     axes[1, 0].set_xlabel('BMI')
220     axes[1, 0].set_ylabel('频数')
221     axes[1, 0].legend()
222     axes[1, 0].grid(True, alpha=0.3)
223
224 # 4. 胎儿健康状态分布
225 if '胎儿健康_编码' in df.columns:
226     health_counts = df['胎儿健康_编码'].value_counts()
227     labels = ['不健康' if x == 0 else '健康' for x in health_counts.index]
228     colors = ['lightcoral', 'lightgreen']
229
230     wedges, texts, autotexts = axes[1, 1].pie(health_counts.values, labels=
231 labels,
232                                           autopct='%1.1f%%', colors=colors,
233 startangle=90)
234     axes[1, 1].set_title('女胎健康状态分布')
235
236 # 添加数量标注
237 for i, (label, count) in enumerate(zip(labels, health_counts.values)):
238     axes[1, 1].text(0.8, 0.8-i*0.1, f'{label}: {count}例',
239                    transform=axes[1, 1].transAxes, fontsize=9,
240                    bbox=dict(boxstyle='round,pad=0.3', facecolor='white',
241 alpha=0.8))
242
243 plt.tight_layout()
244 plt.savefig('女胎数据预处理_概览分析.png', dpi=300, bbox_inches='tight',
245 facecolor='white')
246 plt.show()
247
248 def create_female_categorical_distribution_plots(df):
249     fig, axes = plt.subplots(2, 2, figsize=(15, 12))

```



```

244 fig.suptitle('女胎分类变量分布分析', fontsize=16, fontweight='bold')
245 axes = axes.flatten()
246
247 plot_idx = 0
248
249 # 1. IVF妊娠分布
250 if 'IVF妊娠_编码' in df.columns:
251     ivf_counts = df['IVF妊娠_编码'].value_counts(dropna=False)
252     ivf_labels = {0: '自然受孕', 1: 'IVF试管'}
253
254     colors = plt.cm.Set3(np.linspace(0, 1, len(ivf_counts)))
255     labels = [ivf_labels.get(k, f'编码{k}')] for k in ivf_counts.index]
256
257     wedges, texts, autotexts = axes[plot_idx].pie(ivf_counts.values, labels=
labels,
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

288     axes[plot_idx].set_xticklabels(chromosomes[:len(available_chromosomes)])
289     axes[plot_idx].legend()
290
291     # 添加数值标签
292     for i, (bar1, bar2) in enumerate(zip(bars1, bars2)):
293         if abnormal_counts[i] > 0:
294             axes[plot_idx].text(bar1.get_x() + bar1.get_width()/2,
295                                 bar1.get_height()/2, f'{abnormal_counts[i]}',
296                                 ha='center', va='center', fontweight='bold',
297                                 color='white')
298             axes[plot_idx].text(bar2.get_x() + bar2.get_width()/2,
299                                 abnormal_counts[i] + bar2.get_height()/2, f'{
300 normal_counts[i]}',
301                                 ha='center', va='center', fontweight='bold', color='
302 white')
303
304     plot_idx += 1
305
306     # 3. 怀孕次数分布
307     if '怀孕次数' in df.columns:
308         preg_counts = df['怀孕次数'].value_counts(dropna=False)
309
310         colors = plt.cm.Set3(np.linspace(0, 1, len(preg_counts)))
311         wedges, texts, autotexts = axes[plot_idx].pie(preg_counts.values,
312                                                         labels=[f'{k}次' if not pd.
313 isna(k) else '缺失' for k in preg_counts.index],
314                                                         autopct='%1.1f%%', colors=
315 colors, startangle=90)
316         axes[plot_idx].set_title('女胎怀孕次数分布')
317         plot_idx += 1
318
319     # 4. 健康编码vs原始健康状态对比
320     if '胎儿健康_编码' in df.columns and '胎儿是否健康' in df.columns:
321         # 创建对比分析
322         comparison_data = df[['胎儿是否健康', '胎儿健康_编码']].dropna()
323         if len(comparison_data) > 0:
324             # 统计不一致的情况
325             encoded_healthy = (comparison_data['胎儿健康_编码'] == 1).sum()
326             encoded_unhealthy = (comparison_data['胎儿健康_编码'] == 0).sum()
327
328             bars = axes[plot_idx].bar(['基于染色体的健康', '基于染色体的不健康'],
329                                       [encoded_healthy, encoded_unhealthy],
330                                       color=['lightgreen', 'lightcoral'], alpha=0.8)
331
332             axes[plot_idx].set_title('基于染色体异常的健康状态编码')
333             axes[plot_idx].set_ylabel('样本数')
334

```

```

330         # 添加数量标签
331         for bar, count in zip(bars, [encoded_healthy, encoded_unhealthy]):
332             axes[plot_idx].text(bar.get_x() + bar.get_width()/2, bar.get_height
333                                 () + 5,
334                                 f'{{count}}', ha='center', va='bottom', fontweight
335                                 ='bold')
336
337         plot_idx += 1
338
339     # 隐藏多余的子图
340     for i in range(plot_idx, 4):
341         axes[i].set_visible(False)
342
343     plt.tight_layout()
344     plt.savefig('女胎数据预处理_分类变量分布.png', dpi=300, bbox_inches='tight',
345                 facecolor='white')
346     plt.show()
347
348 def create_female_continuous_distribution_plots(df):
349
350     # 定义女胎相关的连续变量
351     continuous_vars = [
352         '年龄', '身高', '体重', '孕妇BMI',
353         '末次月经距离检测日天数', '检测时怀孕天数',
354         '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
355         'X染色体的Z值', '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC
356         含量',
357         'X染色体浓度', 'GC含量'
358     ]
359
360     available_vars = [var for var in continuous_vars if var in df.columns]
361
362     n_vars = len(available_vars)
363     n_cols = 3
364     n_rows = (n_vars + n_cols - 1) // n_cols
365
366     fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 5*n_rows))
367     fig.suptitle('女胎连续变量分布分析', fontsize=16, fontweight='bold')
368
369     if n_rows == 1:
370         axes = axes.reshape(1, -1) if n_vars > 1 else [axes]
371         axes = axes.flatten() if n_vars > 1 else [axes]
372
373     for i, var in enumerate(available_vars):
374         if i >= len(axes):
375             break

```

```

373     data = df[var].dropna()
374
375     if len(data) == 0:
376         axes[i].text(0.5, 0.5, f'{var}\n无数据', ha='center', va='center',
377                     transform=axes[i].transAxes, fontsize=12)
378         axes[i].set_title(var)
379         continue
380
381     # 直方图 + 核密度估计
382     axes[i].hist(data, bins=30, density=True, alpha=0.7, color='skyblue',
383                 edgecolor='black')
384
385     # 核密度估计
386     try:
387         from scipy.stats import gaussian_kde
388         kde = gaussian_kde(data)
389         x_range = np.linspace(data.min(), data.max(), 100)
390         axes[i].plot(x_range, kde(x_range), 'r-', linewidth=2, label='KDE')
391     except:
392         pass
393
394     # 添加统计信息
395     mean_val = data.mean()
396     median_val = data.median()
397     axes[i].axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f
398                    '均值: {mean_val:.3f}')
399     axes[i].axvline(median_val, color='orange', linestyle='--', linewidth=2,
400                    label=f'中位数: {median_val:.3f}')
401
402     axes[i].set_title(var)
403     axes[i].set_ylabel('密度')
404     axes[i].legend()
405     axes[i].grid(True, alpha=0.3)
406
407     # 隐藏多余的子图
408     for j in range(len(available_vars), len(axes)):
409         axes[j].set_visible(False)
410
411     plt.tight_layout()
412     plt.savefig('女胎数据预处理_连续变量分布.png', dpi=300, bbox_inches='tight',
413               facecolor='white')
414     plt.show()
415
416 def create_female_data_quality_plots(df):
417
418     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
419     fig.suptitle('女胎数据质量分析', fontsize=16, fontweight='bold')

```

```

416
417 # 1. 缺失值分析
418 key_columns = ['年龄', '孕妇BMI', '末次月经距离检测日天数', 'IVF妊娠_编码',
419               '检测时怀孕天数', 'T13_异常', 'T18_异常', 'T21_异常', '胎儿健康_
编码']
420 available_key_cols = [col for col in key_columns if col in df.columns]
421
422 if available_key_cols:
423     missing_counts = []
424     col_names = []
425     for col in available_key_cols:
426         missing_count = df[col].isnull().sum()
427         if missing_count > 0: # 只显示有缺失值的列
428             missing_counts.append(missing_count)
429             col_names.append(col)
430
431     if missing_counts:
432         bars = axes[0, 0].barh(range(len(col_names)), missing_counts, color='
red', alpha=0.7)
433         axes[0, 0].set_yticks(range(len(col_names)))
434         axes[0, 0].set_yticklabels(col_names)
435         axes[0, 0].set_xlabel('缺失值数量')
436         axes[0, 0].set_title('关键字段缺失值统计')
437         axes[0, 0].grid(True, alpha=0.3, axis='x')
438
439         # 添加数值标签
440         for i, (bar, value) in enumerate(zip(bars, missing_counts)):
441             axes[0, 0].text(bar.get_width() + 5, i, f'{value}',
442                             va='center', fontweight='bold')
443     else:
444         axes[0, 0].text(0.5, 0.5, '□ 关键字段都没有缺失值!',
445                         ha='center', va='center', transform=axes[0, 0].transAxes
,
446                         fontsize=14, fontweight='bold', color='green')
447         axes[0, 0].set_title('关键字段缺失值统计')
448
449 # 2. 染色体异常检测率分析
450 chromosome_vars = ['T13_异常', 'T18_异常', 'T21_异常']
451 available_chroms = [var for var in chromosome_vars if var in df.columns]
452
453 if available_chroms:
454     chrom_names = [var.replace('_异常', '') for var in available_chroms]
455     abnormal_rates = [(df[var].sum() / len(df) * 100) for var in
available_chroms]
456
457     bars = axes[0, 1].bar(chrom_names, abnormal_rates, color=['red', 'orange',
'gold'], alpha=0.8)

```

```

458     axes[0, 1].set_title('女胎染色体异常检出率')
459     axes[0, 1].set_ylabel('异常检出率 (%)')
460     axes[0, 1].grid(True, alpha=0.3, axis='y')
461
462     # 添加百分比标签
463     for bar, rate in zip(bars, abnormal_rates):
464         axes[0, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height() +
465             0.1,
466                         f'{rate:.2f}%', ha='center', va='bottom', fontweight='
467 bold')
468
469 # 3. 样本完整性分析
470 complete_data = df.dropna()
471 total_samples = len(df)
472 complete_samples = len(complete_data)
473 incomplete_samples = total_samples - complete_samples
474
475 sizes = [complete_samples, incomplete_samples]
476 labels = [f'完整样本\n{complete_samples}例', f'不完整样本\n{incomplete_samples}
477 例']
478 colors = ['lightgreen', 'lightcoral']
479
480 wedges, texts, autotexts = axes[1, 0].pie(sizes, labels=labels, autopct='%1.1f
481 %%',
482                                             colors=colors, startangle=90)
483 axes[1, 0].set_title('女胎样本完整性分析')
484
485 # 4. 数值变量异常值分析 (以BMI为例)
486 if '孕妇BMI' in df.columns:
487     bmi_data = df['孕妇BMI'].dropna()
488     if len(bmi_data) > 0:
489         # 箱线图检测异常值
490         bp = axes[1, 1].boxplot(bmi_data, patch_artist=True)
491         bp['boxes'][0].set_facecolor('lightblue')
492         bp['boxes'][0].set_alpha(0.7)
493
494         # 计算异常值
495         Q1 = bmi_data.quantile(0.25)
496         Q3 = bmi_data.quantile(0.75)
497         IQR = Q3 - Q1
498         lower_bound = Q1 - 1.5 * IQR
499         upper_bound = Q3 + 1.5 * IQR
500         outliers = bmi_data[(bmi_data < lower_bound) | (bmi_data > upper_bound)
501 ]
502
503     axes[1, 1].set_title(f'女胎BMI异常值检测\n异常值: {len(outliers)} 个 ({
504 len(outliers)/len(bmi_data)*100:.1f}%)')

```

```

499         axes[1, 1].set_ylabel('孕妇BMI')
500         axes[1, 1].grid(True, alpha=0.3)
501     else:
502         axes[1, 1].text(0.5, 0.5, 'BMI数据\n不可用',
503                        ha='center', va='center', transform=axes[1, 1].transAxes,
504                        fontsize=14)
505
506     axes[1, 1].set_title('异常值检测')
507
508     plt.tight_layout()
509     plt.savefig('女胎数据预处理_数据质量分析.png', dpi=300, bbox_inches='tight',
510               facecolor='white')
511     plt.show()
512
513 def create_female_preliminary_correlation_plots(df):
514     # 选择数值变量进行相关性分析
515     numeric_vars = [
516         '年龄', '身高', '体重', '孕妇BMI',
517         '末次月经距离检测日天数', '检测时怀孕天数',
518         '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
519         'X染色体的Z值', '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC
520         含量',
521         'X染色体浓度', 'GC含量',
522         'IVF妊娠_编码', '胎儿健康_编码'
523     ]
524
525     available_vars = [var for var in numeric_vars if var in df.columns]
526
527     # 创建相关性数据
528     corr_data = df[available_vars].select_dtypes(include=[np.number]).dropna()
529
530     correlation_matrix = corr_data.corr()
531
532     fig, axes = plt.subplots(1, 2, figsize=(16, 6))
533     fig.suptitle('女胎数据初步相关性分析', fontsize=16, fontweight='bold')
534
535     # 1. 相关性热图
536     mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
537     sns.heatmap(correlation_matrix, mask=mask, annot=False, cmap='RdBu_r', center
538               =0,
539               square=True, fmt='.3f', cbar_kws={"shrink": .8}, ax=axes[0])
540     axes[0].set_title('变量间相关性热图')
541
542     # 2. 与胎儿健康编码的相关性
543     if '胎儿健康_编码' in correlation_matrix.columns:
544         health_correlations = correlation_matrix['胎儿健康_编码'].drop('胎儿健康_编
545         码')
546         health_correlations = health_correlations.sort_values(key=abs, ascending=

```

```

False)

541
542     colors = ['red' if x < 0 else 'blue' for x in health_correlations.values]
543     bars = axes[1].barh(range(len(health_correlations)), health_correlations.
values,
544                         color=colors, alpha=0.7)
545     axes[1].set_yticks(range(len(health_correlations)))
546     axes[1].set_yticklabels(health_correlations.index)
547     axes[1].set_xlabel('相关系数')
548     axes[1].set_title('与胎儿健康编码的相关性')
549     axes[1].axvline(x=0, color='black', linestyle='-', linewidth=1)
550     axes[1].grid(True, alpha=0.3, axis='x')
551
552     # 添加相关系数标签
553     for i, (bar, value) in enumerate(zip(bars, health_correlations.values)):
554         x_pos = value + (0.01 if value > 0 else -0.01)
555         ha_pos = 'left' if value > 0 else 'right'
556         axes[1].text(x_pos, i, f'{value:.3f}', va='center', ha=ha_pos,
557                     fontweight='bold', fontsize=9)
558     else:
559         # 显示整体相关性强度排序
560         var_correlations = {}
561         for var in correlation_matrix.columns:
562             other_correlations = correlation_matrix[var].drop(var)
563             avg_correlation = other_correlations.abs().mean()
564             var_correlations[var] = avg_correlation
565
566         sorted_vars = sorted(var_correlations.items(), key=lambda x: x[1], reverse=
True)
567
568         variables = [item[0] for item in sorted_vars]
569         correlations = [item[1] for item in sorted_vars]
570
571         bars = axes[1].barh(range(len(variables)), correlations,
572                             color='steelblue', alpha=0.7)
573         axes[1].set_yticks(range(len(variables)))
574         axes[1].set_yticklabels(variables)
575         axes[1].set_xlabel('平均相关性强度')
576         axes[1].set_title('变量相关性强度排序')
577         axes[1].grid(True, alpha=0.3, axis='x')
578
579         # 添加数值标签
580         for i, (bar, value) in enumerate(zip(bars, correlations)):
581             axes[1].text(bar.get_width() + 0.005, i, f'{value:.3f}',
582                         va='center', fontweight='bold', fontsize=9)
583
584     plt.tight_layout()

```



```

585     plt.savefig('女胎数据预处理_初步相关性分析.png', dpi=300, bbox_inches='tight',
586               facecolor='white')
587     plt.show()
588 def save_processed_data(processed_df, output_prefix="女胎数据_预处理后"):
589
590     csv_file = f"{output_prefix}.csv"
591     processed_df.to_csv(csv_file, index=False, encoding='utf-8-sig')
592     print(f"    CSV文件: {csv_file}")
593
594     excel_file = f"{output_prefix}.xlsx"
595     processed_df.to_excel(excel_file, index=False, engine='openpyxl')
596     print(f"    Excel文件: {excel_file}")
597
598     return csv_file, excel_file
599
600 def main():
601     excel_file = "附件.xlsx"
602
603     try:
604         processed_data = preprocess_female_data_simple(excel_file,
605               enable_visualization=True)
606         csv_file, excel_file_out = save_processed_data(processed_data)
607
608         print(f"\n数值特征统计:")
609         numeric_cols = processed_data.select_dtypes(include=[np.number]).columns
610         print(processed_data[numeric_cols].describe().round(2))
611
612         print(f"\n预处理总结:")
613         print(f"    输入文件: 附件.xlsx (女胎检测数据)")
614         print(f"    处理样本: {len(processed_data)} 个")
615         print(f"    生成特征: {processed_data.shape[1]} 个")
616         print(f"    输出文件: {csv_file}, {excel_file_out}")
617
618         print(f"\n生成的可视化图表:")
619         print(f"    • 女胎数据预处理_概览分析.png - 数据基本概况")
620         print(f"    • 女胎数据预处理_分类变量分布.png - 分类变量分布分析")
621         print(f"    • 女胎数据预处理_连续变量分布.png - 连续变量分布分析")
622         print(f"    • 女胎数据预处理_数据质量分析.png - 数据质量评估")
623         print(f"    • 女胎数据预处理_初步相关性分析.png - 变量相关性探索")
624
625         return processed_data
626
627     except FileNotFoundError:
628         print(f"错误: 找不到文件 '{excel_file}'")
629         print("    请确保Excel文件在当前目录下")
630     except Exception as e:

```

```

630     print(f"处理失败: {str(e)}")
631     import traceback
632     traceback.print_exc()
633
634 if __name__ == "__main__":
635     result = main()

```

问题 4.py

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.model_selection import train_test_split, cross_val_score,
    StratifiedKFold
4  from sklearn.preprocessing import StandardScaler, RobustScaler, MinMaxScaler,
    LabelEncoder
5  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier,
    ExtraTreesClassifier
6  from sklearn.linear_model import LogisticRegression, RidgeClassifier
7  from sklearn.svm import SVC
8  from sklearn.naive_bayes import GaussianNB
9  from sklearn.neighbors import KNeighborsClassifier
10 from sklearn.neural_network import MLPClassifier
11 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
12 from sklearn.tree import DecisionTreeClassifier
13 from sklearn.metrics import (classification_report, confusion_matrix, roc_auc_score
    ,
14                               f1_score, matthews_corrcoef, accuracy_score)
15 import warnings
16 warnings.filterwarnings('ignore')
17
18 class RobustModelComparison:
19     def __init__(self):
20         self.models = {}
21         self.results = []
22         self.feature_names = None
23         self.X_processed = None
24         self.y_processed = None
25
26     def load_and_clean_data(self, file_path):
27         self.df = pd.read_csv(file_path, encoding='utf-8')
28         for col in self.df.columns:
29             dtype = self.df[col].dtype
30             null_count = self.df[col].isnull().sum()
31             print(f" {col}: {dtype}, 缺失值: {null_count}")
32         # 目标变量分析
33         target_dist = self.df['胎儿健康_编码'].value_counts().sort_index()
34         print(f"\n目标变量分布:")
35         print(f" 健康(1): {target_dist.get(1, 0)} 例 ({target_dist.get(1, 0)/len(

```

```

self.df)*100:.1f}%")
36     print(f"   异常(0): {target_dist.get(0, 0)} 例 ({target_dist.get(0, 0)/len(
self.df)*100:.1f}%")
37
38     return self.df
39
40 def robust_feature_engineering(self):
41     # 选择数值型特征
42     numeric_features = [
43         # Z值特征（核心）
44         '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值', 'X染色体的Z值'
45     ,
46         # 基本信息
47         '年龄', '身高', '体重', '孕妇BMI', '生产次数',
48         # 妊娠相关
49         '检测抽血次数', '检测时怀孕天数', 'IVF妊娠_编码',
50         # 测序质量
51         '原始读段数', '在参考基因组上比对的比例', '重复读段的比例',
52         '唯一比对的读段数', 'GC含量', '被过滤掉读段数的比例',
53         # 染色体GC含量
54         '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC含量',
55         # X染色体浓度
56         'X染色体浓度'
57     ]
58
59     # 检查特征可用性并确保数值类型
60     available_features = []
61     for feature in numeric_features:
62         if feature in self.df.columns:
63             try:
64                 # 尝试转换为数值类型
65                 self.df[feature] = pd.to_numeric(self.df[feature], errors='
coerce')
66                 available_features.append(feature)
67             except:
68                 print(f"   跳过特征 {feature}: 无法转换为数值类型")
69
70     print(f"   可用数值特征: {len(available_features)} 个")
71
72     # 处理分类特征
73     categorical_features = []
74     if '怀孕次数' in self.df.columns:
75         # 处理怀孕次数（可能包含文本）
76         try:
77             # 提取数字部分
78             self.df['怀孕次数_数值'] = self.df['怀孕次数'].astype(str).str.
extract('(\\d+)').astype(float)

```

```

78         available_features.append('怀孕次数_数值')
79         if '怀孕次数' in available_features:
80             available_features.remove('怀孕次数')
81     except:
82         print(" 怀孕次数特征处理失败")
83
84     # 创建特征矩阵
85     X = self.df[available_features].copy()
86     y = self.df['胎儿健康_编码'].copy()
87
88     # 确保y是数值类型
89     y = pd.to_numeric(y, errors='coerce')
90
91     # 移除y中的缺失值对应的样本
92     valid_indices = ~y.isnull()
93     X = X[valid_indices]
94     y = y[valid_indices]
95
96     print(f" 清理后样本数: {len(X)}")
97
98     # 处理特征缺失值
99     missing_info = []
100    for col in X.columns:
101        missing_count = X[col].isnull().sum()
102        if missing_count > 0:
103            missing_info.append((col, missing_count, missing_count/len(X)*100))
104
105    if missing_info:
106        print(f" 存在缺失值的特征:")
107        for col, count, pct in missing_info:
108            print(f"      {col}: {count} 个 ({pct:.1f}%")
109
110    # 填充缺失值
111    for col in X.columns:
112        if X[col].isnull().sum() > 0:
113            if X[col].dtype in ['float64', 'int64']:
114                # 数值特征用中位数填充
115                median_val = X[col].median()
116                X[col].fillna(median_val, inplace=True)
117            else:
118                # 其他类型用众数填充
119                mode_val = X[col].mode()[0] if not X[col].mode().empty else 0
120                X[col].fillna(mode_val, inplace=True)
121
122    # 确保所有特征都是数值类型
123    for col in X.columns:
124        X[col] = pd.to_numeric(X[col], errors='coerce')

```

```

125         if X[col].isnull().sum() > 0:
126             X[col].fillna(X[col].median(), inplace=True)
127
128     # 创建工程特征
129     # Z值相关特征
130     chromosome_z_cols = ['13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值
131     '']
132
133     available_z_cols = [col for col in chromosome_z_cols if col in X.columns]
134
135     if available_z_cols:
136         # Z值绝对值
137         for col in available_z_cols:
138             X[f'{col}_abs'] = np.abs(X[col])
139
140         # Z值统计特征
141         z_abs_cols = [f'{col}_abs' for col in available_z_cols]
142         X['max_z_abs'] = X[z_abs_cols].max(axis=1)
143         X['mean_z_abs'] = X[z_abs_cols].mean(axis=1)
144         X['std_z_abs'] = X[z_abs_cols].std(axis=1)
145
146         # 异常Z值计数
147         X['z_abnormal_count_2.5'] = (X[z_abs_cols] >= 2.5).sum(axis=1)
148         X['z_abnormal_count_3.0'] = (X[z_abs_cols] >= 3.0).sum(axis=1)
149
150     # BMI相关特征
151     if '孕妇BMI' in X.columns:
152         # BMI分类
153         X['BMI_high'] = (X['孕妇BMI'] >= 35).astype(int)
154         X['BMI_normalized'] = (X['孕妇BMI'] - X['孕妇BMI'].mean()) / X['孕妇BMI
155         ''].std()
156
157     # BMI与Z值交互（如果Z值特征存在）
158     if available_z_cols:
159         for col in available_z_cols:
160             X[f'BMI_{col}_interaction'] = X['孕妇BMI'] * X[f'{col}_abs']
161
162     # 测序质量特征
163     quality_cols = ['在参考基因组上比对的比例', 'GC含量', '重复读段的比例']
164     available_quality_cols = [col for col in quality_cols if col in X.columns]
165
166     if len(available_quality_cols) >= 2:
167         # 质量综合评分
168         if '在参考基因组上比对的比例' in X.columns and 'GC含量' in X.columns:
169             X['sequencing_quality_score'] = (
170                 X['在参考基因组上比对的比例'] *
171                 (1 - np.abs(X['GC含量'] - 0.5))
172             )

```

```

170
171     # GC含量偏离度
172     if 'GC含量' in X.columns:
173         X['GC_deviation'] = np.abs(X['GC含量'] - 0.5)
174
175     # 最终数据验证
176     print(f" 最终特征数: {X.shape[1]}")
177     print(f" 最终样本数: {X.shape[0]}")
178
179     # 检查是否还有非数值数据
180     non_numeric = []
181     for col in X.columns:
182         if not np.issubdtype(X[col].dtype, np.number):
183             non_numeric.append(col)
184
185     if non_numeric:
186         print(f" 警告: 发现非数值列: {non_numeric}")
187         # 强制转换为数值
188         for col in non_numeric:
189             X[col] = pd.to_numeric(X[col], errors='coerce')
190             X[col].fillna(X[col].median(), inplace=True)
191
192     # 检查无穷大和NaN
193     inf_cols = []
194     for col in X.columns:
195         if np.isinf(X[col]).any() or np.isnan(X[col]).any():
196             inf_cols.append(col)
197
198     if inf_cols:
199         print(f" 处理无穷大/NaN值: {inf_cols}")
200         for col in inf_cols:
201             X[col] = X[col].replace([np.inf, -np.inf], np.nan)
202             X[col].fillna(X[col].median(), inplace=True)
203
204     self.feature_names = X.columns.tolist()
205     self.X_processed = X
206     self.y_processed = y
207
208     return X, y
209
210 def setup_robust_models(self):
211     models_config = {
212         'RandomForest': RandomForestClassifier(
213             n_estimators=100,
214             max_depth=10,
215             min_samples_split=10,
216             min_samples_leaf=5,

```

```

217         class_weight='balanced',
218         random_state=42,
219         n_jobs=1 # 避免并行问题
220     ),
221
222     'GradientBoosting': GradientBoostingClassifier(
223         n_estimators=100,
224         learning_rate=0.1,
225         max_depth=6,
226         min_samples_split=20,
227         min_samples_leaf=10,
228         random_state=42
229     ),
230
231     'LogisticRegression': LogisticRegression(
232         class_weight='balanced',
233         random_state=42,
234         max_iter=1000,
235         solver='liblinear'
236     ),
237
238     'SVM_RBF': SVC(
239         kernel='rbf',
240         class_weight='balanced',
241         probability=True,
242         random_state=42,
243         C=1.0,
244         gamma='scale'
245     ),
246
247     'DecisionTree': DecisionTreeClassifier(
248         max_depth=8,
249         min_samples_split=20,
250         min_samples_leaf=10,
251         class_weight='balanced',
252         random_state=42
253     ),
254
255     'KNeighbors': KNeighborsClassifier(
256         n_neighbors=5,
257         weights='uniform'
258     ),
259
260     'GaussianNB': GaussianNB(),
261
262     'LinearDA': LinearDiscriminantAnalysis(),
263

```

```

264         'MLPClassifier': MLPClassifier(
265             hidden_layer_sizes=(50, 25),
266             max_iter=300,
267             random_state=42,
268             early_stopping=True,
269             validation_fraction=0.1
270         )
271     }
272
273     print(f"    配置了 {len(models_config)} 个模型")
274     return models_config
275
276 def evaluate_models_robust(self, X, y):
277     # 数据标准化方法
278     scalers_config = {
279         'RobustScaler': RobustScaler(),
280         'StandardScaler': StandardScaler(),
281         'MinMaxScaler': MinMaxScaler()
282     }
283     # 获取模型配置
284     models_config = self.setup_robust_models()
285     # 数据分割
286     X_train, X_test, y_train, y_test = train_test_split(
287         X, y, test_size=0.2, random_state=42, stratify=y
288     )
289     print(f"    训练集: {X_train.shape[0]} 样本")
290     print(f"    测试集: {X_test.shape[0]} 样本")
291     # 交叉验证设置
292     cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
293     print(f"\n模型性能评估:")
294     print("-" * 100)
295     print(f"{'模型名称':<15} {'标准化':<12} {'CV均值':<8} {'CV标准差':<8} {'测试AUC':<8} {'敏感性':<8} {'特异性':<8} {'F1得分':<8}")
296     print("-" * 100)
297
298     # 评估每个模型
299     for model_name, model in models_config.items():
300         best_result = None
301         best_auc = 0
302
303         for scaler_name, scaler in scalers_config.items():
304             try:
305                 # 数据标准化
306                 X_train_scaled = scaler.fit_transform(X_train.astype(float))
307                 X_test_scaled = scaler.transform(X_test.astype(float))
308
309                 # 确保没有NaN或无穷大

```



```

310         if np.any(np.isnan(X_train_scaled)) or np.any(np.isinf(
X_train_scaled)):
311             print(f"    {model_name} + {scaler_name}: 数据标准化后存在
NaN/Inf")
312             continue
313
314         # 交叉验证
315         try:
316             cv_scores = cross_val_score(
317                 model, X_train_scaled, y_train,
318                 cv=cv, scoring='roc_auc', n_jobs=1
319             )
320             cv_mean = cv_scores.mean()
321             cv_std = cv_scores.std()
322         except Exception as e:
323             print(f"    {model_name} + {scaler_name}: 交叉验证失败 - {
str(e)[:50]}")
324             continue
325
326         # 训练模型
327         try:
328             model.fit(X_train_scaled, y_train)
329         except Exception as e:
330             print(f"    {model_name} + {scaler_name}: 训练失败 - {str(e
)[:50]}")
331             continue
332
333         # 测试集预测
334         try:
335             y_pred = model.predict(X_test_scaled)
336
337             if hasattr(model, 'predict_proba'):
338                 y_pred_proba = model.predict_proba(X_test_scaled)[: , 1]
339                 test_auc = roc_auc_score(y_test, y_pred_proba)
340             else:
341                 test_auc = roc_auc_score(y_test, y_pred)
342         except Exception as e:
343             print(f"    {model_name} + {scaler_name}: 预测失败 - {str(e
)[:50]}")
344             continue
345
346         # 计算其他指标
347         try:
348             cm = confusion_matrix(y_test, y_pred)
349             if cm.shape == (2, 2):
350                 tn, fp, fn, tp = cm.ravel()
351                 sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0

```

```

352         specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
353     else:
354         sensitivity = specificity = 0
355
356         f1 = f1_score(y_test, y_pred, average='weighted')
357     except:
358         sensitivity = specificity = f1 = 0
359
360     # 记录结果
361     result = {
362         'model_name': model_name,
363         'scaler_name': scaler_name,
364         'cv_mean': cv_mean,
365         'cv_std': cv_std,
366         'test_auc': test_auc,
367         'sensitivity': sensitivity,
368         'specificity': specificity,
369         'f1_score': f1,
370         'model': model,
371         'scaler': scaler
372     }
373
374     self.results.append(result)
375
376     # 更新最佳结果
377     if test_auc > best_auc:
378         best_auc = test_auc
379         best_result = result
380
381     except Exception as e:
382         print(f"    {model_name} + {scaler_name}: 整体失败 - {str(e)
383               [:50]}")
384         continue
385
386     # 显示最佳结果
387     if best_result:
388         r = best_result
389         print(f"{r['model_name']:<15} {r['scaler_name']:<12} {r['cv_mean']:<8.4f} {r['cv_std']:<8.4f} {r['test_auc']:<8.4f} {r['sensitivity']:<8.4f} {r['specificity']:<8.4f} {r['f1_score']:<8.4f}")
390     else:
391         print(f"{model_name:<15} {'所有组合':<12} {'失败':<8} {'失败':<8} {'失败':<8} {'失败':<8} {'失败':<8} {'失败':<8} {'失败':<8}")
392
393     print("-" * 100)
394
395     return X_train, X_test, y_train, y_test

```

```

395
396 def analyze_results(self):
397     if not self.results:
398         print(" 没有成功的模型结果")
399         return None
400
401     # 按AUC排序
402     sorted_results = sorted(self.results, key=lambda x: x['test_auc'], reverse=
True)
403
404     print(f"\n成功训练的模型排名:")
405     print("-" * 80)
406     print(f"{'排名':<4} {'模型+标准化':<25} {'AUC':<8} {'敏感性':<8} {'特异性
':<8} {'F1得分':<8}")
407     print("-" * 80)
408
409     for i, result in enumerate(sorted_results[:10]):
410         model_combo = f"{result['model_name']}+{result['scaler_name']}"
411         print(f"{i+1:<4} {model_combo:<25} {result['test_auc']:<8.4f} {result['
sensitivity']:<8.4f} {result['specificity']:<8.4f} {result['f1_score']:<8.4f}")
412
413     print("-" * 80)
414
415     # 最佳模型分析
416     if sorted_results:
417         best_model = sorted_results[0]
418         print(f"\n最佳模型详情:")
419         print(f" 模型: {best_model['model_name']}")
420         print(f" 标准化: {best_model['scaler_name']}")
421         print(f" 测试AUC: {best_model['test_auc']:.4f}")
422         print(f" 交叉验证: {best_model['cv_mean']:.4f} ± {best_model['cv_std
']:.4f}")
423         print(f" 敏感性: {best_model['sensitivity']:.4f}")
424         print(f" 特异性: {best_model['specificity']:.4f}")
425         print(f" F1得分: {best_model['f1_score']:.4f}")
426
427     # 性能评级
428     auc = best_model['test_auc']
429     if auc >= 0.90:
430         grade = "A级 - 优秀"
431     elif auc >= 0.85:
432         grade = "B级 - 良好"
433     elif auc >= 0.80:
434         grade = "C级 - 可接受"
435     else:
436         grade = "D级 - 需改进"
437

```

```

438         print(f"    性能等级: {grade}")
439
440     # 模型成功率统计
441     model_success = {}
442     for result in self.results:
443         model_name = result['model_name']
444         if model_name not in model_success:
445             model_success[model_name] = 0
446             model_success[model_name] += 1
447
448     print(f"\n模型成功率统计:")
449     for model_name, success_count in sorted(model_success.items(), key=lambda x
: x[1], reverse=True):
450         print(f"    {model_name:<15}: {success_count}/3 种标准化方法成功")
451
452     return sorted_results
453
454     def save_results_to_csv(self, sorted_results):
455         if not sorted_results:
456             print("    没有结果可保存")
457             return
458
459         # 准备数据
460         results_data = []
461         for i, result in enumerate(sorted_results):
462             results_data.append({
463                 '排名': i + 1,
464                 '模型名称': result['model_name'],
465                 '标准化方法': result['scaler_name'],
466                 'CV均值': round(result['cv_mean'], 4),
467                 'CV标准差': round(result['cv_std'], 4),
468                 '测试AUC': round(result['test_auc'], 4),
469                 '敏感性': round(result['sensitivity'], 4),
470                 '特异性': round(result['specificity'], 4),
471                 'F1得分': round(result['f1_score'], 4)
472             })
473
474         # 创建DataFrame并保存
475         results_df = pd.DataFrame(results_data)
476
477         # 保存主要结果
478         results_df.to_csv('女胎异常判定_模型对比结果.csv', index=False, encoding='
utf-8-sig')
479         print(f"    主要结果已保存: 女胎异常判定_模型对比结果.csv")
480
481         # 保存Top 10结果
482         top10_df = results_df.head(10)

```

```

483     top10_df.to_csv('女胎异常判定_Top10模型.csv', index=False, encoding='utf-8-
sig')
484     print(f"   Top 10结果已保存：女胎异常判定_Top10模型.csv")
485
486     # 创建详细分析报告
487     if sorted_results:
488         best_model = sorted_results[0]
489
490         # 性能分析数据
491         analysis_data = {
492             '分析项目': [
493                 '最佳模型', '最佳标准化', '最佳AUC', '最佳敏感性', '最佳特异性'
494                 ,
495                 '最佳F1得分', '交叉验证均值', '交叉验证标准差', '成功模型数', '
总测试配置'
496             ],
497             '结果': [
498                 best_model['model_name'],
499                 best_model['scaler_name'],
500                 round(best_model['test_auc'], 4),
501                 round(best_model['sensitivity'], 4),
502                 round(best_model['specificity'], 4),
503                 round(best_model['f1_score'], 4),
504                 round(best_model['cv_mean'], 4),
505                 round(best_model['cv_std'], 4),
506                 len(sorted_results),
507                 len(sorted_results)
508             ]
509         }
510
511         analysis_df = pd.DataFrame(analysis_data)
512         analysis_df.to_csv('女胎异常判定_分析摘要.csv', index=False, encoding='
utf-8-sig')
513         print(f"   分析摘要已保存：女胎异常判定_分析摘要.csv")
514
515         # 创建模型成功率统计
516         model_success = {}
517         for result in sorted_results:
518             model_name = result['model_name']
519             if model_name not in model_success:
520                 model_success[model_name] = []
521             model_success[model_name].append(result['test_auc'])
522
523         success_data = []
524         for model_name, auks in model_success.items():
525             success_data.append({
526                 '模型名称': model_name,

```

```

526         '成功配置数': len(aucs),
527         '平均AUC': round(np.mean(aucs), 4),
528         '最佳AUC': round(max(aucs), 4),
529         '最差AUC': round(min(aucs), 4),
530         'AUC标准差': round(np.std(aucs), 4)
531     })
532
533     success_df = pd.DataFrame(success_data)
534     success_df = success_df.sort_values('最佳AUC', ascending=False)
535     success_df.to_csv('女胎异常判定_模型成功率统计.csv', index=False, encoding=
'utf-8-sig')
536     print(f" 模型统计已保存: 女胎异常判定_模型成功率统计.csv")
537
538     return results_df
539
540 def generate_recommendations(self, sorted_results):
541
542     if not sorted_results:
543         print(" 无可用模型, 建议:")
544         print(" 1. 检查数据质量")
545         print(" 2. 简化特征工程")
546         print(" 3. 增加样本数量")
547         return
548
549     best_model = sorted_results[0]
550
551     print(f"基于最佳模型的建议:")
552     print(f" 推荐模型: {best_model['model_name']}")
553     print(f" 推荐标准化: {best_model['scaler_name']}")
554
555     # 临床应用建议
556     auc = best_model['test_auc']
557     sensitivity = best_model['sensitivity']
558     specificity = best_model['specificity']
559
560     print(f"\n临床应用指导:")
561
562     if auc >= 0.85 and sensitivity >= 0.80 and specificity >= 0.80:
563         print(f" 模型性能良好, 可用于临床辅助诊断")
564         print(f" 建议在医生指导下使用")
565     elif auc >= 0.80:
566         print(f" 模型性能可接受, 建议谨慎使用")
567         print(f" 需要结合其他临床指标")
568     else:
569         print(f" 模型性能不足, 不建议直接临床使用")
570         print(f" 需要进一步优化或收集更多数据")
571

```

```

572 # 改进建议
573 print(f"\n模型改进建议:")
574 if len(sorted_results) < 5:
575     print(f"    • 数据质量问题较多, 建议清理原始数据")
576 if sensitivity < 0.80:
577     print(f"    • 敏感性偏低, 建议调整分类阈值")
578 if specificity < 0.80:
579     print(f"    • 特异性偏低, 建议增加特征或样本")
580
581 print(f"    • 建议收集更多样本进行验证")
582 print(f"    • 考虑使用更复杂的特征工程")
583 print(f"    • 可尝试深度学习方法")
584
585 # 生成建议报告CSV
586 recommendations = []
587
588 # 性能等级
589 if auc >= 0.90:
590     grade = "A级 - 优秀"
591     clinical_use = "可用于临床辅助诊断"
592 elif auc >= 0.85:
593     grade = "B级 - 良好"
594     clinical_use = "可用于临床筛查, 建议结合其他指标"
595 elif auc >= 0.80:
596     grade = "C级 - 可接受"
597     clinical_use = "需要进一步优化后谨慎使用"
598 else:
599     grade = "D级 - 需改进"
600     clinical_use = "不建议直接用于临床应用"
601
602 recommendations.append({
603     '建议类型': '性能等级',
604     '内容': grade,
605     '详细说明': f"基于AUC={auc:.4f}的评级"
606 })
607
608 recommendations.append({
609     '建议类型': '临床应用',
610     '内容': clinical_use,
611     '详细说明': f"考虑敏感性={sensitivity:.4f}, 特异性={specificity:.4f}"
612 })
613
614 recommendations.append({
615     '建议类型': '推荐配置',
616     '内容': f"{best_model['model_name']} + {best_model['scaler_name']}",
617     '详细说明': f"该配置获得最佳AUC={best_model['test_auc']:.4f}"
618 })

```

```

619
620     if sensitivity < 0.80:
621         recommendations.append({
622             '建议类型': '模型改进',
623             '内容': '提高敏感性',
624             '详细说明': '当前敏感性偏低, 建议调整分类阈值或增加训练样本'
625         })
626
627     if specificity < 0.80:
628         recommendations.append({
629             '建议类型': '模型改进',
630             '内容': '提高特异性',
631             '详细说明': '当前特异性偏低, 建议优化特征工程或模型参数'
632         })
633
634     rec_df = pd.DataFrame(recommendations)
635     rec_df.to_csv('女胎异常判定_临床建议.csv', index=False, encoding='utf-8-sig
636 ')
637     print(f"  临床建议已保存: 女胎异常判定_临床建议.csv")
638
639 def run_robust_analysis():
640     # 初始化分析器
641     analyzer = RobustModelComparison()
642
643     # 加载和清理数据
644     df = analyzer.load_and_clean_data("女胎数据_预处理后.csv")
645
646     # 特征工程
647     X, y = analyzer.robust_feature_engineering()
648
649     # 模型评估
650     X_train, X_test, y_train, y_test = analyzer.evaluate_models_robust(X, y)
651
652     # 结果分析
653     sorted_results = analyzer.analyze_results()
654
655     # 生成建议
656     analyzer.generate_recommendations(sorted_results)
657
658     # 保存结果到CSV
659     if sorted_results:
660         results_df = analyzer.save_results_to_csv(sorted_results)
661
662     if sorted_results:
663         print(f"成功训练 {len(analyzer.results)} 个模型配置")
664         print(f"最佳模型AUC: {sorted_results[0]['test_auc']:.4f}")

```



```

665         print(f"\n生成的CSV文件:")
666         print(f" 1. 女胎异常判定_模型对比结果.csv - 完整对比结果")
667         print(f" 2. 女胎异常判定_Top10模型.csv - 前10名模型")
668         print(f" 3. 女胎异常判定_分析摘要.csv - 分析摘要")
669         print(f" 4. 女胎异常判定_模型成功率统计.csv - 模型统计")
670         print(f" 5. 女胎异常判定_临床建议.csv - 临床应用建议")
671     else:
672         print(f"所有模型训练都失败了，请检查数据质量")
673     print(f"*80")
674
675     return analyzer, sorted_results
676
677
678
679 def save_feature_importance(analyzer, sorted_results, output_file='女胎异常判定_特
    征重要性.csv'):
680
681     if not sorted_results or not analyzer.results:
682         print("没有可用的结果来提取特征重要性")
683         return
684
685     best_model_result = sorted_results[0]
686     best_model = best_model_result['model']
687
688     # 检查是否有特征重要性
689     if hasattr(best_model, 'feature_importances_'):
690
691         importance_data = []
692         feature_names = analyzer.feature_names
693         importances = best_model.feature_importances_
694
695         for i, (feature, importance) in enumerate(zip(feature_names, importances)):
696             importance_data.append({
697                 '排名': i + 1,
698                 '特征名称': feature,
699                 '重要性得分': round(importance, 6),
700                 '重要性百分比': round(importance * 100, 2)
701             })
702
703     # 按重要性排序
704     importance_df = pd.DataFrame(importance_data)
705     importance_df = importance_df.sort_values('重要性得分', ascending=False)
706     importance_df['排名'] = range(1, len(importance_df) + 1)
707
708     importance_df.to_csv(output_file, index=False, encoding='utf-8-sig')
709     print(f"特征重要性已保存: {output_file}")
710

```

```

711         print(f"\nTop 10 重要特征:")
712         for _, row in importance_df.head(10).iterrows():
713             print(f"    {row['排名']:2d}. {row['特征名称']:<30}: {row['重要性得分']:.6f} ({row['重要性百分比']:.2f}%)")
714
715         return importance_df
716     else:
717         print(f"最佳模型 ({best_model_result['model_name']}) 不支持特征重要性分析")
718         return None
719
720
721 def generate_summary_report(analyzer, sorted_results, output_file='女胎异常判定_完整报告.csv'):
722
723     summary_data = []
724
725     best_model = sorted_results[0]
726     total_configs = len(analyzer.results)
727
728     summary_data.extend([
729         {'类别': '基本信息', '项目': '数据文件', '值': '女胎数据_预处理后.csv'},
730         {'类别': '基本信息', '项目': '样本总数', '值': len(analyzer.X_processed)},
731         {'类别': '基本信息', '项目': '特征总数', '值': len(analyzer.feature_names)
732     ]),
733     {'类别': '基本信息', '项目': '测试配置数', '值': total_configs},
734     {'类别': '基本信息', '项目': '成功配置数', '值': len(sorted_results)},
735
736     {'类别': '最佳模型', '项目': '模型名称', '值': best_model['model_name']},
737     {'类别': '最佳模型', '项目': '标准化方法', '值': best_model['scaler_name']
738 ]),
739     {'类别': '最佳模型', '项目': '测试AUC', '值': round(best_model['test_auc'],
740 4)},
741     {'类别': '最佳模型', '项目': '交叉验证均值', '值': round(best_model['cv_mean'], 4)},
742     {'类别': '最佳模型', '项目': '交叉验证标准差', '值': round(best_model['cv_std'], 4)},
743     {'类别': '最佳模型', '项目': '敏感性', '值': round(best_model['sensitivity'], 4)},
744     {'类别': '最佳模型', '项目': '特异性', '值': round(best_model['specificity'], 4)},
745     {'类别': '最佳模型', '项目': 'F1得分', '值': round(best_model['f1_score'], 4)},
746 ]),
747
748     # 性能分级
749     auc = best_model['test_auc']
750     if auc >= 0.90:

```

```

748     grade = "A级 - 优秀"
749 elif auc >= 0.85:
750     grade = "B级 - 良好"
751 elif auc >= 0.80:
752     grade = "C级 - 可接受"
753 else:
754     grade = "D级 - 需改进"
755
756 summary_data.append({'类别': '性能评级', '项目': '等级', '值': grade})
757
758 # 模型统计
759 model_stats = {}
760 for result in sorted_results:
761     model_name = result['model_name']
762     if model_name not in model_stats:
763         model_stats[model_name] = []
764     model_stats[model_name].append(result['test_auc'])
765
766 for model_name, aucs in model_stats.items():
767     summary_data.extend([
768         {'类别': f'模型统计_{model_name}', '项目': '成功配置数', '值': len(aucs
769     )},
770         {'类别': f'模型统计_{model_name}', '项目': '平均AUC', '值': round(np.
771     mean(aucs), 4)},
772         {'类别': f'模型统计_{model_name}', '项目': '最佳AUC', '值': round(max(
773     aucs), 4)},
774     ])
775
776 # 创建DataFrame并保存
777 summary_df = pd.DataFrame(summary_data)
778 summary_df.to_csv(output_file, index=False, encoding='utf-8-sig')
779 print(f"完整报告已保存: {output_file}")
780
781 return summary_df
782
783 if __name__ == "__main__":
784     # 运行稳健分析
785     analyzer, results = run_robust_analysis()
786
787     if results and analyzer:
788         # 保存特征重要性
789         save_feature_importance(analyzer, results)
790
791         # 生成完整报告
792         generate_summary_report(analyzer, results)
793

```

792

```
print(f"\n所有结果已保存完毕！")
```